ARMENU

TIZIM ARXITEKTURASI

III — Xavfsizlik va operatsiya

Modulli monolit · Multi-tenant · 1 mln foydalanuvchiga tayyor
Django 5 · PostgreSQL+pgvector · Redis · 8 AI provayder (MCP)

Qismlar: 9 ta
Hajm: 19 594 qator · 900 KB
ADR: 15 ta arxitektura qarori
Kod namunalari: 944 blok
Jadvallar: 2 969 qator
Versiya: 1.0 · 2026-09-04

✔ **Kim uchun:** tizim arxitektori · texnik direktor · investor (texnik due diligence) · dasturchi.
Har qaror sababi bilan yozilgan: nega shunday, muqobili nima edi, qachon qayta ko'riladi.

MUNDARIJA — III — Xavfsizlik va operatsiya

6. **Xavfsizlik**

   STRIDE tahdid modeli, multi-tenant izolyatsiya 4 qatlam, 4 aktor autentifikatsiyasi, rol matritsasi, to'lov xavfsizligi (PCI), rate limiting, audit, O'zbekiston qonunchiligi

7. **Integratsiyalar**

   Adapter naqshi, MCP qatlami 8 provayder, to'lov 3 protokol, fiskal OFD+IKPU, Telegram, CDN CORS, 19 xizmat SLA jadvali, kontrakt testlari

8. **Deploy va operatsiya**

   3 muhit, infratuzilma S0/S3/S5, izolyatsiya kafolati, systemd socket activation, nginx, CI/CD 6 darvoza, monitoring 40+ metrika va 28 alert, backup+verify, 10 runbook, on-call

QISM 6

Xavfsizlik

Versiya: 1.0 · **Sana:** 2026-09-08
Kim uchun: xavfsizlik muhandisi · backend dasturchi · texnik direktor · auditor
Asos: `TZ_MASTER/03_ARXITEKTURA.md` §8, `ARXITEKTURA/05_ISHONCHLILIK.md`

0. XAVFSIZLIK FALSAFASI

> **Eng katta xavf — ma'lumot sizishi emas, MULTI-TENANT sizishi.**
> Restoran A o'z raqobatchisi restoran B ning buyurtmalarini, narxlarini, mijozlarini ko'rsa — bu bitta bug emas, biznesning tugashi.



3 ta o'zgarmas qoida:

#	Qoida	Buzilsa
1	Har DB so'rovi `restaurant_id` bilan chegaralanadi	🔴 Kompaniya tugaydi
2	Narx faqat serverda hisoblanadi	🔴 Moliyaviy zarar
3	Karta ma'lumoti bizga hech qachon kelmaydi	🔴 Jinoiy javobgarlik

1. TAHDID MODELI (STRIDE)

1.1 Aktorlar va ishonch chegaralari



1.2 STRIDE tahlili

🎭 S — Spoofing (o'zini boshqa qilib ko'rsatish)

#	Ssenariy	Ta'sir	🛡 Himoya
S1	Hujumchi QR token'ni taxmin qiladi (`/t/abc123`)	Boshqa stol nomidan buyurtma	Token = 22-belgili `secrets.token_urlsafe(16)` (128 bit); rate limit 60/daq/IP; token restoran+stolga bog'langan
S2	Ofitsiant PIN'ini brute-force	Xodim nomidan amal	PIN 6 raqam + bcrypt(cost=12); 5 xato → 15 daq blok (IP+PIN); qurilma fingerprint
S3	JWT token o'g'irlanishi (XSS orqali)	Sessiya egallanishi	`httpOnly` cookie (JS o'qiy olmaydi) + CSP + qisqa TTL (15 daq) + refresh rotatsiyasi
S4	Soxta to'lov webhook (`POST /webhook/payme/`)	Bepul buyurtma	HMAC/Basic imzo tekshiruvi + IP allowlist + provayderdan `check_transaction` tasdiqlash
S5	Boshqa restoran admin'i sifatida kirish	To'liq egallash	Parol + 2FA (platforma admin uchun majburiy) + qurilma tanish
S6	Telegram bot `chat_id` soxtalashtirish	Boshqa restoran bildirishnomasi	Webhook `secret_token` + `chat_id` DB'da bog'langan, payload'dan olinmaydi
S7	Device cookie klonlash (mijoz sessiyasi)	Boshqa mijozning savati	Cookie = imzolangan (`itsdangerous`), IP+UA fingerprint bilan bog'langan; buyurtmadan keyin ahamiyatsiz

✏ T — Tampering (ma'lumotni o'zgartirish)

#	Ssenariy	Ta'sir	🛡 Himoya
T1	🔴 **Klient narxni o'zgartiradi** (`{"price": 1}`)	Moliyaviy zarar	⚡ **Narx serializer'da** `read_only`; server `Dish.price` dan oladi; klient faqat `dish_id` + `qty` yuboradi
T2	`restaurant_id` ni body'da yuborish	Cross-tenant yozuv	`restaurant_id` **hech qachon** request body'dan olinmaydi — faqat middleware'dan
T3	Buyurtma statusini o'zgartirish (`status: "paid"`)	To'lovsiz taom	Status faqat state-machine orqali; `status` field serializer'da `read_only`
T4	Yuklangan 3D faylga zararli kod	XSS / RCE	MIME + magic bytes + `glTF-validator`; alohida CDN domendan beriladi; `Content-Disposition: attachment`
T5	Chegirma/promokod manipulyatsiyasi	Moliyaviy zarar	Promokod serverda tekshiriladi (mavjudlik, muddat, limit, tenant); klient faqat kod matnini yuboradi
T6	URL parametrida `id` o'zgartirish (IDOR)	Boshqa buyurtmani ko'rish	Object-level permission + tenant scope + `owner_session` tekshiruvi
T7	WS xabar ichida `restaurant_id` almashtirish	Boshqa restoran KDS'iga xabar	Group nomi **serverda** `scope['restaurant_id']` dan quriladi, xabardan emas
T8	Fiskal chek summasini o'zgartirish	Soliq buzilishi	Chek `Order` dan generatsiya qilinadi; `receipt_hash` saqlanadi; OFD javobi solishtiriladi

🙈 R — Repudiation (rad etish)

#	Ssenariy	Ta'sir	🛡 Himoya
R1	Restoran "biz bu buyurtmani olmadik" deydi	Nizo	`AuditLog`: kim, nima, qachon, IP, UA; WS yetkazish tasdiqlanishi
R2	Xodim "men bekor qilmadim" deydi	Ichki o'g'irlik	Har amal `actor_type` + `actor_id` + `pin_session_id` bilan yoziladi
R3	Mijoz "men to'lamadim" deydi	Chargeback	Provayder `txn_id` + fiskal chek + `AuditLog`
R4	Admin log'ni o'chiradi	Iz yo'qoladi	AuditLog **faqat qo'shiladi** (append-only), `DELETE` huquqi hech kimda yo'q; kunlik WORM S3 eksport

🕵 I — Information Disclosure (ma'lumot sizishi)

#	Ssenariy	Ta'sir	🛡 Himoya
I1	🔴 **Cross-tenant sizish** (filtr unutildi)	Kompaniya tugaydi	⚡ 4 qatlamli himoya — §2
I2	Xato sahifasida stack trace	Ichki tuzilma oshkor	`DEBUG=False` ; maxsus 500 sahifa; Sentry'da to'liq, foydalanuvchida `request_id`
I3	API javobida ortiqcha maydon (`cost_price`)	Raqobat sirlari	Serializer'da **oq ro'yxat** (`fields = [...]`), hech qachon `__all__`
I4	Menyu scraping (raqobatchi narxlarni yig'adi)	Biznes zarari	Rate limit + bot aniqlash (Cloudflare) + `robots.txt` + anomal so'rov alerti
I5	S3 bucket ochiq	Barcha 3D/rasm	Bucket private; presigned URL (1 soat); CDN signed URL
I6	Log'da telefon/karta raqami	PII sizishi	⚡ Log filtri — §5.4 maskalash
I7	Timing attack (PIN tekshiruvi)	PIN taxmini	`secrets.compare_digest` + bcrypt (o'zi doimiy vaqt emas, lekin rate limit qoplaydi)
I8	`.git` , `.env` veb orqali ochiq	To'liq egallash	nginx `location ~ /\.` → 404; deploy'da faqat kerakli fayllar
I9	Xato xabari ma'lumot beradi ("bunday PIN yo'q")	Foydalanuvchi ro'yxati	Umumiy xabar: "PIN yoki xodim topilmadi"

⌀ D — Denial of Service

#	Ssenariy	Ta'sir	🛡 Himoya
D1	Volumetric DDoS	Xizmat o'chadi	Cloudflare (L3/L4/L7), Under Attack rejimi
D2	Buyurtma spam (1 sessiya, 1000 buyurtma)	DB to'lishi, oshxona bloklanishi	Rate limit 10/daq/sessiya + stol bo'yicha 30/soat + "shubhali" flag
D3	Katta fayl yuklash	Disk to'lishi	`client_max_body_size 50M` + `MAX_UPLOAD_SIZE` + kvota (restoran bo'yicha)
D4	Og'ir so'rov (`?limit=1000000`)	DB bloklanishi	Pagination majburiy (maks 100) + `statement_timeout=10s`
D5	WS ulanish bombardimoni	Daphne to'lishi	Ulanish limiti (IP bo'yicha 10, sessiya bo'yicha 3) + auth majburiy
D6	AI so'rov spam (xarajat hujumi)	💸 Pul yo'qotish	Rate limit 10/5daq + kunlik xarajat cheki + CB
D7	Regex/ReDoS qidiruvda	CPU 100%	Qidiruv PostgreSQL FTS orqali, regex klientdan qabul qilinmaydi
D8	Zip bomb (3D arxiv)	Disk/RAM	Ochilgan hajm cheki (100 MB), fayl soni cheki

🕷 E — Elevation of Privilege

#	Ssenariy	Ta'sir	🛡 Himoya
E1	Ofitsiant admin API'ga kiradi	Menyu/narx o'zgartirish	Permission klass har view'da; rol JWT ichida, serverda tekshiriladi
E2	Restoran admin platforma admin bo'ladi	Barcha tenantlar	`is_platform_admin` alohida model+2FA; hech qachon API orqali o'zgarmaydi
E3	Mass assignment (`{"role": "admin"}`)	Huquq oshishi	Serializer oq ro'yxati; `role` , `restaurant` , `is_active` — `read_only`
E4	SQL injection → `SET ROLE`	To'liq DB	ORM; `raw()` taqiqlangan (CI linter); parametrlangan so'rov
E5	Deserializatsiya (pickle)	RCE	`pickle` taqiqlangan; navbat payload = JSON + sxema
E6	Bog'liqlik (supply chain)	RCE	`pip-audit` CI'da; `requirements.txt` hash-pinned; Dependabot
E7	Django admin ochiq	To'liq egallash	`/admin/` — IP allowlist + 2FA + boshqa port; production'da o'chirilgan

---

2. ⚡ MULTI-TENANT IZOLYATSIYA — ENG MUHIM BO'LIM

> **Bu bo'lim ARMENU xavfsizligining yuragi.** Boshqa hamma narsa buzilsa tuzatish mumkin. Tenant sizishi — mijoz ishonchi qaytmaydi.

2.1 4 qatlamli himoya



2.2 Qatlam 1 — Middleware

```
# core/middleware/tenant.py
import threading
from django.http import JsonResponse
from django.core.cache import cache
from apps.tenants.models import Restaurant

_local = threading.local()


def current_restaurant_id():
    """Har joydan (signal, servis, worker) joriy tenant."""
    rid = getattr(_local, 'restaurant_id', None)
    if rid is None:
        raise TenantNotSet("restaurant_id o'rnatilmagan — xavfsizlik xatosi")
    return rid


class TenantMiddleware:
    """
    ⚡ restaurant_id ni ANIQLAYDI. Uch manbadan, prioritet tartibida:
      1. JWT claim  (xodim/admin)        — eng ishonchli
      2. QR token   (mijoz)              — DB'dan qidiriladi
      3. Subdomain  (<slug>.armenu.uz)   — DB'dan qidiriladi

    ● HECH QACHON: request.POST['restaurant_id'], GET['rid'], header
    """
    EXEMPT_PREFIXES = ('/healthz', '/metrics', '/webhook/', '/platform/', '/static/')

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        _local.restaurant_id = None
        request.restaurant_id = None
        request.tenant_source = None

        if not any(request.path.startswith(p) for p in self.EXEMPT_PREFIXES):
            rid, source = self._resolve(request)
            if rid is None:
                return JsonResponse(
                    {'error': 'restaurant_not_resolved',
                     'detail': "Restoran aniqlanmadi"}, status=400)
            request.restaurant_id = rid
            request.tenant_source = source
            _local.restaurant_id = rid

        try:
            response = self.get_response(request)
        finally:
            _local.restaurant_id = None          # ⚡ thread reuse xavfi yo'q

        if request.restaurant_id:
            response['X-Tenant-Source'] = request.tenant_source
        return response

    # ---- aniqlash ----
    def _resolve(self, request):
        # 1) JWT (xodim / admin)
        claims = getattr(request, 'auth_claims', None)
        if claims and claims.get('rid'):
            return int(claims['rid']), 'jwt'

        # 2) QR token (mijoz)
        token = request.resolver_match.kwargs.get('token') if request.resolver_match else None
        token = token or request.session.get('qr_token')
        if token:
            rid = self._from_token(token)
            if rid:
                return rid, 'qr_token'

        # 3) Subdomain
        host = request.get_host().split(':')[0]
        parts = host.split('.')
        if len(parts) >= 3 and parts[0] not in ('www', 'api', 'cdn', 'admin'):
            rid = self._from_slug(parts[0])
            if rid:
                return rid, 'subdomain'

        return None, None

    def _from_token(self, token):
        key = f'tenant:token:{token}'
        rid = cache.get(key)
        if rid is None:
            from apps.tenants.models import QrToken
            rid = (QrToken.objects.filter(token=token, is_active=True)
                   .values_list('restaurant_id', flat=True).first())
            cache.set(key, rid or 0, 300)
        return rid or None

    def _from_slug(self, slug):
        key = f'tenant:slug:{slug}'
        rid = cache.get(key)
        if rid is None:
            rid = (Restaurant.objects.filter(slug=slug, is_active=True)
                   .values_list('id', flat=True).first())
            cache.set(key, rid or 0, 300)
        return rid or None
```

⚠ **Worker uchun ham majburiy:**

```
# core/queue/worker.py
def execute_job(job):
    """Worker'da ham tenant kontekst o'rnatiladi — signal/servis ishlashi uchun."""
    rid = job.payload.get('restaurant_id')
    _local.restaurant_id = rid
    try:
        return TASKS[job.task_name](**job.payload)
    finally:
        _local.restaurant_id = None      # ⚡ keyingi job boshqa tenant bo'lishi mumkin
```

2.3 Qatlam 2 — QuerySet Mixin

```
# core/mixins.py
from rest_framework.exceptions import PermissionDenied


class TenantScopedMixin:
    """
    ⚡ HAR ViewSet shu mixin'dan meros olishi SHART.
    CI testi mixin'siz ViewSet'ni topsa — deploy bloklanadi.
    """
    tenant_field = 'restaurant_id'       # murakkab yo'l: 'category__restaurant_id'

    def get_queryset(self):
        qs = super().get_queryset()
        rid = getattr(self.request, 'restaurant_id', None)
        if rid is None:
            raise PermissionDenied("Restoran aniqlanmadi")
        return qs.filter(**{self.tenant_field: rid})

    def perform_create(self, serializer):
        # ⚡ restaurant klientdan EMAS — serverdan
        serializer.save(restaurant_id=self.request.restaurant_id)


class TenantScopedManager(models.Manager):
    """Model darajasidagi qo'shimcha himoya (servis kodida ishlatiladi)."""
    def scoped(self):
        return self.filter(restaurant_id=current_restaurant_id())
```

Model darajasidagi qoida:

```
# apps/menu/models.py
class Dish(models.Model):
    # ⚡ Har tenant-scoped modelda BEVOSITA restaurant FK bo'lishi shart
    # (category orqali emas — filtr sodda va indeks samarali bo'lsin)
    restaurant = models.ForeignKey('tenants.Restaurant', on_delete=models.CASCADE,
                                   related_name='dishes', db_index=True)
    category   = models.ForeignKey('menu.Category', on_delete=models.PROTECT)
    name       = models.CharField(max_length=200)
    price      = models.DecimalField(max_digits=12, decimal_places=2)

    objects = TenantScopedManager()

    class Meta:
        # ⚡ Har kompozit indeks restaurant_id bilan boshlanadi
        indexes = [
            models.Index(fields=['restaurant', 'category', 'is_active']),
            models.Index(fields=['restaurant', 'is_available']),
        ]
        constraints = [
            # Kategoriya boshqa restoranga tegishli bo'lishi mumkin emas
            models.UniqueConstraint(fields=['restaurant', 'slug'],
                                    name='uniq_dish_slug_per_restaurant'),
        ]
```

2.4 Qatlam 3 — Serializer

```
# apps/orders/serializers.py
class OrderItemInSerializer(serializers.Serializer):
    dish_id  = serializers.IntegerField()
    qty      = serializers.IntegerField(min_value=1, max_value=50)
    modifiers= serializers.ListField(child=serializers.IntegerField(),
                                     required=False, max_length=20)
    note     = serializers.CharField(max_length=200, required=False, allow_blank=True)
    # ● price YO'Q — klient narx yubora olmaydi


class OrderCreateSerializer(serializers.ModelSerializer):
    items = OrderItemInSerializer(many=True, min_length=1, max_length=50)

    class Meta:
        model = Order
        # ✦ Qo ro'yxat — hech qachon '__all__'
        fields = ['table_id', 'items', 'comment', 'guest_count']
        # restaurant, status, total, number — serverda

    def validate_table_id(self, value):
        rid = self.context['request'].restaurant_id
        # ✦ Stol SHU restoranniki ekanini tekshirish
        if not Table.objects.filter(id=value, restaurant_id=rid, is_active=True).exists():
            raise serializers.ValidationError("Stol topilmadi")
        return value

    def validate_items(self, items):
        rid = self.context['request'].restaurant_id
        dish_ids = [i['dish_id'] for i in items]
        # ✦ BITTA so'rovda: shu restoran + mavjud + stop-listda emas
        valid = dict(Dish.objects
                     .filter(id__in=dish_ids, restaurant_id=rid,
                             is_active=True, is_available=True)
                     .values_list('id', 'price'))
        missing = set(dish_ids) - set(valid)
        if missing:
            raise serializers.ValidationError(
                f"Taom topilmadi yoki mavjud emas: {sorted(missing)}")
        # Modifikatorlar ham shu restorannikimi?
        mod_ids = {m for i in items for m in i.get('modifiers', [])}
        if mod_ids:
            ok = set(Modifier.objects.filter(id__in=mod_ids, restaurant_id=rid)
                     .values_list('id', flat=True))
            if mod_ids - ok:
                raise serializers.ValidationError("Modifikator topilmadi")
        self.context['dish_prices'] = valid
        return items

    def create(self, validated):
        request = self.context['request']
        rid = request.restaurant_id
        prices = self.context['dish_prices']

        with transaction.atomic():
            order = Order.objects.create(
                restaurant_id=rid,                  # ✦ serverdan
                table_id=validated['table_id'],
                session_key=request.session.session_key,
                status='new',
                comment=validated.get('comment', ''),
            )
            items, total = [], Decimal('0')
            for it in validated['items']:
                # ✦✦ NARX SERVERDA — klient qiymati umuman o'qilmaydi
                unit = prices[it['dish_id']]
                mod_sum = modifier_total(it.get('modifiers', []), rid)
                line = (unit + mod_sum) * it['qty']
                total += line
                items.append(OrderItem(
                    order=order, restaurant_id=rid, dish_id=it['dish_id'],
                    qty=it['qty'], unit_price=unit, modifier_price=mod_sum,
                    line_total=line, note=it.get('note', '')[:200],
                ))
            OrderItem.objects.bulk_create(items)
            order.total = total
            order.save(update_fields=['total'])
            publish('order.created', {'order_id': order.id}, restaurant_id=rid)
        return order
```

FK maydon uchun tenant-scoped queryset:

```
# core/fields.py
class TenantPrimaryKeyRelatedField(serializers.PrimaryKeyRelatedField):
    """Faqat joriy restoran obyektlarini qabul qiladi."""
    def get_queryset(self):
        qs = super().get_queryset()
        rid = self.context['request'].restaurant_id
        return qs.filter(restaurant_id=rid)
```

2.5 Qatlam 4 — WebSocket izolyatsiyasi

> ⚡ WS — eng ko'p unutiladigan sizish nuqtasi.

```
# core/ws/consumers.py
class BaseTenantConsumer(AsyncJsonWebsocketConsumer):
    """Group nomi HAR DOIM serverda quriladi."""

    async def connect(self):
        rid = self.scope.get('restaurant_id')          # auth middleware o'rnatgan
        role = self.scope.get('role')
        if not rid or not role:
            await self.close(code=4401)                # unauthorized
            return
        # ● HECH QACHON: self.scope['url_route']['kwargs']['restaurant_id']
        self.group = f"rest_{rid}_{self.channel_suffix(role)}"
        if not self.role_allowed(role):
            await self.close(code=4403)
            return
        await self.channel_layer.group_add(self.group, self.channel_name)
        await self.accept()

    async def receive_json(self, content):
        # ✦ Kiruvchi xabardagi restaurant_id BUTUNLAY E'TIBORSIZ qoldiriladi
        content.pop('restaurant_id', None)
        await self.handle(content, restaurant_id=self.scope['restaurant_id'])
```

```
# core/ws/sender.py
def ws_send(group: str, payload: dict):
    """✦ Group nomi tekshiriladi — noto'g'ri formatda yuborish mumkin emas."""
    if not re.fullmatch(r'(rest_\d+_(kds|waiters|admin|cashier)|(guest_[a-f0-9]{32})', group):
        raise ValueError(f"Noto'g'ri WS group nomi: {group}")
    async_to_sync(get_channel_layer().group_send)(group, {'type': 'push', **payload})
```

2.6 🔴 Qanday buziladi — real misollar

#	Xato	Kod	Nima bo'ladi	To'g'ri yechim
1	`get_object_or_404` filtrsiz	`dish = get_object_or_404(Dish, pk=pk)`	Har kim har restoran taomini ko'radi	`get_object_or_404(Dish, pk=pk, restaurant_id=request.restaurant_id)`
2	ViewSet mixin'siz	`class DishVS(ModelViewSet): queryset = Dish.objects.all()`	To'liq katalog sizadi	`class DishVS(TenantScopedMixin, ModelViewSet)`
3	Nested serializer FK	`category = PrimaryKeyRelatedField(queryset=Category.objects.all())`	Boshqa restoran kategoriyasiga bog'lash	`TenantPrimaryKeyRelatedField(queryset=Category.objects.all())`
4	Signal'da tenant yo'q	`@receiver(post_save) def f(instance): Stat.objects.create(...)`	Statistika aralashadi	`restaurant_id=instance.restaurant_id` aniq berish
5	Worker'da kontekst yo'q	`def task(dish_id): Dish.objects.get(pk=dish_id)`	Fon ishda sizish	`restaurant_id` payload'da majburiy + `_local` o'rnatish
6	Kesh kaliti tenant'siz	`cache.set(f'menu:{lang}', data)`	🔴 Restoran A menyusi B ga ko'rinadi	`cache.set(f'menu:{rid}:{lang}', data)`
7	WS group klientdan	`group = content['room']`	Boshqa restoran KDS'ini eshitish	Group serverda `scope` dan quriladi
8	Idempotency kalit global	`f'idem:{key}'`	Tenantlar to'qnashadi	`f'idem:{rid}:{key}'`
9	Raw SQL	`Dish.objects.raw("SELECT * FROM dish WHERE id=%s", [pk])`	Filtr yo'q	Raw taqiqlangan; kerak bo'lsa `WHERE restaurant_id=%s` majburiy
10	Aggregate filtrsiz	`Order.objects.aggregate(Sum('total'))`	Butun platforma aylanmasi	`.filter(restaurant_id=rid).aggregate(...)`
11	`related_name` orqali	`restaurant.dishes.all()` ✓ lekin `category.dishes.all()` ⛔	Kategoriya boshqa restoranniki bo'lsa	Kategoriya ham avval scope qilinadi
12	Fayl yo'li tenant'siz	`media/3d/{dish_id}.glb`	ID taxmin qilinsa fayl olinadi	`media/3d/{rid}/{uuid}.glb` + presigned URL
13	Eksport (Excel/PDF)	`qs = Order.objects.filter(date__gte=d)`	Butun platforma hisoboti	Eksport ham `TenantScopedMixin` orqali
14	Admin autocomplete	DRF `SearchFilter` mixin'siz view'da	Qidiruvda boshqa tenant	Autocomplete ham scoped ViewSet
15	`select_related` teskari yo'nalish	`OrderItem.objects.select_related('order')` filtrsiz	Barcha buyurtma satrlari	`OrderItem` da ham bevosita `restaurant_id` bor

2.7 CI tekshiruvi

```
# tests/security/test_tenant_isolation.py
import pytest
from django.apps import apps
from rest_framework.viewsets import GenericViewSet
from core.mixins import TenantScopedMixin

TENANT_MODELS = [m for m in apps.get_models()
                 if any(f.name == 'restaurant' for f in m._meta.fields)]

EXEMPT_VIEWSETS = {'PlatformRestaurantViewSet', 'HealthViewSet'}  # ataylab global


# ─────────────────────────────────────────────────────────────
# TEST 1 — har ViewSet mixin'ga ega (statik introspection)
# ─────────────────────────────────────────────────────────────
def test_all_viewsets_are_tenant_scoped():
    from core.urls_registry import all_viewsets
    bad = []
    for vs in all_viewsets():
        if vs.__name__ in EXEMPT_VIEWSETS:
            continue
        model = getattr(getattr(vs, 'queryset', None), 'model', None)
        if model in TENANT_MODELS and not issubclass(vs, TenantScopedMixin):
            bad.append(vs.__name__)
    assert not bad, f"❌ TenantScopedMixin yo'q ViewSet'lar: {bad}"


# ─────────────────────────────────────────────────────────────
# TEST 2 — har model uchun cross-tenant kirish 403/404
# ─────────────────────────────────────────────────────────────
@pytest.mark.parametrize('model', TENANT_MODELS, ids=lambda m: m.__name__)
@pytest.mark.django_db
def test_cross_tenant_read_denied(model, api_client, rest_a, rest_b, factory):
    obj_b = factory(model, restaurant=rest_b)
    url = detail_url_for(model, obj_b.pk)
    if url is None:
        pytest.skip(f"{model.__name__} uchun API endpoint yo'q")
    api_client.login_as_admin(rest_a)               # A restorani admini
    resp = api_client.get(url)
    assert resp.status_code in (403, 404), \
        f"🔴 SIZISH: {model.__name__} — A restorani B ning obyektini ko'rdi ({resp.status_code})"


@pytest.mark.parametrize('model', TENANT_MODELS, ids=lambda m: m.__name__)
@pytest.mark.django_db
def test_cross_tenant_write_denied(model, api_client, rest_a, rest_b, factory):
    obj_b = factory(model, restaurant=rest_b)
    url = detail_url_for(model, obj_b.pk)
    if url is None:
        pytest.skip()
    api_client.login_as_admin(rest_a)
    for method in ('patch', 'delete'):
        resp = getattr(api_client, method)(url, {})
        assert resp.status_code in (403, 404), \
            f"🔴 SIZISH: {model.__name__} {method.upper()} ruxsat etildi"


# ─────────────────────────────────────────────────────────────
# TEST 3 — boshqa restoran dish_id bilan buyurtma
# ─────────────────────────────────────────────────────────────
@pytest.mark.django_db
def test_order_with_foreign_dish_rejected(api_client, rest_a, rest_b, dish_factory,
                                          table_factory):
    dish_b = dish_factory(restaurant=rest_b, price=50000)
    table_a = table_factory(restaurant=rest_a)
    api_client.as_guest(rest_a, table_a)
    resp = api_client.post('/v1/orders/', {
        'table_id': table_a.id,
        'items': [{'dish_id': dish_b.id, 'qty': 1}],
    }, format='json')
    assert resp.status_code == 400
    assert 'Taom topilmadi' in str(resp.data)


# ─────────────────────────────────────────────────────────────
# TEST 4 — kesh kalitlari tenant bilan
# ─────────────────────────────────────────────────────────────
def test_cache_keys_contain_tenant():
    """Statik tahlil: cache.set/get chaqiruvlarida rid bo'lishi shart."""
    import ast, pathlib
    bad = []
    for path in pathlib.Path('apps').rglob('*.py'):
        tree = ast.parse(path.read_text())
        for node in ast.walk(tree):
            if (isinstance(node, ast.Call)
                    and isinstance(node.func, ast.Attribute)
                    and node.func.attr in ('set', 'get', 'get_or_set')
                    and getattr(node.func.value, 'id', '') == 'cache'
                    and node.args):
                src = ast.unparse(node.args[0])
                if 'rid' not in src and 'restaurant' not in src and 'global:' not in src:
                    bad.append(f"{path}:{node.lineno} → {src}")
    assert not bad, f"❌ Tenant'siz kesh kalitlari:\n" + "\n".join(bad)


# ─────────────────────────────────────────────────────────────
# TEST 5 — narx manipulyatsiyasi
# ─────────────────────────────────────────────────────────────
@pytest.mark.django_db
def test_client_cannot_set_price(api_client, rest_a, dish_factory, table_factory):
    dish = dish_factory(restaurant=rest_a, price=Decimal('75000'))
    table = table_factory(restaurant=rest_a)
    api_client.as_guest(rest_a, table)
    resp = api_client.post('/v1/orders/', {
        'table_id': table.id,
        'items': [{'dish_id': dish.id, 'qty': 2, 'price': 1, 'unit_price': 1}],
    }, format='json')
    assert resp.status_code == 201
    order = Order.objects.get(pk=resp.data['id'])
    assert order.total == Decimal('150000'), "🔴 Klient narxni o'zgartira oldi!"


# ─────────────────────────────────────────────────────────────
# TEST 6 — WS group izolyatsiyasi
# ─────────────────────────────────────────────────────────────
@pytest.mark.asyncio
async def test_ws_cannot_join_foreign_group(rest_a, rest_b):
    comm = WebsocketCommunicator(app, f'/ws/kds/')
    comm.scope['restaurant_id'] = rest_a.id
    comm.scope['role'] = 'cook'
    connected, _ = await comm.connect()
    assert connected
    # B restoraniga xabar yuboramiz — A eshitmasligi kerak
    await ws_send_async(f'rest_{rest_b.id}_kds', {'order_id': 1})
    with pytest.raises(asyncio.TimeoutError):
        await comm.receive_json_from(timeout=1)
```

CI gate:

```
# .github/workflows/security.yml
name: Security Gate
on: [push, pull_request]

jobs:
  tenant-isolation:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: ⚡ Multi-tenant izolyatsiya testlari
        run: pytest tests/security/test_tenant_isolation.py -x -q --no-header
      - name: Taqiqlangan naqshlar (grep gate)
        run: |
          set -e
          fail=0
          echo "→ objects.all() ViewSet'da"
          ! grep -rn "queryset = .*\.objects\.all()" apps/*/views*.py \
              --include="*.py" | grep -v "TenantScoped" || fail=1
          echo "→ .raw( ishlatilishi"
          ! grep -rn "\.raw(" apps/ --include="*.py" || fail=1
          echo "→ fields = '__all__'"
          ! grep -rn "fields = '__all__'" apps/ --include="*.py" || fail=1
          echo "→ pickle"
          ! grep -rn "import pickle" apps/ core/ --include="*.py" || fail=1
          echo "→ mark_safe / |safe"
          ! grep -rn "mark_safe\|{{.*|safe" apps/ templates/ || fail=1
          exit $fail
      - name: Bog'liqlik audit
        run: pip-audit --strict -r requirements.txt
      - name: Statik tahlil
        run: bandit -r apps/ core/ -ll -x tests
```

2.8 Doimiy audit (production'da)

```
# core/security/tenant_audit.py
"""
Har kecha production DB'da tenant butunligini tekshiradi.
Har qanday anomaliya → 🔴 P1 alert.
"""
CHECKS = [
    # (nom, SQL, kutilgan natija)
    ("dish_category_mismatch", """
        SELECT count(*) FROM menu_dish d
        JOIN menu_category c ON c.id = d.category_id
        WHERE d.restaurant_id <> c.restaurant_id
    """, 0),
    ("orderitem_order_mismatch", """
        SELECT count(*) FROM orders_orderitem oi
        JOIN orders_order o ON o.id = oi.order_id
        WHERE oi.restaurant_id <> o.restaurant_id
    """, 0),
    ("order_table_mismatch", """
        SELECT count(*) FROM orders_order o
        JOIN tenants_table t ON t.id = o.table_id
        WHERE o.restaurant_id <> t.restaurant_id
    """, 0),
    ("payment_order_mismatch", """
        SELECT count(*) FROM payments_transaction p
        JOIN orders_order o ON o.id = p.order_id
        WHERE p.restaurant_id <> o.restaurant_id
    """, 0),
    ("null_restaurant_id", """
        SELECT count(*) FROM menu_dish WHERE restaurant_id IS NULL
    """, 0),
]


def run_audit():
    problems = []
    with connections['report'].cursor() as cur:
        for name, sql, expected in CHECKS:
            cur.execute(sql)
            got = cur.fetchone()[0]
            metrics.gauge('armenu_tenant_audit', 0 if got != expected else 1,
                          labels={'check': name})
            if got != expected:
                problems.append(f"{name}: {got} (kutilgan {expected})")
    if problems:
        alert_oncall("🔴🔴 TENANT BUTUNLIGI BUZILGAN:\n" + "\n".join(problems)
                     + "\nQILISH: 1) deploy to'xtatilsin 2) so'nggi migratsiya/PR "
                       "ko'rib chiqilsin 3) ta'sirlangan yozuvlar aniqlansin",
                     severity='P1')
    return problems
```

Anomaliya aniqlash (jonli):

Signal	Chegara	Ma'nosi
Bitta sessiya > 3 ta `restaurant_id` bilan so'rov	5 daq	QR token skanerlash urinishi
`403 tenant_denied` > 20/daq bitta IP dan	1 daq	Cross-tenant zondlash
`X-Tenant-Source: subdomain` mismatch JWT bilan	har hodisa	🔴 Token qayta ishlatish urinishi
Bitta IP dan > 50 ta turli `slug`	10 daq	Tenant ro'yxatini yig'ish

---

3. AUTENTIFIKATSIYA ARXITEKTURASI

3.1 4 xil aktor — umumiy ko'rinish

```
┌──────────────┬──────────────┬──────────────┬──────────┬──────────┐
│   AKTOR      │  AUTH USULI  │    TOKEN     │   TTL    │  RISK    │
├──────────────┼──────────────┼──────────────┼──────────┼──────────┤
│ 🧍 Mijoz     │ 🚫 YO'Q      │ device cookie│ 12 soat  │ 🟢 Past  │
│              │ (QR + cookie)│ + session    │          │          │
├──────────────┼──────────────┼──────────────┼──────────┼──────────┤
│ 👨‍🍳 Ofitsiant│ PIN (6 raqam)│ JWT access   │ 15 daq   │ 🟡 O'rta │
│              │              │ + refresh    │ 12 soat  │          │
├──────────────┼──────────────┼──────────────┼──────────┼──────────┤
│ 🧑‍💼 Admin    │ Email+parol  │ JWT access   │ 15 daq   │ 🟠 Yuqori│
│              │              │ + refresh    │ 7 kun    │          │
├──────────────┼──────────────┼──────────────┼──────────┼──────────┤
│ 🛡 Platforma │ Parol + 2FA  │ JWT access   │ 10 daq   │ 🔴 Kritik│
│   admin      │ (TOTP)       │ + refresh    │ 24 soat  │          │
└──────────────┴──────────────┴──────────────┴──────────┴──────────┘
```

3.2 🧍 Mijoz — autentifikatsiyasiz

> **Qaror:** mijozdan ro'yxatdan o'tish **so'ralmaydi**. Bu mahsulot qarori (konversiya), lekin xavfsizlik oqibatlari bor va ular boshqariladi.

```
  QR skanerlash                Server
       │                         │
       ├── GET /t/K7xQm2p9nR ──► │
       │                         │ 1) token → (restaurant_id, table_id)
       │                         │ 2) rate limit: 60/daq/IP
       │                         │ 3) session yaratish (Django session, Redis)
       │                         │ 4) device_id cookie (imzolangan, 12 soat)
       │ ◄── 200 + Set-Cookie ──┤    httpOnly, Secure, SameSite=Lax
       │     armenu_did=<signed>  │
       │     sessionid=<...>      │
       │                         │
       ├── POST /v1/orders/ ────► │ session → (rid, table_id) tekshiriladi
       │   Cookie: sessionid     │ ● Body'dagi rid/table E'TIBORSIZ
```

```
# apps/guest/auth.py
from itsdangerous import URLSafeTimedSerializer, BadSignature

_signer = URLSafeTimedSerializer(settings.DEVICE_COOKIE_SECRET, salt='armenu-device')
DEVICE_TTL = 12 * 3600


def issue_device_cookie(response, restaurant_id, table_id, request):
    payload = {
        'did': secrets.token_urlsafe(12),
        'rid': restaurant_id,
        'tid': table_id,
        # ⚠ Zaif fingerprint — klonlashni qiyinlashtiradi, lekin UX buzmaydi
        'fp': hashlib.sha256(
            f"{request.META.get('HTTP_USER_AGENT','')}|"
            f"{request.META.get('HTTP_ACCEPT_LANGUAGE','')}".encode()
        ).hexdigest()[:12],
    }
    response.set_cookie(
        'armenu_did', _signer.dumps(payload),
        max_age=DEVICE_TTL, httponly=True, secure=True, samesite='Lax',
        path='/',
    )


def read_device_cookie(request):
    raw = request.COOKIES.get('armenu_did')
    if not raw:
        return None
    try:
        data = _signer.loads(raw, max_age=DEVICE_TTL)
    except BadSignature:
        metrics.inc('armenu_device_cookie_invalid_total')
        return None
    expected_fp = hashlib.sha256(
        f"{request.META.get('HTTP_USER_AGENT','')}|"
        f"{request.META.get('HTTP_ACCEPT_LANGUAGE','')}".encode()
    ).hexdigest()[:12]
    if data.get('fp') != expected_fp:
        # ⚠ Blok qilmaymiz (brauzer yangilanishi mumkin) — faqat belgilaymiz
        data['fp_mismatch'] = True
        metrics.inc('armenu_device_fp_mismatch_total')
    return data
```

Xususiyat	Qiymat	Sabab
Token uzunligi	22 belgi (128 bit entropiya)	Brute-force amaliy jihatdan imkonsiz
Cookie TTL	12 soat	Bitta smena; ertasi kuni yangi sessiya
Session backend	Redis (`django.contrib.sessions.backends.cache`)	Tez, avtomatik muddat
Revoke	QR token `is_active=False` → barcha shu stol sessiyalari	Restoran admin panelidan
Rate limit	60/daq/IP (QR), 10/daq/sessiya (buyurtma)	§8
⛔ Nima saqlanmaydi	Ism, telefon, karta, joylashuv	Faqat ixtiyoriy: chek uchun telefon

QR token rotatsiyasi:

```
# apps/tenants/models.py
class QrToken(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=models.CASCADE)
    table      = models.ForeignKey(Table, on_delete=models.CASCADE)
    token      = models.CharField(max_length=32, unique=True, db_index=True,
                                  default=lambda: secrets.token_urlsafe(16))
    is_active  = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    rotated_at = models.DateTimeField(null=True)

    def rotate(self):
        """QR chop etilgan bo'lsa — eski 30 kun ishlaydi (grace)."""
        old = self.token
        self.token = secrets.token_urlsafe(16)
        self.rotated_at = timezone.now()
        self.save()
        QrTokenGrace.objects.create(old_token=old, new=self,
                                    expires_at=timezone.now() + timedelta(days=30))
```

3.3 👨‍🍳 Ofitsiant — PIN + JWT

```
  Ofitsiant planshet              Server
      |                             |
      |── POST /v1/staff/login/ ──→ | 1) rate limit: 5/15daq (IP + PIN prefiks)
      |   {pin: "482913",           | 2) restaurant_id — subdomain'dan
      |    device_id: "tab-03"}     | 3) bcrypt.checkpw(pin, staff.pin_hash)
      |                             | 4) staff.is_active + smena tekshiruvi
      |                             | 5) device allowlist (ixtiyoriy)
      |←── 200 ─────────────────────|
      |   access  (15 daq, body)    |   ⚡ access — body'da (SPA ishlatadi)
      |   Set-Cookie: refresh       |   ⚡ refresh — httpOnly cookie
      |   (12 soat, httpOnly)       |
      |                             |
      |── GET /v1/orders/ ─────────→| Authorization: Bearer <access>
      |                             | claims: {sub, rid, role, sid, exp}
      |                             |
      |── POST /v1/staff/refresh/ ─→| ⚡ Rotatsiya: eski refresh BEKOR qilinadi
      |   Cookie: refresh           |   Qayta ishlatilsa → butun oila revoke
      |←── yangi access + refresh ──|
```

```
# apps/staff/auth.py
import bcrypt, jwt, secrets
from django.core.cache import cache

ACCESS_TTL  = 15 * 60
REFRESH_TTL = 12 * 3600
MAX_PIN_ATTEMPTS = 5
LOCKOUT = 15 * 60


class StaffAuthService:

    def login(self, pin: str, restaurant_id: int, request):
        # 1) Rate limit — IP va restoran bo'yicha
        rl_key = f'pinfail:{restaurant_id}:{client_ip(request)}'
        if int(cache.get(rl_key) or 0) >= MAX_PIN_ATTEMPTS:
            audit(request, 'staff.login.locked', restaurant_id=restaurant_id)
            raise Throttled(detail="Juda ko'p urinish. 15 daqiqadan keyin qayta urining")

        # 2) PIN tekshiruvi — faqat shu restoran xodimlari
        staff = self._find_by_pin(pin, restaurant_id)
        if staff is None:
            cache.set(rl_key, int(cache.get(rl_key) or 0) + 1, LOCKOUT)
            audit(request, 'staff.login.failed', restaurant_id=restaurant_id)
            # ⚡ Umumiy xabar — xodim borligini oshkor qilmaydi
            raise AuthenticationFailed("PIN noto'g'ri")

        cache.delete(rl_key)
        session_id = secrets.token_urlsafe(16)
        access  = self._issue_access(staff, session_id)
        refresh = self._issue_refresh(staff, session_id)
        audit(request, 'staff.login.success', restaurant_id=restaurant_id,
              actor_id=staff.id)
        return access, refresh

    def _find_by_pin(self, pin, restaurant_id):
        """⚠ bcrypt sekin — barcha xodimni tekshirish qimmat.
        Yechim: PIN'ning HMAC prefiksi bo'yicha indeks."""
        lookup = hmac.new(settings.PIN_LOOKUP_KEY, pin.encode(), 'sha256').hexdigest()[:16]
        candidates = Staff.objects.filter(
            restaurant_id=restaurant_id, pin_lookup=lookup, is_active=True)
        for s in candidates:                    # odatda 0 yoki 1 ta
            if bcrypt.checkpw(pin.encode(), s.pin_hash.encode()):
                return s
        return None

    def _issue_access(self, staff, session_id):
        return jwt.encode({
            'sub': staff.id, 'rid': staff.restaurant_id,
            'role': staff.role, 'sid': session_id, 'typ': 'access',
            'iat': int(time.time()), 'exp': int(time.time()) + ACCESS_TTL,
        }, settings.JWT_SECRET, algorithm='HS256')

    def _issue_refresh(self, staff, session_id, family=None):
        family = family or secrets.token_urlsafe(12)
        token = secrets.token_urlsafe(32)
        cache.set(f'refresh:{token}',
                  {'sub': staff.id, 'rid': staff.restaurant_id,
                   'sid': session_id, 'family': family},
                  REFRESH_TTL)
        cache.sadd(f'refresh_family:{family}', token)
        return token

    def refresh(self, token, request):
        data = cache.get(f'refresh:{token}')
        if not data:
            # ⚡ Ishlatilgan/mavjud bo'lmagan token — o'g'irlik belgisi
            self._revoke_family_by_token(token)
            audit(request, 'staff.refresh.reuse_detected', severity='high')
            raise AuthenticationFailed("Sessiya bekor qilindi")

        cache.delete(f'refresh:{token}')          # ⚡ bir martalik
        staff = Staff.objects.get(pk=data['sub'], is_active=True)
        new_access  = self._issue_access(staff, data['sid'])
        new_refresh = self._issue_refresh(staff, data['sid'], family=data['family'])
        return new_access, new_refresh

    def revoke_all(self, staff_id):
        """Xodim ishdan bo'shatilganda / PIN o'zgarganda."""
        cache.set(f'staff_revoked_after:{staff_id}', int(time.time()), 86400 * 30)
```

Access token'ni bekor qilish (JWT stateless muammosi):

```
# core/auth/jwt_backend.py
def validate_access(token):
    claims = jwt.decode(token, settings.JWT_SECRET, algorithms=['HS256'])
    # ⚡ Oora ro'yxat: 15 daqiqalik oyna uchun Redis lookup (arzon)
    revoked_after = cache.get(f"staff_revoked_after:{claims['sub']}")
    if revoked_after and claims['iat'] < revoked_after:
        raise AuthenticationFailed("Sessiya bekor qilingan")
    if cache.get(f"session_revoked:{claims['sid']}"):
        raise AuthenticationFailed("Sessiya bekor qilingan")
    return claims
```

3.4 👨‍💼 Restoran admin — parol + JWT

Xususiyat	Qiymat
Parol siyosati	Min 10 belgi, 3 xil sinf, `HaveIBeenPwned` k-anonymity tekshiruvi
Hash	Argon2id (`memory_cost=64MB, time_cost=3, parallelism=4`)
Access TTL	15 daqiqa
Refresh TTL	7 kun (rotatsiya bilan)
Rate limit	5/15daq (IP+email), 20/soat (email)
Blok	5 xato → 15 daq; 15 xato → 24 soat + email xabar
Parol tiklash	Bir martalik token, 30 daq, email orqali, eski sessiyalar revoke
Parol o'zgarishi	Barcha refresh token revoke + email xabarnoma
Sessiya ro'yxati	Admin o'z faol sessiyalarini ko'radi va uzadi (IP, qurilma, vaqt)
2FA	Ixtiyoriy (tavsiya etiladi), majburiy — moliyaviy huquqli admin uchun

3.5 🛡 Platforma admin — 2FA majburiy




```python
# apps/platform/auth.py
import pyotp

class PlatformAuthService:
    ACCESS_TTL  = 10 * 60          # ⚡ eng qisqa
    REFRESH_TTL = 24 * 3600
    MFA_TTL     = 5 * 60

    def login_step1(self, email, password, request):
        ip = client_ip(request)
        if not ip_in_allowlist(ip):
            audit(request, 'platform.login.ip_denied', severity='high')
            raise PermissionDenied("IP ruxsat etilmagan")
        if rate_exceeded(f'plogin:{ip}', limit=3, window=900):
            raise Throttled()

        admin = PlatformAdmin.objects.filter(email__iexact=email, is_active=True).first()
        if not admin or not argon2_verify(admin.password_hash, password):
            audit(request, 'platform.login.failed', severity='high')
            raise AuthenticationFailed("Email yoki parol noto'g'ri")

        mfa_token = secrets.token_urlsafe(24)
        cache.set(f'mfa:{mfa_token}', {'admin_id': admin.id, 'ip': ip}, self.MFA_TTL)
        return mfa_token

    def login_step2(self, mfa_token, code, request):
        data = cache.get(f'mfa:{mfa_token}')
        if not data or data['ip'] != client_ip(request):
            raise AuthenticationFailed("Sessiya muddati tugadi")
        admin = PlatformAdmin.objects.get(pk=data['admin_id'])

        # ⚡ Kod qayta ishlatilmasin (replay himoyasi)
        used_key = f'totp_used:{admin.id}:{code}'
        if cache.get(used_key):
            audit(request, 'platform.2fa.replay', severity='critical')
            raise AuthenticationFailed("Kod allaqachon ishlatilgan")

        totp = pyotp.TOTP(decrypt_secret(admin.totp_secret_enc))
        if not totp.verify(code, valid_window=1):
            if not self._check_recovery_code(admin, code):
                audit(request, 'platform.2fa.failed', severity='high')
                raise AuthenticationFailed("Kod noto'g'ri")

        cache.set(used_key, 1, 90)
        cache.delete(f'mfa:{mfa_token}')
        audit(request, 'platform.login.success', severity='info')
        notify_telegram(f"🛡 Platforma admin kirdi: {admin.email} · {client_ip(request)}")
        return self._issue_tokens(admin)
```

3.6 Umumiy jadval — TTL / refresh / revoke / rate limit

Aktor	Access TTL	Refresh TTL	Refresh rotatsiya	Revoke usuli	Login rate limit	Sessiya limiti
Mijoz	— (session)	—	—	QR token `is_active=False`	60/daq/IP	3/qurilma
Ofitsiant	15 daq	12 soat	✓ (oila revoke)	`revoke_all(staff_id)`	5/15daq (IP+PIN)	2 qurilma
Oshpaz (KDS)	60 daq[1]	24 soat	✓	`revoke_all`	5/15daq	5 ekran
Kassir	15 daq	12 soat	✓	`revoke_all`	5/15daq	2 qurilma
Restoran admin	15 daq	7 kun	✓	Parol o'zgarishi / sessiya uzish	5/15daq (IP+email)	5 sessiya
Platforma admin	10 daq	24 soat	✓	Darhol (barcha)	3/15daq + IP allowlist	2 sessiya

[1] *KDS — devor ekranida, doim ochiq turadi; qurilma jismonan oshxonada (past risk).*

---

4. AVTORIZATSIYA

4.1 Rol matritsasi (5 rol × 22 amal)

#	Amal	👤 guest	🤵 waiter	👨‍🍳 cook	💰 cashier	👔 admin	🛡 platform
1	Menyu ko'rish	✓	✓	✓	✓	✓	✓
2	AR/3D ko'rish	✓	✓	✓	✓	✓	✓
3	Buyurtma yaratish	✓	✓	⛔	✓	✓	⛔
4	O'z buyurtmasini ko'rish	✓[1]	✓	✓	✓	✓	⛔[2]
5	Barcha buyurtmalarni ko'rish	⛔	✓	✓	✓	✓	⛔[2]
6	Buyurtmaga qo'shish	✓[1]	✓	⛔	✓	✓	⛔
7	Buyurtma satrini o'chirish	⛔	✓[3]	⛔	✓[3]	✓	⛔
8	Buyurtma statusini o'zgartirish	⛔	✓[4]	✓[5]	⛔	✓	⛔
9	Buyurtmani bekor qilish	⛔	⛔	⛔	✓	✓	⛔
10	To'lov qabul qilish (naqd)	⛔	✓	⛔	✓	✓	⛔
11	Onlayn to'lov boshlash	✓	✓	⛔	✓	✓	⛔
12	Qaytarish (refund)	⛔	⛔	⛔	✓[6]	✓	⛔
13	Chegirma qo'llash	⛔	⛔	⛔	✓[7]	✓	⛔
14	Ofitsiant chaqirish	✓	⛔	⛔	⛔	⛔	⛔
15	Stop-list boshqarish	⛔	⛔	✓	⛔	✓	⛔
16	Taom yaratish/tahrirlash	⛔	⛔	⛔	⛔	✓	⛔
17	Narx o'zgartirish	⛔	⛔	⛔	⛔	✓	⛔
18	3D model yuklash	⛔	⛔	⛔	⛔	✓	✓
19	Xodim qo'shish / PIN berish	⛔	⛔	⛔	⛔	✓	⛔
20	Moliyaviy hisobot	⛔	⛔	⛔	✓[8]	✓	⛔[2]
21	Restoran sozlamalari	⛔	⛔	⛔	⛔	✓	✓
22	Restoran yaratish / tarif	⛔	⛔	⛔	⛔	⛔	✓
23	AuditLog ko'rish	⛔	⛔	⛔	⛔	✓[9]	✓
24	Platforma statistikasi	⛔	⛔	⛔	⛔	⛔	✓

Izohlar:
[1] Faqat o'z sessiyasi · [2] ⚡ Platforma admin **mijoz ma'lumotini ko'rmaydi** (faqat agregat) ·
[3] Faqat `new` / `confirmed` statusda · [4] `confirmed` → `served` · [5] `confirmed` → `cooking` → `ready` ·
[6] Kunlik limit bilan · [7] Maks 10%, undan yuqorisi — admin · [8] Faqat o'z smenasi ·
[9] Faqat o'z restorani

4.2 Permission klasslari

```
# core/permissions.py
from rest_framework.permissions import BasePermission

ROLE_MATRIX = {
    'order.create':       {'guest', 'waiter', 'cashier', 'admin'},
    'order.list_all':     {'waiter', 'cook', 'cashier', 'admin'},
    'order.cancel':       {'cashier', 'admin'},
    'order.status.cook':  {'cook', 'admin'},
    'order.status.serve':  {'waiter', 'admin'},
    'payment.cash':       {'waiter', 'cashier', 'admin'},
    'payment.refund':     {'cashier', 'admin'},
    'discount.apply':     {'cashier', 'admin'},
    'menu.write':         {'admin'},
    'menu.stoplist':      {'cook', 'admin'},
    'staff.manage':       {'admin'},
    'report.finance':     {'cashier', 'admin'},
    'settings.restaurant': {'admin', 'platform_admin'},
    'platform.manage':    {'platform_admin'},
    'audit.read':         {'admin', 'platform_admin'},
}


class HasAction(BasePermission):
    """View'da `required_action = 'order.cancel'` deb belgilanadi."""
    message = "Bu amal uchun ruxsat yo'q"

    def has_permission(self, request, view):
        action = getattr(view, 'required_action', None)
        if action is None:
            raise ImproperlyConfigured(f"{view.__class__.__name__}.required_action yo'q")
        role = getattr(request, 'role', 'anonymous')
        allowed = role in ROLE_MATRIX.get(action, set())
        if not allowed:
            audit(request, 'authz.denied', extra={'action': action, 'role': role})
        return allowed


class IsOwnOrderOrStaff(BasePermission):
    """Object-level: mijoz FAQAT o'z buyurtmasini ko'radi."""
    def has_object_permission(self, request, view, obj):
        # 1) Tenant chegarasi (mixin allaqachon filtrlagan, lekin ikkinchi tekshiruv)
        if obj.restaurant_id != request.restaurant_id:
            audit(request, 'authz.cross_tenant_object', severity='critical',
                  extra={'model': obj.__class__.__name__, 'pk': obj.pk})
            return False
        role = getattr(request, 'role', 'anonymous')
        if role in ('waiter', 'cook', 'cashier', 'admin'):
            return True
        # 2) Mijoz — sessiya mosligini tekshirish
        return obj.session_key == request.session.session_key


class CanChangeOrderStatus(BasePermission):
    """State machine: kim qaysi o'tishni qila oladi."""
    TRANSITIONS = {
        'cook':    {('confirmed', 'cooking'), ('cooking', 'ready')},
        'waiter':  {('ready', 'served'), ('new', 'confirmed')},
        'cashier': {('served', 'paid'), ('new', 'cancelled'), ('confirmed', 'cancelled')},
        'admin':   'ALL',
    }

    def has_object_permission(self, request, view, obj):
        role = request.role
        target = request.data.get('status')
        allowed = self.TRANSITIONS.get(role, set())
        if allowed == 'ALL':
            return True
        return (obj.status, target) in allowed
```

ViewSet'da qo'llanilishi:

```
# apps/orders/views.py
class OrderViewSet(TenantScopedMixin, viewsets.ModelViewSet):
    queryset = Order.objects.select_related('table').prefetch_related('items__dish')
    permission_classes = [HasAction, IsOwnOrderOrStaff]

    @property
    def required_action(self):
        return {'create': 'order.create', 'list': 'order.list_all',
                'retrieve': 'order.list_all', 'destroy': 'order.cancel'}.get(
                    self.action, 'order.list_all')

    def get_queryset(self):
        qs = super().get_queryset()               # ← tenant filtri
        if self.request.role == 'guest':
            qs = qs.filter(session_key=self.request.session.session_key)
        return qs

    @action(detail=True, methods=['post'],
            permission_classes=[HasAction, CanChangeOrderStatus])
    def change_status(self, request, pk=None):
        order = self.get_object()                 # ← object permission ishlaydi
        new_status = request.data['status']
        order.transition_to(new_status, actor=request.actor)
        return Response(OrderSerializer(order).data)
```

---

5. MA'LUMOT HIMOYASI

5.1 Transit — TLS 1.3

```
# /etc/nginx/snippets/tls.conf
ssl_protocols             TLSv1.2 TLSv1.3;   # 1.0/1.1 o'chirilgan
ssl_prefer_server_ciphers off;               # TLS 1.3 da klient tanlaydi
ssl_ciphers               ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305;
ssl_ecdh_curve            X25519:secp384r1;
ssl_session_timeout       1d;
ssl_session_cache         shared:SSL:20m;
ssl_session_tickets       off;               # forward secrecy
ssl_stapling              on;
ssl_stapling_verify       on;

add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;
add_header X-Content-Type-Options    "nosniff" always;
add_header X-Frame-Options           "DENY" always;
add_header Referrer-Policy           "strict-origin-when-cross-origin" always;
add_header Permissions-Policy        "camera=(self), microphone=(), geolocation=(), payment=()" always;
add_header Content-Security-Policy   "default-src 'self'; script-src 'self' 'nonce-$request_id' https://cdn.armenu.uz; style-src 'self' 'unsafe-inline'; img-src 'self' data: https://cdn.armenu.uz; model-src 'self' https://cdn.armenu.uz; connect-src 'self' wss://armenu.uz; frame-ancestors
```

Yo'nalish	Himoya
Brauzer ↔ Cloudflare	TLS 1.3, HSTS preload
Cloudflare ↔ origin	TLS 1.2+ (Full Strict), origin sertifikat
Web ↔ PostgreSQL	`sslmode=verify-full`, private VPC
Web ↔ Redis	TLS + `requirepass`, private VPC
Web ↔ S3	HTTPS, SigV4
Web ↔ to'lov/OFD	TLS 1.2+, ba'zi provayderlarda mTLS
Node ↔ node (VPC)	WireGuard tunnel (S3+ bosqichda)

5.2 At-rest — shifrlash

Qatlam	Nima	Usul	Kalit boshqaruvi
Disk	Butun DB volume	LUKS (dm-crypt, AES-256-XTS)	Kalit boot'da HSM/tpm dan
DB — maydon	`totp_secret`, `provider_api_key`, `pin_lookup_key`	`pgcrypto` / ilova darajasida Fernet	`SECRET_KEY_ENCRYPTION` (env, rotatsiya 90 kun)
DB — parol	Admin paroli	Argon2id (qaytarilmaydi)	—
DB — PIN	Ofitsiant PIN	bcrypt cost=12 + HMAC lookup	`PIN_LOOKUP_KEY`
Backup	PG dump/basebackup	`age` (X25519 + ChaCha20-Poly1305)	Ochiq kalit serverda, yopiq kalit offline seyfda
S3 media	3D/rasm	SSE-S3 (AES-256)	AWS/MinIO boshqaradi
Log	Arxiv	S3 SSE + WORM (Object Lock)	—

```python
# core/crypto.py
from cryptography.fernet import Fernet, MultiFernet

def _fernets():
    """Kalit rotatsiyasi: birinchi kalit shifrlaydi, hammasi ochadi."""
    keys = settings.FIELD_ENCRYPTION_KEYS      # ['yangi', 'eski', ...]
    return MultiFernet([Fernet(k) for k in keys])


class EncryptedTextField(models.TextField):
    """Shaffof shifrlangan maydon."""
    def get_prep_value(self, value):
        if value in (None, ''):
            return value
        return _fernets().encrypt(str(value).encode()).decode()

    def from_db_value(self, value, expression, connection):
        if value in (None, ''):
            return value
        try:
            return _fernets().decrypt(value.encode()).decode()
        except Exception:
            logger.error("Shifrni ochib bo'lmadi — kalit rotatsiyasi tekshirilsin")
            return None
```

5.3 PII maydonlari ro'yxati

#	Maydon	Model	Toifa	Saqlash muddati	Shifrlash	Log'da	Eksport
1	`phone`	`Order` (chek uchun, ixtiyoriy)	🟠 PII	1 yil	⛔ (indeks kerak)	🔒 maskalangan	✅ so'rov bo'yicha
2	`email`	`RestaurantAdmin`, `PlatformAdmin`	🟠 PII	Hisob faol + 1 yil	⛔	🔒 maskalangan	✅
3	`full_name`	`Staff`	🟠 PII	Ishdan bo'shagach 3 yil (mehnat qonuni)	⛔	🔒	✅
4	`pin_hash`	`Staff`	🔴 Maxfiy	Faol	bcrypt	⛔ hech qachon	⛔
5	`password_hash`	`*Admin`	🔴 Maxfiy	Faol	Argon2id	⛔	⛔
6	`totp_secret_enc`	`PlatformAdmin`	🔴 Maxfiy	Faol	Fernet	⛔	⛔
7	`session_key`	`Order`	⚪ Psevdonim	90 kun	⛔	🔒 qisqartirilgan	⛔
8	`device_id`	Cookie	⚪ Psevdonim	12 soat	Imzolangan	⛔	⛔
9	`ip_address`	`AuditLog`	🟠 PII	1 yil	⛔	✅ (audit maqsadi)	✅
10	`user_agent`	`AuditLog`	⚪	1 yil	⛔	✅	✅
11	`telegram_chat_id`	`Restaurant`, `Staff`	🟠 PII	Faol	⛔	🔒	✅
12	`card_mask` (`****1234`)	`PaymentTransaction`	⚪	5 yil (moliya)	⛔	✅ (allaqachon maskalangan)	✅
13	`provider_txn_id`	`PaymentTransaction`	⚪	5 yil	⛔	✅	✅
14	`ai_chat_message`	`AiConversation`	🟠 PII bo'lishi mumkin	30 kun	⛔	⛔	✅
15	`review_text`	`Review`	🟠	3 yil	⛔	⛔	✅
16	**⛔ Karta raqami / CVV**	—	🔴	**HECH QACHON SAQLANMAYDI**	—	—	—

5.4 Log'da maskalash

```python
# core/logging/filters.py
import re

PATTERNS = [
    # Karta raqami (Luhn'ga qaramay — har 13-19 raqamli ketma-ketlik)
    (re.compile(r'\b(\d{4})[\s\-]?\d{4,11}[\s\-]?(\d{4})\b'), r'\1********\2'),
    # O'zbekiston telefon raqami
    (re.compile(r'\b998(\d{2})(\d{3})(\d{2})(\d{2})\b'), r'998\1***\4'),
    (re.compile(r'\+998[\s\-]?\d{2}[\s\-]?\d{3}[\s\-]?\d{2}[\s\-]?\d{2}'), '+998**_***_**_**'),
    # Email
    (re.compile(r'\b([A-Za-z0-9._%+-]{1,2})[A-Za-z0-9._%+-]*@([A-Za-z0-9.-]+)\b'),
     r'\1***@\2'),
    # JWT / Bearer
    (re.compile(r'eyJ[\w\-]+\.[\w\-]+\.[\w\-]+'), '<JWT>'),
    (re.compile(r'(Bearer|Basic)\s+\S+', re.I), r'\1 <REDACTED>'),
    # Sirlar
    (re.compile(r'("?(?:password|pin|secret|api_key|token|authorization)"?\s*[:=]\s*)'
                r'"?[^",}\s]+', re.I), r'\1"<REDACTED>"'),
]

SENSITIVE_KEYS = {'password', 'pin', 'secret', 'token', 'api_key', 'card',
                  'cvv', 'authorization', 'totp', 'refresh'}


class PiiMaskFilter(logging.Filter):
    def filter(self, record):
        if isinstance(record.msg, str):
            for pat, repl in PATTERNS:
                record.msg = pat.sub(repl, record.msg)
        if record.args:
            record.args = tuple(self._mask(a) for a in record.args)
        return True

    def _mask(self, value):
        if isinstance(value, dict):
            return {k: ('<REDACTED>' if k.lower() in SENSITIVE_KEYS else self._mask(v))
                    for k, v in value.items()}
        if isinstance(value, str):
            for pat, repl in PATTERNS:
                value = pat.sub(repl, value)
        return value
```

```python
# config/settings.py
LOGGING = {
    'version': 1,
    'filters': {'pii': {'()': 'core.logging.filters.PiiMaskFilter'}},
    'handlers': {
        'file':   {'class': 'logging.handlers.WatchedFileHandler',
                   'filename': '/srv/armenu/logs/app.log', 'filters': ['pii']},
        'sentry': {'class': 'sentry_sdk.integrations.logging.EventHandler',
                   'filters': ['pii'], 'level': 'ERROR'},
    },
    'root': {'handlers': ['file', 'sentry'], 'level': 'INFO'},
}

# Sentry — qo'shimcha himoya
sentry_sdk.init(
    dsn=env('SENTRY_DSN'),
    send_default_pii=False,                # ⚡ majburiy False
    before_send=strip_pii_from_event,
    traces_sample_rate=0.05,
)
```

6. TO'LOV XAVFSIZLIGI

6.1 PCI DSS — nega bizga to'liq sertifikat kerak emas



PCI DSS talab	Bizga tegishlimi	Sabab
Karta ma'lumotini shifrlab saqlash	⛔ Yo'q	Umuman saqlamaymiz
Karta tarmog'ini segmentlash	⛔ Yo'q	CDE (Cardholder Data Environment) yo'q
Choraklik ASV skanerlash	🔘 Qisman	To'lov sahifasi bizda emas, lekin redirect bizda
SAQ A (self-assessment)	✅ **Ha**	To'liq autsorsing qilingan e-commerce uchun
Xavfsiz kod amaliyoti	✅ Ha	Umumiy talab
Kirish nazorati va audit	✅ Ha	Umumiy talab
TLS 1.2+	✅ Ha	Umumiy talab

> **Xulosa:** ARMENU **SAQ A** darajasidagi savdogar. Karta ma'lumoti hech qachon bizning tarmoqqa, serverga, log'ga, DBga kirmaydi. To'liq PCI DSS sertifikatsiya **kerak emas**, lekin SAQ A yiliga bir marta to'ldiriladi.

⛔ Qat'iy taqiqlar (CI'da tekshiriladi):

```
# tests/security/test_no_card_data.py
FORBIDDEN_FIELDS = ['card_number', 'pan', 'cvv', 'cvc', 'expiry',
                    'card_holder', 'track_data', 'pin_block']

def test_no_card_fields_in_models():
    bad = []
    for model in apps.get_models():
        for f in model._meta.fields:
            if any(x in f.name.lower() for x in FORBIDDEN_FIELDS):
                if f.name == 'card_mask':          # ✅ maska ruxsat
                    continue
                bad.append(f"{model.__name__}.{f.name}")
    assert not bad, f"🚨 PCI: karta maydonlari topildi: {bad}"


def test_no_card_data_in_code():
    """Kod ichida karta ma'lumoti bilan ishlash urinishi."""
    out = subprocess.run(
        ['grep', '-rniE', r'(card_number|pan\s*=|cvv|cvc2|track2)', 'apps/', 'core/'],
        capture_output=True, text=True)
    assert out.returncode != 0, f"🚨 PCI: {out.stdout}"
```

6.2 Webhook imzo tekshiruvi

Provayder	Imzo usuli	Header	Qo'shimcha
Payme	HTTP Basic: `Paycom:<merchant_key>`	`Authorization`	JSON-RPC; IP allowlist
Click	MD5: `click_trans_id + service_id + SECRET_KEY + merchant_trans_id + amount + action + sign_time`	body `sign_string`	Form-encoded
Uzum	HMAC-SHA256(body, secret)	`X-Signature`	Base64

```
# apps/payments/webhooks.py
import hmac, hashlib, base64, ipaddress

class WebhookVerifier:
    """⚡ Har webhook 4 bosqichdan o'tadi: IP → imzo → replay → summa."""

    @staticmethod
    def verify_payme(request):
        # 1) IP allowlist
        if not ip_allowed(client_ip(request), settings.PAYME_IPS):
            raise WebhookRejected('ip_not_allowed')
        # 2) Basic auth imzosi
        auth = request.headers.get('Authorization', '')
        if not auth.startswith('Basic '):
            raise WebhookRejected('no_auth')
        decoded = base64.b64decode(auth[6:]).decode()
        login, _, key = decoded.partition(':')
        expected = settings.PAYME_MERCHANT_KEY
        # ⚡ Doimiy vaqtli solishtirish — timing attack himoyasi
        if login != 'Paycom' or not hmac.compare_digest(key, expected):
            audit_webhook_failure('payme', 'bad_key', request)
            raise WebhookRejected('bad_signature')
        return json.loads(request.body)

    @staticmethod
    def verify_click(request):
        if not ip_allowed(client_ip(request), settings.CLICK_IPS):
            raise WebhookRejected('ip_not_allowed')
        d = request.POST
        raw = (f"{d['click_trans_id']}{d['service_id']}{settings.CLICK_SECRET_KEY}"
               f"{d['merchant_trans_id']}{d.get('merchant_prepare_id','')}"
               f"{d['amount']}{d['action']}{d['sign_time']}")
        expected = hashlib.md5(raw.encode()).hexdigest()
        if not hmac.compare_digest(expected, d.get('sign_string', '')):
            audit_webhook_failure('click', 'bad_sign', request)
            raise WebhookRejected('bad_signature')
        return d.dict()

    @staticmethod
    def verify_uzum(request):
        if not ip_allowed(client_ip(request), settings.UZUM_IPS):
            raise WebhookRejected('ip_not_allowed')
        sig = request.headers.get('X-Signature', '')
        expected = base64.b64encode(
            hmac.new(settings.UZUM_SECRET.encode(), request.body, hashlib.sha256).digest()
        ).decode()
        if not hmac.compare_digest(expected, sig):
            audit_webhook_failure('uzum', 'bad_hmac', request)
            raise WebhookRejected('bad_signature')
        # 3) Replay: timestamp 5 daqiqadan eski bo'lmasin
        ts = int(request.headers.get('X-Timestamp', 0))
        if abs(time.time() - ts) > 300:
            raise WebhookRejected('stale_request')
        return json.loads(request.body)
```

6.3 ⚡ Summa manipulyatsiyasi oldini olish

```python
# apps/payments/services.py
from decimal import Decimal

def create_payment(order_id, provider, request):
    """
    ⚡ Summa HECH QACHON klientdan olinmaydi.
    Klient faqat order_id va provider yuboradi.
    """
    order = Order.objects.select_for_update().get(
        pk=order_id, restaurant_id=request.restaurant_id)     # ⚡ tenant

    if order.status not in ('confirmed', 'served'):
        raise ValidationError("Bu buyurtma uchun to'lov mumkin emas")

    # ⚡ Summa buyurtma satrlaridan QAYTA hisoblanadi (order.total ga ishonmaymiz)
    computed = sum((i.line_total for i in order.items.all()), Decimal('0'))
    computed += order.service_fee_amount
    computed -= order.discount_amount
    if computed != order.total:
        alert_oncall(f"🚨 Buyurtma #{order.id} summasi mos emas: "
                     f"total={order.total}, hisoblangan={computed}", severity='P1')
        raise ValidationError("Buyurtma summasida nomuvofiqlik")

    if computed <= 0 or computed > Decimal('100000000'):      # sanity
        raise ValidationError("Noto'g'ri summa")

    txn = PaymentTransaction.objects.create(
        restaurant_id=order.restaurant_id, order=order,
        provider=provider, amount=computed, status='pending',
        idempotency_key=f'{order.id}:{computed}',
    )
    gateway = get_gateway(provider)
    txn.checkout_url = gateway.create_invoice(txn)             # tiyin/so'm konvertatsiya
    txn.save(update_fields=['checkout_url'])
    return txn


def apply_webhook(provider, data):
    """Webhook kelganda — summa MOS kelishi shart."""
    with transaction.atomic():
        txn = PaymentTransaction.objects.select_for_update().get(
            provider=provider, provider_txn_id=data['txn_id'])

        # ⚡ Idempotency: allaqachon qayta ishlangan bo'lsa — hech narsa qilinmaydi
        if txn.status in ('paid', 'cancelled'):
            return txn

        # ⚡ Summa tekshiruvi — provayder boshqa summa yubORSA REJECT
        remote_amount = Decimal(data['amount']) / 100          # tiyin → so'm
        if remote_amount != txn.amount:
            audit_payment_mismatch(txn, remote_amount)
            alert_oncall(f"🚨 To'lov summasi mos emas! txn={txn.id} "
                         f"kutilgan={txn.amount} kelgan={remote_amount}", severity='P1')
            raise WebhookRejected('amount_mismatch')

        txn.status = 'paid'
        txn.paid_at = timezone.now()
        txn.card_mask = data.get('card_mask', '')[:20]         # faqat maska
        txn.save()
        txn.order.mark_paid()
        publish('payment.paid', {'txn_id': txn.id},
                restaurant_id=txn.restaurant_id)               # → fiskal, WS
    return txn
```

To'lov xavfsizligi chek-listi:

#	Nazorat	Amalga oshirish
1	Summa serverda hisoblanadi	`create_payment` — satrlardan qayta hisob
2	Klient summa yubora olmaydi	Serializer'da `amount` maydoni yo'q
3	Webhook summasi solishtiriladi	`apply_webhook` — mos kelmasa reject + P1
4	Idempotent	`provider_txn_id` unique + `select_for_update`
5	Replay himoyasi	Timestamp + ishlatilgan `txn_id`
6	IP allowlist	Har provayder uchun ro'yxat
7	Imzo doimiy vaqtli	`hmac.compare_digest`
8	Tenant chegarasi	`restaurant_id` har so'rovda
9	Refund limiti	Kunlik summa cheki (rol bo'yicha)
10	Reconciliation	Har 5 daqiqada `check_transaction` (healer #14)
11	Har amal audit	`AuditLog` + moliyaviy alohida jurnal
12	2× to'lov himoyasi	Buyurtma `paying` da lock (saga S1)

---

7. KIRISH VALIDATSIYASI

7.1 Kirish nuqtalari xaritasi

#	Kirish nuqtasi	Manba	Validatsiya	Risk
1	QR token (`/t/<token>`)	URL	Regex `^[A-Za-z0-9_-]{20,24}$` + DB lookup + rate limit	🔴
2	Buyurtma API	JSON body	DRF serializer, tenant-scoped FK, min/max	🔴
3	To'lov webhook	Tashqi POST	Imzo + IP + summa + replay	🔴
4	Fayl yuklash (3D)	multipart	MIME + magic + hajm + validator + kvota	🔴
5	Fayl yuklash (rasm)	multipart	Pillow verify + qayta kodlash + EXIF tozalash	🔴
6	Menyu qidiruv	Query string	Uzunlik ≤ 100, PostgreSQL FTS (regex yo'q)	🟢
7	AI chat xabari	JSON	Uzunlik ≤ 500, prompt injection filtri	🔴
8	Admin forma	JSON/form	Serializer + oq ro'yxat	🔴
9	WS xabar	JSON	Sxema + hajm ≤ 16 KB + rate	🔴
10	Telegram webhook	Tashqi POST	`secret_token` header	🟢
11	Til/valyuta parametri	Query/header	Enum (`uz`, `ru`, `en`)	🟢
12	Pagination	Query	`limit ≤ 100`, `offset ≤ 10000`	🟢
13	CSV/Excel import (menyu)	Fayl	Hajm, qator soni, formula injection (`=`, `+`, `-`, `@`)	🔴

7.2 Fayl yuklash — 3D model

```
# apps/ar3d/validators.py
import magic, struct
from django.core.exceptions import ValidationError

ALLOWED_3D = {
    'glb':  {'mime': {'model/gltf-binary', 'application/octet-stream'},
             'magic': b'glTF', 'max_mb': 30},
    'gltf': {'mime': {'model/gltf+json', 'application/json'},
             'magic': b'{', 'max_mb': 10},
    'usdz': {'mime': {'model/vnd.usdz+zip', 'application/zip'},
             'magic': b'PK\x03\x04', 'max_mb': 30},
}
ALLOWED_IMG = {'jpg': 10, 'jpeg': 10, 'png': 10, 'webp': 10}


def validate_3d_file(uploaded, restaurant_id):
    # ── 1. Kengaytma ──────────────────────────────
    ext = uploaded.name.rsplit('.', 1)[-1].lower()
    if ext not in ALLOWED_3D:
        raise ValidationError(f"Ruxsat etilmagan format: .{ext}")
    spec = ALLOWED_3D[ext]

    # ── 2. Hajm ───────────────────────────────────
    if uploaded.size > spec['max_mb'] * 1024 * 1024:
        raise ValidationError(f"Fayl {spec['max_mb']} MB dan katta")
    if uploaded.size < 100:
        raise ValidationError("Fayl juda kichik / bo'sh")

    # ── 3. MIME (kutubxona bilan, kengaytmaga ishonmaymiz) ──
    head = uploaded.read(8192)
    uploaded.seek(0)
    mime = magic.from_buffer(head, mime=True)
    if mime not in spec['mime']:
        raise ValidationError(f"Fayl turi mos emas: {mime}")

    # ── 4. ⚡ Magic bytes ─────────────────────────
    if not head.lstrip().startswith(spec['magic']):
        raise ValidationError("Fayl imzosi buzilgan (magic bytes)")

    # ── 5. GLB tuzilmasi (chunk header) ───────────
    if ext == 'glb':
        magic_, version, length = struct.unpack('<4sII', head[:12])
        if version != 2:
            raise ValidationError("Faqat glTF 2.0 qo'llab-quvvatlanadi")
        if length != uploaded.size:
            raise ValidationError("GLB uzunligi mos emas — fayl buzilgan")

    # ── 6. USDZ — zip bomb himoyasi ───────────────
    if ext == 'usdz':
        import zipfile
        uploaded.seek(0)
        with zipfile.ZipFile(uploaded) as zf:
            total = sum(i.file_size for i in zf.infolist())
            if total > 200 * 1024 * 1024:
                raise ValidationError("Arxiv ochilganda juda katta (zip bomb)")
            if len(zf.infolist()) > 500:
                raise ValidationError("Arxivda juda ko'p fayl")
            for info in zf.infolist():
                if info.filename.startswith('/') or '..' in info.filename:
                    raise ValidationError("Xavfli fayl yo'li (path traversal)")
        uploaded.seek(0)

    # ── 7. Restoran kvotasi ───────────────────────
    used = Model3D.objects.filter(restaurant_id=restaurant_id).aggregate(
        s=models.Sum('file_size'))['s'] or 0
    quota = Restaurant.objects.get(pk=restaurant_id).storage_quota_bytes
    if used + uploaded.size > quota:
        raise ValidationError("Saqlash kvotasi tugadi")

    # ── 8. Chuqur validatsiya — workerda (sandbox) ──
    return True     # gltf-validator worker'da ishlaydi, natija DB'ga yoziladi


def validate_image(uploaded):
    from PIL import Image
    ext = uploaded.name.rsplit('.', 1)[-1].lower()
    if ext not in ALLOWED_IMG:
        raise ValidationError("Faqat JPG, PNG, WEBP")
    if uploaded.size > ALLOWED_IMG[ext] * 1024 * 1024:
        raise ValidationError("Rasm juda katta")
    try:
        img = Image.open(uploaded)
        img.verify()                        # struktura tekshiruvi
        uploaded.seek(0)
        img = Image.open(uploaded)
        if img.width > 8000 or img.height > 8000:
            raise ValidationError("Rasm o'lchami juda katta (decompression bomb)")
    except Exception:
        raise ValidationError("Rasm buzilgan yoki noto'g'ri format")

    # ⚡ Qayta kodlash — EXIF, ICC, yashirin payload olib tashlanadi
    uploaded.seek(0)
    clean = Image.open(uploaded).convert('RGB')
    out = io.BytesIO()
    clean.save(out, format='WEBP', quality=85, method=4)
    out.seek(0)
    return out
```

Saqlash qoidalari:

Qoida	Amalga oshirish
Fayl nomi klientdan olinmaydi	`f'{rid}/{uuid4()}.{ext}'`
Yo'l tenant bilan bo'linadi	`media/3d/{restaurant_id}/...`
Alohida domendan beriladi	`cdn.armenu.uz` (cookie'siz, sandbox)
`Content-Type` majburiy	S3 metadata'da, `X-Content-Type-Options: nosniff`
Bajariladigan fayl bo'lmaydi	`.html`, `.svg`, `.js` — 3D/rasm yo'lida taqiqlangan
Presigned URL	1 soat, faqat GET

7.3 SQL injection

```
# ❌ HECH QACHON
Dish.objects.raw(f"SELECT * FROM menu_dish WHERE name LIKE '%{q}%'")
cursor.execute(f"SELECT * FROM orders WHERE id = {order_id}")
Dish.objects.extra(where=[f"price > {min_price}"])

# ✅ DOIM
Dish.objects.filter(name__icontains=q, restaurant_id=rid)
cursor.execute("SELECT * FROM orders WHERE id = %s AND restaurant_id = %s",
               [order_id, rid])

# ✅ Murakkab so'rov kerak bo'lsa — parametrlangan + tenant majburiy
with connections['read'].cursor() as cur:
    cur.execute("""
        SELECT d.id, d.name, count(oi.id) AS cnt
        FROM menu_dish d
        LEFT JOIN orders_orderitem oi ON oi.dish_id = d.id
        WHERE d.restaurant_id = %s AND oi.created_at >= %s
        GROUP BY d.id ORDER BY cnt DESC LIMIT %s
    """, [rid, since, min(limit, 100)])
```

CI gate: `grep -rn "\.raw(\|\.extra(\|cursor.execute(f\"" apps/` → build fail.

7.4 XSS va CSP

```
# Django template — avtomatik escape YOQILGAN (o'zgartirilmaydi)
# ❌ TAQIQLANGAN:  {{ user_input|safe }}  ·  mark_safe(user_input)
# ✅ HTML kerak bo'lsa — bleach bilan tozalash:

import bleach

ALLOWED_TAGS = ['b', 'i', 'em', 'strong', 'p', 'br', 'ul', 'ol', 'li']
ALLOWED_ATTRS = {}

def clean_html(raw):
    return bleach.clean(raw, tags=ALLOWED_TAGS, attributes=ALLOWED_ATTRS, strip=True)
```

```
// static/js/render.js — ❌ innerHTML ishlatilmaydi
// ✗  el.innerHTML = `<div>${dish.name}</div>`;
// ✓
const div = document.createElement('div');
div.textContent = dish.name;            // avtomatik escape
el.appendChild(div);
```

CSP nonce (inline script kerak bo'lsa):

```
# core/middleware/csp.py
class CspNonceMiddleware:
    def __call__(self, request):
        request.csp_nonce = secrets.token_urlsafe(16)
        response = self.get_response(request)
        response['Content-Security-Policy'] = (
            "default-src 'self'; "
            f"script-src 'self' 'nonce-{request.csp_nonce}' https://cdn.armenu.uz; "
            "style-src 'self' 'unsafe-inline'; "
            "img-src 'self' data: blob: https://cdn.armenu.uz; "
            "connect-src 'self' wss://armenu.uz https://cdn.armenu.uz; "
            "frame-ancestors 'none'; object-src 'none'; base-uri 'self'; "
            "form-action 'self'; upgrade-insecure-requests; "
            "report-uri /csp-report/"
        )
        return response
```

7.5 CSRF

Yo'l	Himoya
Django forma (admin)	`{% csrf_token %}` + `CsrfViewMiddleware`
DRF sessiya auth (mijoz)	`X-CSRFToken` header majburiy
DRF JWT auth (xodim)	CSRF kerak emas (Authorization header, cookie emas)...
...lekin refresh cookie	`SameSite=Strict` + `/v1/staff/refresh/` da CSRF token
Webhook	CSRF exempt + imzo tekshiruvi (§6.2)
WS	`Origin` header tekshiruvi (`AllowedHostsOriginValidator`)

```
# config/settings.py
CSRF_COOKIE_SECURE   = True
CSRF_COOKIE_HTTPONLY = False          # JS o'qishi kerak (SPA header uchun)
CSRF_COOKIE_SAMESITE = 'Lax'
CSRF_TRUSTED_ORIGINS = ['https://armenu.uz', 'https://*.armenu.uz']
SESSION_COOKIE_SECURE   = True
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = 'Lax'
```

7.6 Prompt injection (AI chat)

```
# apps/ai/guard.py
INJECTION_PATTERNS = [
    r'ignore\s+(all\s+)?previous\s+instructions',
    r'system\s*:',  r'</?(system|assistant)>',
    r'you\s+are\s+now',  r'oldingi\s+ko[\'']rsatmalar',
    r'reveal\s+(your\s+)?(prompt|instructions)',
]

def sanitize_ai_input(text: str) -> str:
    if len(text) > 500:
        raise ValidationError("Xabar juda uzun")
    for pat in INJECTION_PATTERNS:
        if re.search(pat, text, re.I):
            metrics.inc('armenu_ai_injection_blocked_total')
            raise ValidationError("So'rov qabul qilinmadi")
    return text.replace('\x00', '').strip()


def build_prompt(user_msg, restaurant_id):
    """Foydalanuvchi matni ALOHIDA blokda, tizim ko'rsatmasi bilan aralashmaydi.
    AI javobi hech qachon kod sifatida bajarilmaydi va DB so'rovi qurmaydi."""
    return [
        {'role': 'system', 'content': SYSTEM_PROMPT + menu_context(restaurant_id)},
        {'role': 'user',   'content': f'<user_question>{sanitize_ai_input(user_msg)}'
                                      f'</user_question>'},
    ]
```

8. RATE LIMITING

8.1 Limitlar jadvali

#	Endpoint / amal	Limit	Kalit	Jazo	Daraja
1	`GET /t/<token>` (QR kirish)	60/daq	IP	429, 60 s	○
2	`GET /v1/menu/`	120/daq	IP	429, 60 s	○
3	`GET /v1/ar/<id>/`	200/daq	IP	429	○
4	`POST /v1/orders/`	10/daq	session	429 + shubhali flag	◎
5	`POST /v1/orders/` (stol bo'yicha)	30/soat	table_id	429 + admin xabar	◎
6	`POST /v1/call-waiter/`	1/daq	table_id	Jim rad (UI "yuborildi")	○
7	`POST /v1/ai/chat/`	10/5daq	session	429 + FAQ taklifi	◎
8	`POST /v1/ai/chat/` (restoran)	500/soat	restaurant_id	Degraded rejim	◎
9	`POST /v1/staff/login/`	5/15daq	IP + PIN prefiks	15 daq blok	●
10	`POST /v1/admin/login/`	5/15daq	IP + email	15 daq → 24 soat	●
11	`POST /platform/login/`	3/15daq	IP	1 soat blok + alert	●
12	`POST /v1/payments/`	5/daq	session	429	●
13	Webhook (har provayder)	300/daq	provayder IP	429 (provayder retry qiladi)	○
14	`POST /v1/media/upload/`	20/soat	restaurant_id	429	◎
15	Admin API (umumiy)	300/daq	admin_id	429	○
16	`GET /v1/reports/`	30/daq	admin_id	429	○
17	WS ulanish	10/daq	IP	Ulanish rad (4429)	◎
18	WS xabar	60/daq	connection	Xabar tashlanadi	◎
19	Parol tiklash	3/soat	email	Jim rad	◎
20	Eksport (Excel/PDF)	10/soat	admin_id	429	○

8.2 Redis amalga oshirish

```
# core/ratelimit.py
"""
Sliding-window rate limiter (Redis sorted set).
Fixed-window'dan aniqroq: chegara atrofida 2× burst bo'lmaydi.
"""
import time
from django.core.cache import caches

_r = caches['default'].client.get_client(write=True)

_SCRIPT = _r.register_script("""
local key   = KEYS[1]
local now   = tonumber(ARGV[1])
local window= tonumber(ARGV[2])
local limit = tonumber(ARGV[3])
local id    = ARGV[4]

redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
local count = redis.call('ZCARD', key)
if count >= limit then
  local oldest = redis.call('ZRANGE', key, 0, 0, 'WITHSCORES')
  return {0, count, math.ceil(oldest[2] + window - now)}
end
redis.call('ZADD', key, now, id)
redis.call('EXPIRE', key, window + 1)
return {1, count + 1, 0}
""")


class RateLimit:
    def __init__(self, name, limit, window, penalty=None):
        self.name, self.limit, self.window = name, limit, window
        self.penalty = penalty or window

    def check(self, key_value) -> tuple[bool, int]:
        """(ruxsat, retry_after)"""
        blocked_key = f'rl_block:{self.name}:{key_value}'
        ttl = _r.ttl(blocked_key)
        if ttl and ttl > 0:
            return False, ttl

        key = f'rl:{self.name}:{key_value}'
        now = time.time()
        try:
            ok, count, retry = _SCRIPT(keys=[key],
                                       args=[now, self.window, self.limit,
                                             f'{now}:{secrets.token_hex(4)}'])
        except Exception:
            # ⚠ Redis o'chgan — FAIL-OPEN (xizmat to'xtamasin)
            metrics.inc('armenu_ratelimit_fail_open_total', labels={'name': self.name})
            return True, 0

        if not ok:
            metrics.inc('armenu_ratelimit_hit_total', labels={'name': self.name})
            if self.penalty > self.window:
                _r.setex(blocked_key, self.penalty, 1)
            return False, int(retry)
        return True, 0


LIMITS = {
    'qr':           RateLimit('qr',           60,  60),
    'menu':         RateLimit('menu',        120,  60),
    'order_session': RateLimit('order_session', 10,  60),
    'order_table':  RateLimit('order_table',  30, 3600),
    'ai_chat':      RateLimit('ai_chat',      10,  300),
    'staff_login':  RateLimit('staff_login',   5,  900, penalty=900),
    'admin_login':  RateLimit('admin_login',   5,  900, penalty=900),
    'platform_login':RateLimit('platform_login', 3,  900, penalty=3600),
    'payment':      RateLimit('payment',       5,   60),
    'upload':       RateLimit('upload',       20, 3600),
    'ws_connect':   RateLimit('ws_connect',   10,   60),
}


def rate_limit(name, key_fn):
    """Dekorator: @rate_limit('order_session', lambda r: r.session.session_key)"""
    def deco(view):
        @functools.wraps(view)
        def wrapper(request, *a, **kw):
            ok, retry = LIMITS[name].check(key_fn(request))
            if not ok:
                audit(request, 'ratelimit.exceeded', extra={'limit': name})
                resp = JsonResponse(
                    {'error': 'rate_limited',
                     'detail': f"Juda ko'p so'rov. {retry} soniyadan keyin urining"},
                    status=429)
                resp['Retry-After'] = str(retry)
                return resp
            return view(request, *a, **kw)
        return wrapper
    return deco
```

8.3 DDoS himoya (Cloudflare)

Qatlam	Chora	Sozlama
L3/L4	Volumetric DDoS	Cloudflare avtomatik (Anycast)
L7	So'rov toshqini	WAF Managed Rules (OWASP Core)
L7	Bot	Bot Fight Mode: `cf.client.bot` → JS challenge
L7	Firewall Rules	`/platform/*` → faqat UZ + IP allowlist
L7	Rate Limiting Rules	`/v1/orders/` → 20/daq/IP (bizning limitdan yuqori — ikkinchi to'siq)
L7	Under Attack	Insidentda qo'lda yoqiladi (5 s JS challenge)
Origin	Cloudflare-only	nginx faqat Cloudflare IP diapazonlarini qabul qiladi

```
# /etc/nginx/snippets/cloudflare.conf — origin himoyasi
# ⚡ Faqat Cloudflare orqali kelgan trafik qabul qilinadi
set_real_ip_from 173.245.48.0/20;
set_real_ip_from 103.21.244.0/22;
# ... (to'liq ro'yxat cron bilan yangilanadi: ops/cf_ips.sh)
real_ip_header CF-Connecting-IP;

geo $cf_ip { default 0; 173.245.48.0/20 1; 103.21.244.0/22 1; /* ... */ }

server {
    if ($cf_ip = 0) { return 403; }     # Cloudflare'dan tashqari — rad
    ...
}
```

```
# Cloudflare Firewall Rules (dashboard)
1. (http.request.uri.path contains "/platform/") and (ip.geoip.country ne "UZ")
   → Block
2. (http.request.uri.path contains "/webhook/") and not (ip.src in $payment_ips)
   → Block
3. (cf.threat_score gt 30) and (http.request.uri.path contains "/v1/orders/")
   → Managed Challenge
4. (http.user_agent contains "python-requests" or "curl") and
   (http.request.uri.path contains "/v1/menu/")
   → Managed Challenge          # scraping himoyasi
```

9. AUDIT VA MONITORING

9.1 AuditLog modeli

```python
# core/audit/models.py
class AuditLog(models.Model):
    """
    ⚡ FAQAT QO'SHILADI (append-only).
    DELETE/UPDATE huquqi hech kimda yo'q — DB darajasida ham.
    """
    id            = models.BigAutoField(primary_key=True)
    ts            = models.DateTimeField(auto_now_add=True, db_index=True)
    restaurant_id = models.IntegerField(null=True, db_index=True)

    # KIM
    actor_type    = models.CharField(max_length=20)     # guest|waiter|cook|cashier|admin|platform|system
    actor_id      = models.IntegerField(null=True)
    actor_label   = models.CharField(max_length=100, blank=True)   # "Aziz (ofitsiant)"
    session_id    = models.CharField(max_length=64, blank=True)

    # NIMA
    action        = models.CharField(max_length=64, db_index=True)  # 'order.cancel'
    object_type   = models.CharField(max_length=64, blank=True)
    object_id     = models.CharField(max_length=64, blank=True)
    changes       = models.JSONField(default=dict)      # {"status": ["new","cancelled"]}

    # QAYERDAN
    ip            = models.GenericIPAddressField(null=True)
    user_agent    = models.CharField(max_length=300, blank=True)
    request_id    = models.CharField(max_length=36, blank=True)

    # QANDAY
    severity      = models.CharField(max_length=10, default='info')  # info|warn|high|critical
    result        = models.CharField(max_length=10, default='ok')    # ok|denied|error

    class Meta:
        indexes = [
            models.Index(fields=['restaurant_id', '-ts']),
            models.Index(fields=['action', '-ts']),
            models.Index(fields=['actor_type', 'actor_id', '-ts']),
            models.Index(fields=['severity', '-ts'],
                         condition=models.Q(severity__in=['high', 'critical']),
                         name='audit_severe_idx'),
        ]
        # ⚡ Oylik partition (03_MALUMOT_ARXITEKTURASI.md §4)
```

```sql
-- ⚡ DB darajasida o'zgartirishni taqiqlash
REVOKE UPDATE, DELETE, TRUNCATE ON core_auditlog FROM armenu_app;
GRANT  INSERT, SELECT               ON core_auditlog TO   armenu_app;

CREATE RULE audit_no_update AS ON UPDATE TO core_auditlog DO INSTEAD NOTHING;
CREATE RULE audit_no_delete AS ON DELETE TO core_auditlog DO INSTEAD NOTHING;
```

9.2 Nima yoziladi

Toifa	Hodisalar	Severity
Auth	`*.login.success`, `*.login.failed`, `*.login.locked`, `*.logout`, `*.refresh.reuse_detected`, `platform.2fa.replay`	info / high / critical
Buyurtma	`order.create`, `order.cancel`, `order.status_change`, `order.item_remove`	info / warn
Moliya	`payment.create`, `payment.paid`, `payment.refund`, `discount.apply`, `payment.mismatch`	info / high
Menyu	`dish.create`, `dish.update`, `dish.price_change`, `dish.delete`, `stoplist.toggle`	info / warn
Xodim	`staff.create`, `staff.pin_change`, `staff.deactivate`, `staff.role_change`	warn
Sozlama	`restaurant.settings_change`, `payment_provider.change`, `ofd.credentials_change`	high
Xavfsizlik	`authz.denied`, `authz.cross_tenant_object`, `ratelimit.exceeded`, `webhook.rejected`, `file.rejected`	high / critical
Platforma	`platform.restaurant_create`, `platform.tariff_change`, `platform.impersonate`	high
Ma'lumot	`data.export`, `data.bulk_delete`, `pii.access`	high

```python
# core/audit/service.py
def audit(request, action, *, object_type='', object_id='', changes=None,
          severity='info', result='ok', restaurant_id=None, actor_id=None,
          extra=None):
    """Har muhim amalda chaqiriladi. ⚡ Hech qachon xato tashlamaydi."""
    try:
        AuditLog.objects.create(
            restaurant_id=restaurant_id or getattr(request, 'restaurant_id', None),
            actor_type=getattr(request, 'role', 'anonymous'),
            actor_id=actor_id or getattr(getattr(request, 'actor', None), 'id', None),
            actor_label=str(getattr(request, 'actor', '') or '')[:100],
            session_id=(getattr(request, 'session', None) and
                        request.session.session_key or '')[:64],
            action=action, object_type=object_type, object_id=str(object_id)[:64],
            changes=changes or {},
            ip=client_ip(request),
            user_agent=request.META.get('HTTP_USER_AGENT', '')[:300],
            request_id=getattr(request, 'request_id', ''),
            severity=severity, result=result,
        )
        if severity in ('high', 'critical'):
            metrics.inc('armenu_audit_severe_total', labels={'action': action})
        if severity == 'critical':
            alert_oncall(f"🚨 Xavfsizlik: {action} · {client_ip(request)}", severity='P1')
    except Exception:
        logger.exception("[AUDIT] yozib bo'lmadi: %s", action)   # ⚡ so'rovni buzmaydi
```

9.3 Saqlash muddatlari

Ma'lumot	Muddat	Sabab	Keyin
AuditLog — moliyaviy	**5 yil**	Soliq kodeksi	S3 WORM arxiv
AuditLog — xavfsizlik (high/critical)	3 yil	Insident tergovi	S3 arxiv
AuditLog — oddiy (info)	1 yil	Operatsion	Partition drop
Fiskal cheklar	5 yil	Soliq talabi	S3 WORM
To'lov tranzaksiyalari	5 yil	Moliya	Arxiv
Buyurtmalar	3 yil	Biznes analitika	Agregatga aylantiriladi
Mijoz sessiyasi	90 kun	Texnik	O'chiriladi
AI suhbat	30 kun	Sifat nazorati	O'chiriladi
Nginx access log	90 kun	Xavfsizlik	S3
Ilova log	30 kun	Debug	S3 (90 kun)
Metrikalar	1 yil	Trend	Downsampled

9.4 Shubhali harakat aniqlash

```
# core/security/anomaly.py
"""Har 5 daqiqada AuditLog'ni tahlil qiladi."""

RULES = [
    # (nom, SQL/ORM, chegara, severity, xabar)
    ('brute_force_pin',
     lambda: AuditLog.objects.filter(action='staff.login.failed',
                                     ts__gte=ago(minutes=15))
             .values('ip').annotate(n=Count('id')).filter(n__gte=15),
     'high', "PIN brute-force urinishi"),

    ('cross_tenant_probing',
     lambda: AuditLog.objects.filter(action='authz.cross_tenant_object',
                                     ts__gte=ago(minutes=10))
             .values('ip').annotate(n=Count('id')).filter(n__gte=3),
     'critical', "🔴 Cross-tenant zondlash"),

    ('token_enumeration',
     lambda: AuditLog.objects.filter(action='qr.token_invalid', ts__gte=ago(minutes=10))
             .values('ip').annotate(n=Count('id')).filter(n__gte=50),
     'high', "QR token taxmin qilish"),

    ('unusual_refund_volume',
     lambda: AuditLog.objects.filter(action='payment.refund', ts__gte=ago(hours=1))
             .values('actor_id', 'restaurant_id').annotate(n=Count('id')).filter(n__gte=10),
     'high', "Ko'p qaytarish — ichki o'g'irlik ehtimoli"),

    ('after_hours_admin',
     lambda: AuditLog.objects.filter(actor_type='admin', ts__gte=ago(minutes=5),
                                     ts__hour__in=[1, 2, 3, 4, 5])
             .exclude(action__startswith='order.'),
     'warn', "Ish vaqtidan tashqari admin faoliyati"),

    ('price_change_burst',
     lambda: AuditLog.objects.filter(action='dish.price_change', ts__gte=ago(minutes=10))
             .values('restaurant_id').annotate(n=Count('id')).filter(n__gte=50),
     'warn', "Ommaviy narx o'zgarishi"),

    ('impersonation',
     lambda: AuditLog.objects.filter(action='platform.impersonate', ts__gte=ago(minutes=5)),
     'high', "Platforma admin restoran nomidan kirdi"),

    ('mass_export',
     lambda: AuditLog.objects.filter(action='data.export', ts__gte=ago(hours=1))
             .values('actor_id').annotate(n=Count('id')).filter(n__gte=20),
     'high', "Ommaviy ma'lumot eksporti"),

    ('new_ip_admin',
     lambda: _admin_new_ip_events(),
     'warn', "Admin yangi IP dan kirdi"),

    ('webhook_signature_failures',
     lambda: AuditLog.objects.filter(action='webhook.rejected', ts__gte=ago(minutes=10))
             .values('ip').annotate(n=Count('id')).filter(n__gte=5),
     'critical', "🔴 Soxta to'lov webhook urinishi"),
]


def scan():
    for name, query, severity, message in RULES:
        hits = list(query())
        if not hits:
            continue
        metrics.inc('armenu_anomaly_total', labels={'rule': name})
        alert_oncall(
            f"{'🔴' if severity == 'critical' else '🟠'} {message}\n"
            f"Qoida: {name} · Hodisalar: {len(hits)}\n"
            f"Tafsilot: {hits[:5]}\n"
            f"QILISH: /ops/audit/?rule={name} sahifasini oching",
            severity='P1' if severity == 'critical' else 'P2')
```

---

10. SIRLAR BOSHQARUVI

10.1 Saqlash

```
┌──────────────────────────────────────────────────────┐
│  S0–S2 bosqich:  .env fayl                           │
│    /srv/armenu/.env  ·  chmod 600  ·  chown armenu:armenu │
│    systemd: EnvironmentFile=/srv/armenu/.env         │
│    Git'da: .env.example (qiymatsiz shablon)          │
│    Backup: ops/secrets.enc (SOPS + age), git repo'da │
├──────────────────────────────────────────────────────┤
│  S3+ bosqich:  HashiCorp Vault                       │
│    Dinamik DB credentials (TTL 1 soat)               │
│    Transit engine (maydon shifrlash)                 │
│    Audit log (kim qaysi sirni o'qidi)                │
└──────────────────────────────────────────────────────┘
```

10.2 Sirlar ro'yxati va rotatsiya

#	Sir	Qayerda	Rotatsiya	Rotatsiya ta'siri
1	`SECRET_KEY` (Django)	.env	90 kun	Barcha sessiya bekor (kechasi)
2	`JWT_SECRET`	.env	90 kun	Barcha access token bekor (15 daq)
3	`DEVICE_COOKIE_SECRET`	.env	90 kun	Mijoz cookie'lari bekor (qayta skanerlash)
4	`PIN_LOOKUP_KEY`	.env	⚠ 1 yil	Barcha `pin_lookup` qayta hisoblanadi (migratsiya)
5	`FIELD_ENCRYPTION_KEYS`	.env (ro'yxat)	90 kun	MultiFernet — eski kalit ochish uchun qoladi
6	`DB_PASSWORD`	.env / Vault	90 kun	PgBouncer reload
7	`REDIS_PASSWORD`	.env	90 kun	Servis restart
8	`PAYME_MERCHANT_KEY`	.env	Provayder talabi	Provayder kabinetida sinxron
9	`CLICK_SECRET_KEY`	.env	Provayder talabi	Sinxron
10	`UZUM_SECRET`	.env	Provayder talabi	Sinxron
11	`OFD_API_TOKEN` (×2)	.env	180 kun	OFD kabinetida
12	AI API kalitlari (×6)	.env	90 kun	Provayder kabinetida
13	`TELEGRAM_BOT_TOKEN`	.env	Kerak bo'lganda	BotFather
14	`S3_ACCESS_KEY` / `SECRET`	.env	90 kun	IAM'da yangi kalit → eski o'chirish
15	`BACKUP_AGE_PRIVATE_KEY`	⚡ Offline seyf (2 nusxa)	1 yil	Yangi backuplar yangi kalit bilan
16	SSH deploy kaliti	CI secret	90 kun	`authorized_keys` yangilanishi
17	TLS sertifikat	certbot	90 kun (avto)	nginx reload
18	`SENTRY_DSN`	.env	Kerak bo'lganda	—

```
#!/usr/bin/env bash
# ops/secrets/rotate.sh — 90 kunlik rotatsiya
set -euo pipefail
SECRET_NAME=$1
NEW=$(openssl rand -base64 48 | tr -d '\n')

# 1) Yangi qiymatni QO'SHAMIZ (eskisi hali ishlaydi — MultiFernet uslubi)
python ops/secrets/add_key.py "$SECRET_NAME" "$NEW"
sudo systemctl reload armenu

# 2) Grace davri: 24 soat (eski token/sessiya tugashi uchun)
echo "⏳ 24 soat kutiladi (eski kalit bilan yaratilgan tokenlar tugasin)"

# 3) Eskisini olib tashlash (alohida skript, cron bilan +1 kun)
# python ops/secrets/drop_old_key.py "$SECRET_NAME"

# 4) Audit
python manage.py audit_log --action=secret.rotated --object="$SECRET_NAME"
```

10.3 Git'ga tushmaslik — pre-commit hook

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.0
    hooks:
      - id: gitleaks

  - repo: https://github.com/Yelp/detect-secrets
    rev: v1.5.0
    hooks:
      - id: detect-secrets
        args: ['--baseline', '.secrets.baseline']

  - repo: local
    hooks:
      - id: no-env-files
        name: "🚫 .env fayllari commit qilinmasin"
        entry: bash -c 'git diff --cached --name-only | grep -E "^\.env$|\.env\.(prod|local)$" && { echo "🚫 .env commit qilinmoqda!"; exit 1; } || exit 0'
        language: system
        pass_filenames: false

      - id: no-hardcoded-secrets
        name: "🔑 Kodda qattiq yozilgan sir"
        entry: bash -c 'grep -rnEi "(SECRET_KEY|API_KEY|PASSWORD|TOKEN)\s*=\s*[\x27\"][A-Za-z0-9+/=_-]{16,}" --include="*.py" apps/ core/ config/ && exit 1 || exit 0'
        language: system
        pass_filenames: false

      - id: tenant-isolation-quick
        name: "⚡ Tenant izolyatsiya tez tekshiruv"
        entry: bash -c 'grep -rn "objects\.all()" apps/*/views*.py | grep -v TenantScoped && exit 1 || exit 0'
        language: system
        pass_filenames: false
```

```
# .gitignore
.env
.env.*
!.env.example
*.pem
*.key
!*.pub
ops/secrets/*.dec
/srv/
backup/
```

Agar sir git'ga tushib qolsa:

```
[1] DARHOL sirni ROTATSIYA qiling (repo tozalashdan OLDIN!)
    → sir allaqachon oshkor; git tarixini tozalash yetarli emas
[2] git filter-repo --invert-paths --path <fayl>
[3] Force push + barcha fork/klonlar haqida xabar
[4] GitHub Support'ga murojaat (kesh tozalash)
[5] AuditLog: kim, qachon, qaysi sirni oshkor qildi
[6] Post-mortem
```

---

11. O'ZBEKISTON QONUNCHILIGI

11.1 Shaxsiy ma'lumotlar to'g'risidagi qonun (O'RQ-547, 02.07.2019)

Talab	Modda	ARMENU'da qanday bajariladi
Rozilik olish	20-modda	QR sahifada birinchi kirishda banner: "Davom etish orqali siz [Maxfiylik siyosati]ga rozilik bildirasiz". Telefon so'ralganda alohida checkbox
Maqsadni ko'rsatish	5-modda	Maxfiylik siyosatida: nima, nima uchun, qancha vaqt
Minimallik	5-modda	⚡ Mijozdan ism/telefon **so'ralmaydi**; telefon faqat chek uchun ixtiyoriy
⚡ Lokalizatsiya	27¹-modda	🟢 **Serverlar O'zbekistonda** — DB, backup, log. Uzbektelecom/UZINFOCOM DC
Xavfsizlik choralari	24-modda	Shifrlash, kirish nazorati, audit — shu hujjat
Kirish huquqi	19-modda	`GET /v1/me/data/` — mijoz o'z ma'lumotini oladi (JSON)
Tuzatish huquqi	19-modda	Admin panel / qo'llab-quvvatlash orqali
O'chirish huquqi	22-modda	`DELETE /v1/me/data/` — 30 kun ichida bajariladi
Operator ro'yxati	7-modda	Davlat personal ma'lumotlar bazalari reyestrida ro'yxatdan o'tish
Buzilish haqida xabar	24-modda	Insident → 72 soat ichida vakolatli organga

```
┌──────────────────────────────────────────────────┐
│  ⚡ MA'LUMOT LOKALIZATSIYASI — ARXITEKTURAVIY MAJBURIYAT      │
│                                                  │
│  🟢 O'ZBEKISTONDA (majburiy):                    │
│     • PostgreSQL primary + replica               │
│     • Backup asosiy nusxasi (S3 Toshkent)        │
│     • Redis (sessiya, PII kesh)                  │
│     • Ilova log (IP, UA)                         │
│     • AuditLog                                   │
│                                                  │
│  🔴 CHET ELDA (PII'siz bo'lishi shart):          │
│     • CDN (3D fayl, rasm — shaxsiy ma'lumot yo'q) │
│     • DR backup nusxasi (⚠ shifrlangan, kalit UZ'da) │
│     • Prometheus/Grafana (agregat metrikalar)    │
│     • AI provayderlar (⚠ §11.2)                  │
└──────────────────────────────────────────────────┘
```

11.2 AI provayderlar va ma'lumot uzatish

> ⚠ **Muammo:** AI provayderlar (Gemini, Claude, GPT) chet elda. Ularga yuborilgan ma'lumot chegara orqali o'tadi.

Chora	Amalga oshirish
PII yuborilmaydi	AI so'rovga faqat: menyu matni + mijoz savoli. ⛔ Telefon, ism, buyurtma tarixi, IP yo'q
Anonimlashtirish	`session_key` o'rniga tasodifiy `conversation_id`
Rozilik	AI chat birinchi ochilganda: "Savolingiz AI xizmatiga yuboriladi"
Saqlanmaslik	Provayder API'da `store=false` / zero-retention rejimi (mavjud bo'lsa)
Lokal fallback	pgvector semantik qidiruv — PII umuman chiqmaydi
Log	AI so'rov/javob 30 kun, faqat UZ serverida

11.3 Fiskal talablar

Talab	Hujjat	ARMENU'da
Onlayn nazorat-kassa mashinasi (NKM)	VM №943 (2019)	OFD integratsiyasi (UZKASSA / e-POS)
Har savdo uchun fiskal chek	Soliq kodeksi 268-modda	Har `payment.paid` → fiskal chek
IKPU (MXIK) kodi har mahsulotda	VM №1030	Har `Dish` da `ikpu_code` majburiy maydon; `tasnif.soliq.uz` dan tekshiriladi
QR-chek mijozga	VM №943	Chek QR → Telegram / SMS / ekranda
Chekni 5 yil saqlash	Soliq kodeksi	S3 WORM arxiv
Chek raqami uzluksiz	OFD talabi	Sequence, bo'shliqsiz; bo'shliq → alert
Bekor qilish/qaytarish cheki	VM №943	`refund` → alohida fiskal hujjat
QQS stavkasi	Soliq kodeksi	Har taomda `vat_rate` (0% / 12%), restoran rejimiga qarab

```
# apps/fiscal/models.py
class Dish(models.Model):
    ikpu_code = models.CharField(max_length=17, db_index=True,
                                 help_text="MXIK/IKPU kodi — majburiy")
    package_code = models.CharField(max_length=20, blank=True)
    vat_rate  = models.DecimalField(max_digits=4, decimal_places=2, default=12)

    def clean(self):
        # ⚡ Fiskal chek yuborilmasligi uchun IKPU'siz taom sotilmaydi
        if self.is_active and not self.ikpu_code:
            raise ValidationError({'ikpu_code': "IKPU kodi majburiy (soliq talabi)"})
```

11.4 Rozilik olish oqimi




```python
# apps/legal/models.py
class ConsentLog(models.Model):
    """Rozilik dalili — nizo bo'lsa isbot."""
    restaurant_id = models.IntegerField(db_index=True)
    session_key   = models.CharField(max_length=64, db_index=True)
    consent_type  = models.CharField(max_length=32)     # privacy|phone|ai|marketing
    version       = models.CharField(max_length=16)     # siyosat versiyasi
    granted       = models.BooleanField()
    ts            = models.DateTimeField(auto_now_add=True)
    ip            = models.GenericIPAddressField(null=True)
    user_agent    = models.CharField(max_length=300, blank=True)
```

12. XAVFSIZLIK CHEK-LISTI — DEPLOY OLDIDAN (25 punkt)

🔧 Konfiguratsiya

#	Tekshiruv	Buyruq / usul	✔
1	`DEBUG = False`	`python manage.py diffsettings | grep DEBUG`	☐
2	`ALLOWED_HOSTS` aniq (`*` emas)	settings ko'rigi	☐
3	`SECRET_KEY` production'ga xos, git'da yo'q	`gitleaks detect`	☐
4	`SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE` = True	`manage.py check --deploy`	☐
5	`SECURE_HSTS_SECONDS ≥ 31536000` + preload	`check --deploy`	☐
6	`/admin/` o'chirilgan yoki IP allowlist + 2FA	nginx config	☐
7	`.env` chmod 600, `.git` veb orqali yopiq	`curl https://armenu.uz/.env` → 404	☐

🔐 Auth va avtorizatsiya

#	Tekshiruv	✔
8	Barcha ViewSet'larda `permission_classes` bor (default `AllowAny` emas)	☐
9	Har ViewSet'da `required_action` belgilangan	☐
10	JWT TTL jadvalga mos (access ≤ 15 daq)	☐
11	Refresh rotatsiya + oila revoke ishlayapti	☐
12	Platforma admin 2FA majburiy	☐

⚡ Multi-tenant

#	Tekshiruv	✔
13	`pytest tests/security/test_tenant_isolation.py` — 100% o'tdi	☐
14	Barcha kesh kalitlarida `restaurant_id` bor	☐
15	WS group nomlari serverda quriladi	☐
16	Tenant audit skripti 0 anomaliya qaytardi	☐

💳 To'lov va fiskal

#	Tekshiruv	✔
17	Karta maydonlari yo'q (`test_no_card_data` o'tdi)	☐
18	Webhook imzo + IP allowlist barcha 3 provayderda	☐
19	Summa serverda hisoblanadi (`test_client_cannot_set_price` o'tdi)	☐
20	Har taomda IKPU kodi bor	☐

🛡 Umumiy

#	Tekshiruv	✔
21	`pip-audit` — 0 ta yuqori/kritik zaiflik	☐
22	`bandit -r apps/ core/ -ll` — 0 ta yuqori	☐
23	Rate limit barcha kritik endpointlarda	☐
24	Log'da PII yo'q (namuna log 10 qator qo'lda tekshirildi)	☐
25	Backup shifrlangan + tiklash sinovi so'nggi 7 kunda o'tgan	☐

```bash
#!/usr/bin/env bash
# ops/security/preflight.sh — deploy oldidan avtomatik tekshiruv
set -e
echo "═══ ARMENU Xavfsizlik Preflight ═══"
python manage.py check --deploy --fail-level WARNING
pytest tests/security/ -q --no-header
gitleaks detect --no-git --redact
pip-audit --strict -r requirements.txt
bandit -r apps/ core/ -ll -q -x tests
python manage.py tenant_audit --fail-on-anomaly
curl -sf -o /dev/null -w '%{http_code}' https://armenu.uz/.env | grep -q 404
python manage.py backup_verify --max-age-days=7
echo "✓ Barcha tekshiruvlar o'tdi — deploy'ga ruxsat"
```

13. INSIDENTGA JAVOB

13.1 Darajalar

Daraja	Ta'rif	Misol	Javob vaqti	Kim
🔴 **P1**	Ma'lumot sizishi / to'liq to'xtash / moliyaviy zarar	Cross-tenant sizish, DB egallanishi, to'lov manipulyatsiyasi	**15 daqiqa** (24/7)	Texnik direktor + SRE + backend lead
🟠 **P2**	Jiddiy degradatsiya / potensial sizish	Auth chetlab o'tish urinishi, PII log'da, provayder yiqilishi	**1 soat** (ish vaqti), 4 soat (tunda)	SRE + backend
🟡 **P3**	Cheklangan ta'sir	Rate limit chetlab o'tish, XSS (self), zaiflik topildi	**24 soat**	Backend
⚪ **P4**	Ta'sirsiz / profilaktik	Eskirgan kutubxona, konfiguratsiya tavsiyasi	**7 kun**	Navbatdagi sprint

13.2 Javob rejasi (P1 namunasi — cross-tenant sizish)



13.3 Aloqa matritsasi

Kim	Qachon xabar beriladi	Kanal	Kim xabar beradi
Incident commander	Darhol	PagerDuty / qo'ng'iroq	Avtomatik alert
SRE navbatchi	Darhol (P1/P2)	Telegram + qo'ng'iroq	Avtomatik
Backend lead	P1: darhol, P2: 1 soat	Telegram	IC
Texnik direktor	P1: darhol	Qo'ng'iroq	IC
Bosh direktor	P1 (ma'lumot sizishi)	Qo'ng'iroq	Texnik direktor
Ta'sirlangan restoranlar	30 daq ichida	Telegram + email + qo'ng'iroq	Qo'llab-quvvatlash
Barcha mijozlar	Agar keng ta'sir	Status sahifa + email	Marketing
Vakolatli organ	72 soat ichida (PM sizishi)	Rasmiy xat	Yuridik
To'lov provayderi	To'lovga tegishli bo'lsa	Provayder kanali	Texnik direktor

13.4 Post-mortem shabloni

```
# Insident hisoboti — INC-2026-NN

**Sana:** · **Daraja:** P_ · **Davomiylik:** _ daqiqa
**Incident commander:** · **Ta'sirlangan xizmat:**

## 1. Qisqacha (2-3 jumla)
## 2. Ta'sir
   - Ta'sirlangan restoranlar / foydalanuvchilar:
   - Sizgan ma'lumot (bo'lsa):
   - Moliyaviy zarar:
   - Error budget sarfi:
## 3. Vaqt chizig'i (UTC)
   | Vaqt | Hodisa | Kim |
## 4. Ildiz sabab (5 marta "nega")
## 5. Nima yaxshi ishladi
## 6. Nima yomon ishladi
## 7. Omad (bexosdan qutulib qolgan joylar)
## 8. Harakat rejasi
   | # | Harakat | Turi (oldini olish/aniqlash/yumshatish) | Kim | Muddat | Ticket |
## 9. Regressiya testi
   - [ ] Shu bug'ni qaytadan ushlaydigan test yozildi va CI'ga qo'shildi

⚡ Aybsiz (blameless): odam emas, TIZIM aybdor. Odam xato qila olgan bo'lsa —
tizim uni to'xtatishi kerak edi.
```

14. XULOSA — XAVFSIZLIK QISQACHA



Raqamlarda:

Ko'rsatkich	Qiymat
STRIDE tahdid ssenariylari	44
Multi-tenant himoya qatlamlari	4 + kunlik audit
Ma'lum sizish naqshlari (kataloglangan)	15
Autentifikatsiya aktorlari	4
Rol × amal matritsasi	6 × 24
Rate limit qoidalari	20
PII maydonlari (kataloglangan)	16
Sirlar (rotatsiya jadvali bilan)	18
Anomaliya aniqlash qoidalari	10
Deploy oldidan chek-list	25 punkt
CI xavfsizlik gate'lari	8

QISM 7

Integratsiyalar

Versiya: 1.0 · **Sana:** 2026-09-08
Kim uchun: backend dasturchi · integratsiya muhandisi · texnik direktor
Oldingi: `06_XAVFSIZLIK.md` · **Keyingi:** `08_DEPLOY_VA_OPERATSIYA.md`

---

1. INTEGRATSIYA XARITASI

1.1 To'liq ko'rinish



1.2 Kritiklik darajasi va oqibati

Xizmat	Yo'nalish	Kritiklik	Yiqilsa nima bo'ladi	Fallback
Payme / Click / Uzum	ikki tomonlama	🔴 Yuqori	Ilovada to'lash yo'q	Boshqa provayder → **naqd (ofitsiantga)**
OFD (UZKASSA/e-POS)	chiquvchi	🔴 Yuqori	Chek yuborilmaydi (qonun buzilishi)	2-OFD → navbat (48 s) → qo'lda kassa
CDN / S3	chiquvchi	🔴 Yuqori	3D fayl yuklanmaydi → AR yo'q	Origin → lokal kesh → **rasm fallback**
PostgreSQL	ichki	🔴 Yuqori	— (bu integratsiya emas, yadro)	replica (o'qish)
AI (8 provayder)	chiquvchi	🟡 O'rta	Chat/tarjima/3D yo'q	Zanjir 8 ta → "Ofitsiantni chaqiring"
Telegram	ikki tomonlama	🟡 O'rta	Push yo'q, Mini App yo'q	WS + brauzer notification
tasnif.soliq.uz (IKPU)	chiquvchi	🟡 O'rta	Yangi taomga IKPU topilmaydi	Kesh → qo'lda kiritilgan kod
Google Places	chiquvchi	🟢 Past	Sharh filtri ishlamaydi	Yashiriladi
CBU kurs	chiquvchi	🟢 Past	Valyuta konvertatsiya eskiradi	Oxirgi ma'lum kurs

1.3 Bir qarashda raqamlar

Ko'rsatkich	Qiymat
Tashqi chiquvchi integratsiya	**16 ta** endpoint guruhi
Kiruvchi webhook	**5 ta** (Payme, Click, Uzum, Telegram, OFD)
Adapter sinflar soni	**16** (`base.py` interfeysi bilan)
Circuit breaker sonlari	**16** (har adapterga 1 ta)
Oylik tashqi xarajat (500 restoran)	**~$620**

---

2. UMUMIY INTEGRATSIYA NAQSHI

> ⚡ **Qoida:** Kod HECH QACHON tashqi provayderni to'g'ridan-to'g'ri chaqirmaydi.
> Kod faqat **niyat** bildiradi. Qaysi provayder — registry hal qiladi.

2.1 4 qatlamli naqsh



2.2 Nega aynan shu naqsh

Sabab	Amaldagi foyda
Provayder almashadi	Payme narxi oshdi → Click birinchi. 1 qator config
API o'zgaradi	OFD v2 chiqdi → faqat 1 ta adapter fayli o'zgaradi
Test oson	Adapter mock qilinadi, biznes logika tegilmaydi
Xato qamalgan	Tashqi kutubxona `Exception`'i adapter'dan chiqmaydi
Kuzatuv bir joyda	Metric/log/circuit breaker router'da — 16 joyda takrorlanmaydi
Kelajak	Yangi provayder = 1 fayl + registry'ga 1 qator

2.3 Baza interfeysi (barcha domenlar uchun umumiy)

```
# integrations/base.py
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Any


@dataclass
class IntegrationRequest:
    """Har qanday tashqi chaqiruv uchun umumiy so'rov."""
    op: str                          # 'charge', 'refund', 'send_receipt', ...
    payload: dict = field(default_factory=dict)
    idempotency_key: str = ''        # ⚡ har yozuv operatsiyasida MAJBURIY
    timeout: int = 15
    restaurant_id: int | None = None
    meta: dict = field(default_factory=dict)


@dataclass
class IntegrationResponse:
    ok: bool
    data: Any = None
    provider: str = ''
    external_id: str = ''            # tashqi tizimdagi ID
    error_code: str = ''             # ichki normallashtirilgan kod
    error_msg: str = ''
    retryable: bool = False          # ⚡ qayta urinsa bo'ladimi
    latency_ms: int = 0
    raw: dict = field(default_factory=dict)


class BaseAdapter(ABC):
    """Har bir tashqi xizmat adapteri shu interfeysni bajaradi."""

    name: str                # 'payme'
    domain: str              # 'payment' | 'fiscal' | 'ai' | 'messaging' | 'storage'
    priority: int = 100      # kichik = birinchi
    supported_ops: set = set()

    # --- majburiy ---
    @abstractmethod
    def is_configured(self) -> bool:
        """API kalitlar to'liqmi (kalit yo'q → registry bu adapterni tashlab ketadi)"""

    @abstractmethod
    def execute(self, req: IntegrationRequest) -> IntegrationResponse:
        """Asosiy chaqiruv. HECH QACHON exception ko'tarmaydi — Response qaytaradi."""

    # --- ixtiyoriy, standart amalga oshirish ---
    def health_check(self) -> bool:
        return self.is_configured()

    def verify_webhook(self, headers: dict, body: bytes) -> bool:
        """Kiruvchi webhook imzosini tekshirish (to'lov/Telegram uchun)"""
        return False

    def normalize_error(self, exc: Exception) -> tuple[str, bool]:
        """(error_code, retryable)"""
        import requests
        if isinstance(exc, requests.Timeout):
            return 'TIMEOUT', True
        if isinstance(exc, requests.ConnectionError):
            return 'NETWORK', True
        return 'UNKNOWN', False
```

2.4 Umumiy router (barcha domen router'lari shundan meros oladi)

```
# integrations/router.py
import time
import logging
from django.core.cache import cache
from core.resilience import CircuitBreaker
from core import metrics
from .base import IntegrationRequest, IntegrationResponse

log = logging.getLogger('integrations')


class BaseRouter:
    """
    Provayder tanlash + circuit breaker + retry + idempotency + metrics.
    Domen router'lari (PaymentRouter, FiscalRouter, MCPRouter) shundan meros oladi.
    """
    domain = ''
    registry: list = []                # adapter sinflari
    cb_fail_threshold = 5
    cb_timeout = 60
    max_retries = 2
    idempotency_ttl = 86400            # 24 soat

    def __init__(self):
        self.adapters = [cls() for cls in self.registry]
        self.breakers = {
            a.name: CircuitBreaker(
                f'{self.domain}.{a.name}',
                fail_threshold=self.cb_fail_threshold,
                timeout=self.cb_timeout,
            )
            for a in self.adapters
        }

    # ---------- asosiy ----------
    def execute(self, req: IntegrationRequest, prefer: str = '') -> IntegrationResponse:
        # 1. Idempotency — bir xil kalit bilan ikkinchi marta kelsa, saqlangan javob
        if req.idempotency_key:
            cached = cache.get(self._idem_key(req))
            if cached:
                log.info('[%s] idempotent takror: %s', self.domain, req.idempotency_key)
                return IntegrationResponse(**cached)

        candidates = self._candidates(req.op, prefer)
        if not candidates:
            return IntegrationResponse(ok=False, error_code='NO_PROVIDER')

        errors = []
        for adapter in candidates:
            br = self.breakers[adapter.name]
            if br.is_open():
                errors.append(f'{adapter.name}:circuit_open')
                metrics.inc(f'cb.{self.domain}.{adapter.name}.rejected')
                continue

            resp = self._call_with_retry(adapter, req)

            if resp.ok:
                br.on_success()
                self._record(adapter, req, resp)
                if req.idempotency_key:
                    cache.set(self._idem_key(req), resp.__dict__, self.idempotency_ttl)
                return resp

            # ⚡ retryable EMAS xato (masalan "mablag' yetarli emas") → 
            #    keyingi provayderga o'tish MA'NOSIZ, darhol qaytariladi
            if not resp.retryable:
                self._record(adapter, req, resp)
                return resp

            br.on_failure()
            errors.append(f'{adapter.name}:{resp.error_code}')
            log.warning('[%s] %s yiqildi (%s) → keyingisi',
                        self.domain, adapter.name, resp.error_code)

        log.error('[%s] BARCHA provayder yiqildi: %s', self.domain, errors)
        return IntegrationResponse(ok=False, error_code='ALL_PROVIDERS_FAILED',
                                   raw={'errors': errors})

    # ---------- yordamchilar ----------
    def _candidates(self, op, prefer=''):
        ps = [a for a in self.adapters if a.is_configured() and op in a.supported_ops]
        ps.sort(key=lambda a: (a.name != prefer, a.priority))
        return ps

    def _call_with_retry(self, adapter, req):
        delay = 0.5
        last = None
        for attempt in range(self.max_retries + 1):
            t0 = time.time()
            try:
                resp = adapter.execute(req)
            except Exception as exc:                    # adapter buzilib chiqsa ham
                code, retryable = adapter.normalize_error(exc)
                resp = IntegrationResponse(ok=False, provider=adapter.name,
                                           error_code=code, error_msg=str(exc),
                                           retryable=retryable)
            resp.latency_ms = resp.latency_ms or int((time.time() - t0) * 1000)
            if resp.ok or not resp.retryable:
                return resp
            last = resp
            if attempt < self.max_retries:
                time.sleep(delay)
                delay *= 3
        return last

    def _idem_key(self, req):
        return f'idem:{self.domain}:{req.op}:{req.idempotency_key}'

    def _record(self, adapter, req, resp):
        metrics.observe(f'external.latency_ms', resp.latency_ms,
                        {'domain': self.domain, 'provider': adapter.name})
        metrics.inc(f'external.{"success" if resp.ok else "failure"}',
                    {'domain': self.domain, 'provider': adapter.name})
        IntegrationLog.objects.create(
            domain=self.domain, provider=adapter.name, op=req.op,
            restaurant_id=req.restaurant_id, ok=resp.ok,
            external_id=resp.external_id, error_code=resp.error_code,
            latency_ms=resp.latency_ms,
            request_digest=mask_secrets(req.payload),      # ⚡ PII/kalit maskalanadi
        )
```

2.5 Papka tuzilishi

```
apps/integrations/
├── base.py                  # BaseAdapter, IntegrationRequest/Response
├── router.py                # BaseRouter (yuqoridagi)
├── models.py                # IntegrationLog, WebhookEvent
├── masking.py               # mask_secrets() — log uchun
│
├── ai/                      # ⚡ MCP qatlami (§3)
│   ├── base.py              # MCPProvider, MCPRequest/Response
│   ├── router.py            # MCPRouter
│   ├── registry.py
│   └── providers/
│       ├── claude.py  gemini.py  openai.py  groq.py
│       ├── deepseek.py  mistral.py
│       └── meshy.py  tripo.py
│
├── payment/                 # §4
│   ├── router.py  registry.py  webhooks.py  reconcile.py
│   └── providers/  payme.py  click.py  uzum.py
│
├── fiscal/                  # §5
│   ├── router.py  registry.py  ikpu.py  retry.py
│   └── providers/  uzkassa.py  epos.py
│
├── messaging/               # §6
│   ├── router.py  queue.py  ratelimit.py
│   └── providers/  telegram.py
│
└── storage/                 # §7
    ├── router.py  cdn.py
    └── providers/  s3.py  minio.py
```

---

3. ⚡ AI / MCP QATLAMI

> Talab: "*eng kamida 6 ta AI ga MCP yozilgan va API key orqali muammosiz ishlashi*"
> Bajarildi: **8 ta provayder** (+1 ixtiyoriy).

3.1 To'liq arxitektura



3.2 Interfeys — `MCPProvider` ABC

```python
# apps/integrations/ai/base.py
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Any


@dataclass
class MCPRequest:
    task: str                           # chat|translate|vision|image_to_3d|embedding
    prompt: str = ''
    system: str = ''
    messages: list = field(default_factory=list)
    images: list = field(default_factory=list)     # URL yoki base64
    json_mode: bool = False
    max_tokens: int = 1500
    temperature: float = 0.3
    timeout: int = 30
    extra: dict = field(default_factory=dict)


@dataclass
class MCPResponse:
    ok: bool
    content: Any = None
    provider: str = ''
    model: str = ''
    tokens_in: int = 0
    tokens_out: int = 0
    cost_usd: float = 0.0
    latency_ms: int = 0
    error: str = ''
    raw: dict = field(default_factory=dict)


class MCPProvider(ABC):
    name: str
    supported_tasks: set
    priority: dict              # {'chat': 20, 'translate': 10}
    price_in: float             # $ / 1M kirish token
    price_out: float            # $ / 1M chiqish token

    @abstractmethod
    def is_configured(self) -> bool: ...

    @abstractmethod
    def call(self, req: MCPRequest) -> MCPResponse:
        """» HECH QACHON exception ko'tarmaydi — MCPResponse(ok=False) qaytaradi."""

    def health_check(self) -> bool:
        try:
            return self.call(MCPRequest(task='chat', prompt='ping',
                                        max_tokens=5, timeout=8)).ok
        except Exception:
            return False

    def estimate_cost(self, tin: int, tout: int) -> float:
        return (tin * self.price_in + tout * self.price_out) / 1_000_000
```

3.3 Router — 7 bosqich

#	Bosqich	Nima qiladi	Yiqilsa
1	**Kesh**	`sha256(task+system+prompt+temp)` → Redis (TTL 3600s). Faqat `chat` / `translate`	Keshsiz davom
2	**Byudjet**	Restoran kunlik limiti (`AI_DAILY_LIMIT_PER_RESTAURANT=$2`) + platforma oylik ($500)	`AI_BUDGET_EXCEEDED`
3	**Nomzodlar**	`is_configured()` va `task in supported_tasks` → `priority` bo'yicha saralash	`NO_PROVIDER_FOR_<task>`
4	**Circuit breaker**	`OPEN` bo'lgan provayder chetlab o'tiladi (3 xato → 120s)	keyingisiga
5	**Chaqiruv**	`provider.call(req)`	keyingisiga
6	**Yozuv**	`AIUsage` + Prometheus metrikalar	log
7	**Keshga qo'yish**	Muvaffaqiyatli javob keshlanadi	—

```
# apps/integrations/ai/router.py
import hashlib
import logging
from django.core.cache import cache
from django.conf import settings
from django.db.models import Sum
from django.utils import timezone
from core.resilience import CircuitBreaker
from core import metrics
from .base import MCPRequest, MCPResponse
from .registry import ALL_PROVIDERS

log = logging.getLogger('ai.mcp')

CACHEABLE = {'translate', 'chat', 'embedding'}


class MCPRouter:
    def __init__(self):
        self.providers = [cls() for cls in ALL_PROVIDERS]
        self.breakers = {
            p.name: CircuitBreaker(f'ai.{p.name}', fail_threshold=3, timeout=120)
            for p in self.providers
        }

    def run(self, req: MCPRequest, restaurant_id=None, use_cache=True) -> MCPResponse:
        # [1] KESH
        ck = self._cache_key(req)
        if use_cache and req.task in CACHEABLE:
            hit = cache.get(ck)
            if hit:
                metrics.inc('ai.cache.hit', {'task': req.task})
                return MCPResponse(**hit)
            metrics.inc('ai.cache.miss', {'task': req.task})

        # [2] BYUDJET
        if restaurant_id and not self._check_budget(restaurant_id):
            return MCPResponse(ok=False, error='AI_BUDGET_EXCEEDED')
        if not self._check_platform_budget():
            return MCPResponse(ok=False, error='AI_PLATFORM_BUDGET_EXCEEDED')

        # [3] NOMZODLAR
        candidates = self._candidates(req.task)
        if not candidates:
            return MCPResponse(ok=False, error=f'NO_PROVIDER_FOR_{req.task}')

        errors = []
        for p in candidates:
            br = self.breakers[p.name]
            # [4] CIRCUIT BREAKER
            if br.is_open():
                errors.append(f'{p.name}:circuit_open')
                continue

            # [5] CHAQIRUV
            log.info('[MCP] %s → %s', req.task, p.name)
            resp = p.call(req)

            if resp.ok:
                br.on_success()
                self._record(resp, restaurant_id, req.task)      # [6]
                if use_cache and req.task in CACHEABLE:          # [7]
                    cache.set(ck, resp.__dict__, settings.AI_CACHE_TTL)
                return resp

            br.on_failure()
            errors.append(f'{p.name}:{resp.error[:80]}')
            metrics.inc('ai.provider.failure', {'provider': p.name})
            log.warning('[MCP] %s xato: %s → keyingisi', p.name, resp.error)

        log.error('[MCP] BARCHA provayder yiqildi: %s', errors)
        metrics.inc('ai.all_failed', {'task': req.task})
        return MCPResponse(ok=False, error='ALL_PROVIDERS_FAILED',
                           raw={'errors': errors})

    # ---------------- yordamchilar ----------------
    def _candidates(self, task):
        ps = [p for p in self.providers
              if p.is_configured() and task in p.supported_tasks]
        return sorted(ps, key=lambda p: p.priority.get(task, 999))

    def _cache_key(self, req):
        raw = f'{req.task}|{req.system}|{req.prompt}|{req.temperature}|{req.max_tokens}'
        return f'mcp:{hashlib.sha256(raw.encode()).hexdigest()[:32]}'

    def _check_budget(self, restaurant_id) -> bool:
        from apps.ai.models import AIUsage
        today = timezone.now().date()
        spent = AIUsage.objects.filter(
            restaurant_id=restaurant_id, created_at__date=today
        ).aggregate(s=Sum('cost_usd'))['s'] or 0
        if spent >= settings.AI_DAILY_LIMIT_PER_RESTAURANT:
            log.warning('[MCP] Byudjet tugadi: rest=%s $%.2f', restaurant_id, spent)
            metrics.inc('ai.budget.exceeded', {'restaurant': restaurant_id})
            return False
        return True

    def _check_platform_budget(self) -> bool:
        from apps.ai.models import AIUsage
        m0 = timezone.now().replace(day=1, hour=0, minute=0, second=0, microsecond=0)
        spent = AIUsage.objects.filter(created_at__gte=m0) \
                    .aggregate(s=Sum('cost_usd'))['s'] or 0
        return spent < settings.AI_MONTHLY_LIMIT_TOTAL

    def _record(self, resp, restaurant_id, task):
        from apps.ai.models import AIUsage
        AIUsage.objects.create(
            restaurant_id=restaurant_id, provider=resp.provider, model=resp.model,
            task=task, tokens_in=resp.tokens_in, tokens_out=resp.tokens_out,
            cost_usd=resp.cost_usd, latency_ms=resp.latency_ms)
        metrics.inc('ai.request.success', {'provider': resp.provider, 'task': task})
        metrics.observe('ai.latency_ms', resp.latency_ms, {'provider': resp.provider})
        metrics.add('ai.cost_usd', resp.cost_usd,
                    {'provider': resp.provider, 'model': resp.model})


mcp = MCPRouter()      # singleton — Django app ready() da tekshiriladi
```

3.4 8 provayder jadvali

#	Provayder	Model	Vazifalar	Narx ($/1M in-out)	Env kalit	Prioritet (chat/trans/vision/3d)
1	**Anthropic Claude**	`claude-haiku-4-5`	chat, translate, vision, reasoning	0,80 / 4,00	`ANTHROPIC_API_KEY`	20 / **10** / 15 / —
2	**Google Gemini**	`gemini-2.0-flash`	chat, translate, vision, embedding	0,10 / 0,40	`GEMINI_API_KEY`	**10** / 20 / **10** / —
3	**OpenAI**	`gpt-4o-mini`	chat, translate, vision	0,15 / 0,60	`OPENAI_API_KEY`	40 / 30 / 20 / —
4	**Groq**	`llama-3.3-70b`	chat (juda tez, ~0,3 s)	0,59 / 0,79	`GROQ_API_KEY`	15 / — / — / —
5	**DeepSeek**	`deepseek-chat`	chat, translate (eng arzon)	0,14 / 0,28	`DEEPSEEK_API_KEY`	30 / 25 / — / —
6	**Mistral**	`mistral-small`	chat, translate, embedding	0,20 / 0,60	`MISTRAL_API_KEY`	50 / 40 / — / —
7	**Meshy**	`meshy-4`	image_to_3d	~$0,50 / model	`MESHY_API_KEY`	— / — / — / **10**
8	**Tripo**	`tripo-v2`	image_to_3d	~$0,80 / model	`TRIPO_API_KEY`	— / — / — / 20
9*	**Rodin** (ixt.)	`rodin-gen1`	image_to_3d (yuqori sifat)	~$1,50 / model	`RODIN_API_KEY`	— / — / — / 30

> Kichik raqam = birinchi chaqiriladi. `—` = bu vazifani qo'llab-quvvatlamaydi.

3.5 Vazifa turlari

`task`	Zanjir (prioritet bo'yicha)	Keshlanadi	Timeout	Qayerda ishlatiladi
`chat`	Gemini → Groq → Claude → DeepSeek → OpenAI → Mistral	✅ 1 soat	30 s	Mijoz chatboti (P-12)
`translate`	Claude → Gemini → DeepSeek → OpenAI → Mistral	✅ 24 soat	45 s	Menyu avto-tarjima (uz/ru/en)
`vision`	Gemini → Claude → OpenAI	❌	40 s	Taom rasmi sifatini baholash
`image_to_3d`	Meshy → Tripo → (Rodin)	❌	600 s ⚡	3D model generatsiyasi (worker'da!)
`embedding`	Gemini → Mistral	✅ 7 kun	20 s	3D kutubxona semantik moslashtirish (pgvector)
`reasoning`	Claude → OpenAI	❌	60 s	Menyu-injiniring tahlili, ROI hisoboti

> ⚡ `image_to_3d` HECH QACHON web so'rovda chaqirilmaydi — faqat `armenu-worker@model3d`.

3.6 Xarajat nazorati — 4 qatlam

```
┌──────────────────────────────────────────────────────────┐
│ [1] KESH          bir xil so'rov → 0 $ (chat −35% hit)   │
├──────────────────────────────────────────────────────────┤
│ [2] KONTEKST      menyu 8000 → 1500 token (kesilgan format)  │
│                   "ID|Nom|Narx|Teg|Allergen|Vaqt"        │
├──────────────────────────────────────────────────────────┤
│ [3] RESTORAN      AI_DAILY_LIMIT_PER_RESTAURANT = $2/kun  │
│     LIMITI        oshsa → AI_BUDGET_EXCEEDED, chat o'chadi  │
├──────────────────────────────────────────────────────────┤
│ [4] PLATFORMA     AI_MONTHLY_LIMIT_TOTAL = $500/oy        │
│     LIMITI        80% → 🟡 alert, 100% → 🔴 to'xtatiladi   │
└──────────────────────────────────────────────────────────┘
```

Vazifa	Chastota (500 rest.)	Provayder	Oylik $
Chatbot	~50 000	Gemini→Groq	45
Tarjima	~5 000	Claude→Gemini	12
3D generatsiya	~800	Meshy→Tripo	**400**
Taom tavsifi	~2 000	Claude	8
Menyu tahlili	~500	Claude	15
Sharh sentiment	~10 000	Gemini→DeepSeek	6
Rasm sifati (vision)	~3 000	Gemini	5
Embedding	~5 000	Gemini	2
JAMI			**~$493/oy**

> 500 restoran × 590k so'm ≈ $23 300/oy daromad → AI = **2,1%**. ✅ Sog'lom.

3.7 Yangi provayder qo'shish — 5 qadam

```
QADAM 1 — Adapter fayli yozish (~80 qator)
  apps/integrations/ai/providers/newai.py
  class NewAIProvider(MCPProvider):
      name = 'newai'
      supported_tasks = {'chat', 'translate'}
      priority = {'chat': 35, 'translate': 45}
      price_in, price_out = 0.05, 0.15
      def is_configured(self): return bool(self.key)
      def call(self, req): ...  # HECH QACHON raise qilmaydi

QADAM 2 — Registry'ga qo'shish (1 qator)
  apps/integrations/ai/registry.py
  ALL_PROVIDERS = [..., NewAIProvider]

QADAM 3 — .env ga kalit
  NEWAI_API_KEY=...
  settings.AI_KEYS dan avtomatik o'qiladi

QADAM 4 — Test (kontrakt testi avtomatik qo'llanadi)
  pytest tests/integrations/test_mcp_contract.py -k newai
  → is_configured, call, health_check, xato→Response tekshiriladi

QADAM 5 — Kuzatuv
  Grafana "AI provayderlar" paneli avtomatik yangi qatorni ko'rsatadi
  (metrika label'i `provider` bo'yicha).

⏱ Jami vaqt: ~40 daqiqa. Biznes logikaga 0 ta o'zgarish.
```

3.8 API kalit xavfsizligi

Chora	Amalda
Kalit kodda YO'Q	Faqat `.env` (600, `armenu:armenu`)
`.env` git'da YO'Q	`.gitignore` + CI'da `gitleaks` skani
Log'da maskalanadi	`mask_secrets()` → `sk-ant-***a3f`
Frontendga uzatilmaydi	Barcha AI chaqiruvi faqat backendda
Rotatsiya	90 kunda bir marta (kalendarda)
Har provayderga limit	Provayder panelida hard cap
Ishga tushishda tekshiruv	`apps.py ready()` — 0 provayder → `log.error`

---

4. TO'LOV INTEGRATSIYASI

> ⚡ **Oltin qoida:** Narx faqat serverda hisoblanadi. Klient yuborgan summa**ishonchsiz**.
> ⚡ **Ikkinchi qoida:** Har to'lov operatsiyasi idempotent. Dublikat to'lov = 0.

4.1 3 provayder

Provayder	Protokol	Auth	Callback	Xususiyat
Payme (Paycom)	JSON-RPC 2.0, **6 metod**	Basic `Paycom:<KEY>`	bizga POST	Merchant API, eng keng tarqalgan
Click	REST, **2 bosqich** (Prepare + Complete)	`sign_string` MD5	bizga POST ×2	Eski, lekin ommaviy
Uzum Bank	REST + **HMAC-SHA256**	`X-Signature` header	bizga POST	Yangi, Uzum ekotizimi

> Kutubxona: `paytechuz` — 3 tasini bitta interfeys ostida beradi.
> Biz baribir adapter yozamiz — kutubxona buzilsa/eskirsa almashtirish uchun.

4.2 Umumiy to'lov oqimi

```
[1] Mijoz "To'lash" bosadi (savat: 3 taom)
     │
     ▼
[2] POST /v1/payments/create/
     { order_id: 4471, provider: 'payme' }
     + X-Idempotency-Key: <uuid>
     │
     ▼
[3] ⚡ SERVER_SUMMANI O'ZI HISOBLAYDI
     amount = Σ(OrderItem.price × qty) + service_fee − discount
     (klient yuborgan summa E'TIBORGA OLINMAYDI)
     │
     ▼
[4] Payment.objects.create(
       order=..., amount=amount, provider='payme',
       state='created', idempotency_key=...)
     │
     ▼
[5] PaymentRouter.execute(op='create_invoice', ...)
      └→ PaymeAdapter → checkout URL / deeplink
     │
     ▼
[6] Mijoz provayder sahifasiga/ilovasiga o'tadi, to'laydi
     │
     ▼┌──────────────────────────────────────────────────┐
[7] ☁ WEBHOOK bizga keladi (POST /webhooks/payme/)      │
     │                                                  │
     ├─ [7.1] IMZO tekshiriladi ──────── ✗ → 401       │
     ├─ [7.2] WebhookEvent.create(raw) ← xom saqlanadi  │
     ├─ [7.3] IDEMPOTENCY: shu transaction_id bor?      │
     │        ha → oldingi javob qaytariladi ↩ TUGADI   │
     ├─ [7.4] SUMMA solishtiriladi ──────── ✗ → xato kodi │
     ├─ [7.5] transaction.atomic():                     │
     │          Payment.state = 'paid'                  │
     │          Order.payment_status = 'paid'           │
     │          FinancialOperation.create()             │
     └─ [7.6] on_commit():                              │
              ├─ WS → mijoz, ofitsiant, admin           │
              ├─ enqueue('fiscal', order.id)  ⚡ CHEK    │
              └─ enqueue('notify', ...)                 │
     │                                                  │
     ▼                                                  │
[8] Provayderga 200 + {result: {...}} qaytariladi ──────┘
     │
     ▼
[9] Mijoz sahifasi WS orqali "✓ To'landi" ko'radi
```

4.3 Payme — JSON-RPC 6 metod

```
# apps/integrations/payment/providers/payme.py
import base64
import time
from django.conf import settings
from ..base import BaseAdapter, IntegrationRequest, IntegrationResponse

# Payme xato kodlari (spetsifikatsiya bo'yicha)
ERR_TRANSPORT        = -32300
ERR_ACCESS_DENIED    = -32504
ERR_INVALID_AMOUNT   = -31001
ERR_ORDER_NOT_FOUND  = -31050
ERR_ORDER_AVAILABLE  = -31051   # buyurtma to'lovga tayyor emas
ERR_CANT_PERFORM     = -31008


class PaymeAdapter(BaseAdapter):
    name = 'payme'
    domain = 'payment'
    priority = 10
    supported_ops = {'create_invoice', 'check', 'cancel', 'refund'}

    BASE = 'https://checkout.paycom.uz'

    def __init__(self):
        self.merchant_id = settings.PAYME_MERCHANT_ID
        self.key = settings.PAYME_KEY              # webhook uchun
        self.test = settings.PAYME_TEST_MODE

    def is_configured(self):
        return bool(self.merchant_id and self.key)

    # ---------- CHIQUVCHI: to'lov havolasi ----------
    def execute(self, req: IntegrationRequest) -> IntegrationResponse:
        if req.op != 'create_invoice':
            return IntegrationResponse(ok=False, provider=self.name,
                                       error_code='OP_NOT_SUPPORTED')
        p = req.payload
        # Payme uchun tiyin (×100)
        raw = f"m={self.merchant_id};ac.order_id={p['order_id']};a={int(p['amount'] * 100)}"
        url = f"{self.BASE}/{base64.b64encode(raw.encode()).decode()}"
        return IntegrationResponse(ok=True, provider=self.name,
                                   data={'checkout_url': url})

    # --------- KIRUVCHI: webhook imzosi ---------
    def verify_webhook(self, headers: dict, body: bytes) -> bool:
        """Authorization: Basic base64('Paycom:' + KEY)"""
        auth = headers.get('Authorization', '')
        if not auth.startswith('Basic '):
            return False
        try:
            decoded = base64.b64decode(auth[6:]).decode()
        except Exception:
            return False
        login, _, key = decoded.partition(':')
        return login == 'Paycom' and hmac_equal(key, self.key)
```

6 ta majburiy metod (Merchant API):

#	Metod	Nima qiladi	Bizda	Xato kodi
1	`CheckPerformTransaction`	Buyurtma bormi, summa to'g'rimi	Order tekshiriladi	`-31050`, `-31001`
2	`CreateTransaction`	Tranzaksiya ochish	`Payment.state='pending'`	`-31008`
3	`PerformTransaction`	⚡ **To'lov tasdiqlash**	`state='paid'` + fiskal navbat	`-31008`
4	`CancelTransaction`	Bekor / qaytarish	`state='cancelled'` + restock	`-31007`
5	`CheckTransaction`	Holat so'rash	Payment holati	—
6	`GetStatement`	Davr bo'yicha ro'yxat	⚡ **reconciliation** asosi	—

```
# apps/integrations/payment/webhooks.py — Payme JSON-RPC dispatcher
PAYME_METHODS = {
    'CheckPerformTransaction': _check_perform,
    'CreateTransaction':       _create,
    'PerformTransaction':      _perform,
    'CancelTransaction':       _cancel,
    'CheckTransaction':        _check,
    'GetStatement':            _statement,
}


@csrf_exempt
@require_POST
def payme_webhook(request):
    adapter = PaymeAdapter()

    # [1] IMZO
    if not adapter.verify_webhook(dict(request.headers), request.body):
        return JsonResponse({'error': {'code': ERR_ACCESS_DENIED,
                                       'message': 'Access denied'}}, status=200)

    body = json.loads(request.body)
    method, params, rpc_id = body.get('method'), body.get('params', {}), body.get('id')

    # [2] XOM SAQLASH (har doim, qayta ishlashdan OLDIN)
    WebhookEvent.objects.create(provider='payme', method=method,
                                raw=body, received_at=timezone.now())

    handler = PAYME_METHODS.get(method)
    if not handler:
        return JsonResponse({'error': {'code': ERR_TRANSPORT,
                                       'message': 'Method not found'}, 'id': rpc_id})

    try:
        result = handler(params)                    # ichida idempotency bor
        return JsonResponse({'result': result, 'id': rpc_id})
    except PaymeError as e:
        return JsonResponse({'error': {'code': e.code, 'message': e.message},
                             'id': rpc_id})
    except Exception:
        log.exception('[payme] webhook xatosi')
        return JsonResponse({'error': {'code': ERR_TRANSPORT,
                                       'message': 'Internal'}, 'id': rpc_id})
```

4.4 Click — Prepare + Complete




```
def click_sign(params, secret_key, stage: str) -> str:
    """
    prepare : md5(click_trans_id + service_id + SECRET + merchant_trans_id
                  + amount + action + sign_time)
    complete: md5(click_trans_id + service_id + SECRET + merchant_trans_id
                  + merchant_prepare_id + amount + action + sign_time)
    """
    parts = [str(params['click_trans_id']), str(params['service_id']), secret_key,
             str(params['merchant_trans_id'])]
    if stage == 'complete':
        parts.append(str(params['merchant_prepare_id']))
    parts += [str(params['amount']), str(params['action']), str(params['sign_time'])]
    return hashlib.md5(''.join(parts).encode()).hexdigest()


CLICK_ERRORS = {
     0: 'Muvaffaqiyatli',
    -1: 'SIGN CHECK FAILED',
    -2: "Noto'g'ri summa",
    -3: 'Harakat topilmadi',
    -4: "Allaqachon to'langan",        # ⚡ idempotency signali
    -5: 'Foydalanuvchi topilmadi',
    -6: 'Tranzaksiya topilmadi',
    -8: "So'rov xatosi",
    -9: 'Tranzaksiya bekor qilingan',
}
```

4.5 Uzum — HMAC-SHA256

```
def uzum_verify(headers: dict, body: bytes, secret: str) -> bool:
    """
    X-Signature: hex(hmac_sha256(secret, timestamp + '.' + body))
    X-Timestamp: unix soniya — ±5 daqiqadan eski bo'lsa RAD ETILADI (replay himoya)
    """
    sig = headers.get('X-Signature', '')
    ts = headers.get('X-Timestamp', '')
    if not sig or not ts:
        return False
    if abs(int(time.time()) - int(ts)) > 300:
        log.warning('[uzum] eskirgan webhook (replay?): ts=%s', ts)
        return False
    expected = hmac.new(secret.encode(), f'{ts}.'.encode() + body,
                        hashlib.sha256).hexdigest()
    return hmac.compare_digest(sig, expected)
```

4.6 Webhook xavfsizligi — 7 qoida

#	Qoida	Nega
1	**Imzo/auth avval tekshiriladi**	Soxta to'lov bildirishnomasi
2	`hmac.compare_digest` ishlatiladi	Timing attack
3	**Xom body avval saqlanadi** (`WebhookEvent`)	Kod xato bersa ham ma'lumot yo'qolmaydi
4	**Timestamp oynasi** (Uzum: 5 daq)	Replay attack
5	**Summa qayta solishtiriladi**	Provayder/klient summasiga ishonilmaydi
6	**IP allowlist** (nginx darajasida)	Qo'shimcha qatlam
7	Javob **doim 200** + protokol xato kodi	Provayder cheksiz retry qilmasin

```
# nginx: to'lov webhook — faqat provayder IP'lari + rate limit
location /webhooks/payme/ {
    # allow 185.8.212.0/24;   # Payme (real ro'yxat provayderdan olinadi)
    # deny all;
    limit_req zone=webhook burst=50 nodelay;
    proxy_pass http://armenu_web;
}
```

4.7 Idempotency — 3 qatlam

```
┌──────────────────────────────────────────────────────┐
│ [1] KLIENT → BIZ                                     │
│     X-Idempotency-Key: <uuid>  (mijoz "To'lash"ni 2 marta  │
│     bossa ham 1 ta Payment)                          │
│     Redis: idem:payment:create:<uuid> → javob, TTL 24 soat  │
├──────────────────────────────────────────────────────┤
│ [2] PROVAYDER → BIZ (webhook)                        │
│     UNIQUE(provider, external_transaction_id) — DB darajasida  │
│     Takror kelsa → oldingi natija qaytariladi (yangi yozuv YO'Q)│
├──────────────────────────────────────────────────────┤
│ [3] BIZ → PROVAYDER (refund/cancel)                  │
│     IntegrationRequest.idempotency_key = f'refund:{payment.id}' │
└──────────────────────────────────────────────────────┘
```

```
class Payment(models.Model):
    order = models.ForeignKey(Order, on_delete=models.PROTECT)
    restaurant = models.ForeignKey(Restaurant, on_delete=models.PROTECT)  # ⚡ tenant
    provider = models.CharField(max_length=20)
    amount = models.DecimalField(max_digits=14, decimal_places=2)
    state = models.CharField(max_length=20, default='created')
    idempotency_key = models.CharField(max_length=64, db_index=True)
    external_transaction_id = models.CharField(max_length=128, blank=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=['provider', 'external_transaction_id'],
                condition=~models.Q(external_transaction_id=''),
                name='uniq_provider_external_txn'),
            models.UniqueConstraint(
                fields=['restaurant', 'idempotency_key'],
                name='uniq_restaurant_idem_key'),
        ]
```

4.8 Reconciliation (mos kelishni tekshirish)

> ⚡ Webhook yo'qolishi mumkin. **Kunlik reconciliation — yagona haqiqat manbai.**

```
# apps/integrations/payment/reconcile.py
# Cron: har kuni 04:00 (armenu-worker@analytics)

def reconcile_day(day: date, provider: str) -> dict:
    """
    Provayder GetStatement/ro'yxatini bizning Payment bilan solishtiradi.
    """
    external = PaymentRouter().execute(IntegrationRequest(
        op='statement', payload={'from': day, 'to': day}), prefer=provider).data

    ours = {p.external_transaction_id: p for p in Payment.objects.filter(
        provider=provider, created_at__date=day).exclude(external_transaction_id='')}

    report = {'ok': 0, 'missing_local': [], 'missing_remote': [], 'amount_mismatch': []}

    for txn in external:
        p = ours.pop(txn['id'], None)
        if p is None:
            # 🔴 Provayderda to'landi, bizda YO'Q → webhook yo'qolgan
            report['missing_local'].append(txn['id'])
            recover_lost_payment(provider, txn)          # avtomatik tiklash
        elif abs(p.amount - Decimal(txn['amount'])) > Decimal('0.01'):
            report['amount_mismatch'].append((p.id, p.amount, txn['amount']))
        else:
            report['ok'] += 1

    # Bizda paid, provayderda yo'q → 🔴 juda jiddiy
    report['missing_remote'] = [p.id for p in ours.values() if p.state == 'paid']

    if report['missing_local'] or report['missing_remote'] or report['amount_mismatch']:
        alert_oncall(f'🔴 P1 To\'lov nomuvofiqligi ({provider}, {day}): {report}')
    return report
```

Nomuvofiqlik	Ma'nosi	Harakat
`missing_local`	Provayderda to'langan, bizda yo'q	Webhook yo'qolgan → **avtomatik tiklash** + fiskal navbat
`missing_remote`	Bizda `paid`, provayderda yo'q	🔴 P1 — qo'lda tekshiruv, moliya bilan
`amount_mismatch`	Summa farq qiladi	🔴 P1 — qo'lda, audit log

4.9 Xato holatlari

Holat	Belgisi	Tizim harakati	Mijoz ko'radi
Provayder timeout	15 s javob yo'q	Circuit breaker +1, keyingi provayder	"Boshqa usul tanlang"
Mablag' yetarli emas	`-31001` / provayder kodi	⚡ `retryable=False` — zanjir davom etmaydi	"Mablag' yetarli emas"
Buyurtma allaqachon to'langan	`-4` (Click)	Idempotent → 200 OK	"✓ To'langan"
Ikkilangan webhook	Bir xil `transaction_id`	DB `UNIQUE` → oldingi javob	—
Barcha provayder yiqilgan	`ALL_PROVIDERS_FAILED`	🔴 alert + naqd rejimga o'tish	"Ofitsiantga naqd to'lang"
Summa mos emas	webhook amount ≠ Payment.amount	Rad etiladi + 🔴 P1 alert	"Xatolik, ofitsiantni chaqiring"
Qaytarish (refund)	Admin bosadi	`op='refund'` + `Order` restock	Push bildirishnoma

4.10 Test rejimi

Provayder	Sandbox	Test karta	Rejim
Payme	`test.paycom.uz`	Payme kabinetdan	`PAYME_TEST_MODE=1`
Click	`my.click.uz` (test service_id)	Click kabinetdan	`CLICK_TEST_MODE=1`
Uzum	`test-api.uzumbank.uz`	Uzum kabinetdan	`UZUM_TEST_MODE=1`

```
# ⚡ QAT'IY: prod'da test rejimi yoqilgan bo'lsa — ishga tushmaydi
# apps/integrations/payment/apps.py
def ready(self):
    if not settings.DEBUG and any([settings.PAYME_TEST_MODE,
                                   settings.CLICK_TEST_MODE,
                                   settings.UZUM_TEST_MODE]):
        raise ImproperlyConfigured(
            "🚨 PROD'da to'lov TEST rejimi yoqilgan! .env tekshiring.")
```

---

5. FISKAL INTEGRATSIYA (OFD)

5.1 Nima uchun majburiy

> 🇺🇿 O'zbekiston Respublikasi Soliq kodeksi: har bir naqd/karta to'lov uchun **fiskal chek** OFD (Operator Fiskal Ma'lumotlar) orqali Soliq qo'mitasiga yuborilishi **shart**. 2026-yildan **raqamli chek** (QR havola) qonuniy kuchga ega — qog'oz chop etish shart emas.

⚡ **Bu ARMENU'ning eng kuchli sotuv argumenti:** restoran allaqachon fiskal talabga duch keladi — biz uni menyu bilan birga hal qilamiz.

5.2 2 adapter + IKPU

```
┌──────────────────────────────────────────────────────┐
│ FiscalRouter                                         │
│   priority 10 → UZKASSA   (asosiy)                   │
│   priority 20 → e-POS     (zaxira)                   │
│   ikkalasi yiqilsa → NAVBAT (48 soat) → qo'lda kassa │
└──────────────────────┬───────────────────────────────┘
                       │
        ┌──────────────┴──────────────┐
        ▼                             ▼
┌──────────────┐                ┌──────────────┐
│    UZKASSA   │                │     e-POS    │
│ REST + token │                │ REST + token │
└──────┬───────┘                └──────┬───────┘
       └──────────────┬────────────────┘
                      ▼
              ┌──────────────┐
              │ Soliq qo'mitasi │
              └──────────────┘

┌──────────────────────────────────────────────────────┐
│ tasnif.soliq.uz — IKPU (MXIK) kod qidiruvi           │
│   har taom uchun majburiy tovar tasnif kodi          │
│   AI (Gemini) → nom bo'yicha taklif → admin tasdiqlaydi │
└──────────────────────────────────────────────────────┘
```

5.3 Adapter

```
# apps/integrations/fiscal/providers/uzkassa.py
import requests
import time
from decimal import Decimal
from django.conf import settings
from ..base import BaseAdapter, IntegrationRequest, IntegrationResponse

VAT_RATE = Decimal('12')          # QQS 12%


class UzkassaAdapter(BaseAdapter):
    name = 'uzkassa'
    domain = 'fiscal'
    priority = 10
    supported_ops = {'send_receipt', 'refund_receipt', 'check_status', 'close_shift'}

    BASE = 'https://api.uzkassa.uz/v1'

    def __init__(self):
        self.token = settings.UZKASSA_TOKEN
        self.terminal_id = settings.UZKASSA_TERMINAL_ID

    def is_configured(self):
        return bool(self.token and self.terminal_id)

    def execute(self, req: IntegrationRequest) -> IntegrationResponse:
        t0 = time.time()
        try:
            if req.op == 'send_receipt':
                return self._send(req, t0)
            if req.op == 'refund_receipt':
                return self._refund(req, t0)
            if req.op == 'check_status':
                return self._status(req, t0)
            return IntegrationResponse(ok=False, provider=self.name,
                                       error_code='OP_NOT_SUPPORTED')
        except Exception as exc:
            code, retryable = self.normalize_error(exc)
            return IntegrationResponse(ok=False, provider=self.name,
                                       error_code=code, error_msg=str(exc),
                                       retryable=retryable,
                                       latency_ms=int((time.time() - t0) * 1000))

    def _send(self, req, t0):
        o = req.payload                     # order dict (servis tayyorlaydi)
        body = {
            'terminal_id': self.terminal_id,
            'receipt_type': 0,                      # 0 = sotuv
            'received_cash': o['cash_tiyin'],
            'received_card': o['card_tiyin'],
            'external_id': req.idempotency_key,     # ⚡ OFD tomonda idempotency
            'items': [{
                'name':     it['name'][:128],
                'barcode':  '',
                'label':    '',
                'spic':     it['ikpu'],             # ⚡ IKPU kodi MAJBURIY
                'package_code': it.get('package_code', ''),
                'units':    it.get('units', 1),
                'price':    it['price_tiyin'],
                'amount':   it['qty'] * 1000,       # OFD: 1 dona = 1000
                'vat':      int(Decimal(it['price_tiyin']) * it['qty']
                                * VAT_RATE / (100 + VAT_RATE)),
                'vat_percent': int(VAT_RATE),
                'other':    0,
            } for it in o['items']],
        }
        r = requests.post(f'{self.BASE}/receipt', json=body, timeout=req.timeout,
                          headers={'Authorization': f'Bearer {self.token}'})
        latency = int((time.time() - t0) * 1000)

        if r.status_code >= 500:
            return IntegrationResponse(ok=False, provider=self.name,
                                       error_code='OFD_5XX', retryable=True,
                                       latency_ms=latency)
        data = r.json()
        if r.status_code == 200 and data.get('success'):
            return IntegrationResponse(
                ok=True, provider=self.name,
                external_id=data['fiscal_sign'],
                data={'fiscal_sign': data['fiscal_sign'],
                      'qr_url': data['qr_code_url'],
                      'receipt_date': data['date_time'],
                      'terminal_id': self.terminal_id},
                latency_ms=latency)

        # 4xx — bizning ma'lumotda xato, retry MA'NOSIZ
        return IntegrationResponse(ok=False, provider=self.name,
                                   error_code=data.get('error_code', 'OFD_4XX'),
                                   error_msg=data.get('message', ''),
                                   retryable=False, latency_ms=latency)
```

5.4 Chek yuborish oqimi

```
[1] Payment.state → 'paid'   (webhook yoki naqd tasdiqlash)
     │
     ▼ transaction.on_commit()
[2] enqueue('fiscal', {'order_id': ..., 'attempt': 0})
     │                        ⚡ WEB SO'ROVDA CHAQIRILMAYDI
     ▼
[3] armenu-worker@fiscal oladi
     │
     ▼
[4] Chek ma'lumoti yig'iladi
     ├─ har OrderItem uchun IKPU kodi (Dish.ikpu_code)
     ├─ ✖ IKPU yo'q → 🔴 alert + admin panelda "IKPU kiriting"
     ├─ QQS 12% hisoblanadi (narx ichida)
     └─ naqd/karta ajratiladi
     │
     ▼
[5] FiscalRouter.execute(op='send_receipt', idempotency_key=f'ord:{id}')
     ├─ UZKASSA (priority 10) ──── ✔ → [6]
     ├─ ✖ retryable → e-POS (20) ─ ✔ → [6]
     └─ ✖ ikkalasi ham → [7] NAVBAT
     │
     ▼
[6] FiscalReceipt.objects.create(
      order=..., provider='uzkassa', fiscal_sign=..., qr_url=..., status='sent')
     ├─ WS → mijoz: "🧾 Chek tayyor" + QR havola
     ├─ Telegram → mijozga raqamli chek (agar ulangan bo'lsa)
     └─ metrics.inc('fiscal.sent')
     │
     ▼ TUGADI ✔
[7] NAVBAT: FiscalReceipt(status='queued', next_retry_at=+5 daqiqa)
     └─ 48 soatlik retry jadvali ($5.5)
     └─ 🔴 alert: fiskal navbat depth > 50 → P1
```

5.5 48 soatlik retry jadvali

#	Kechikish	Jami o'tgan vaqt	Jitter	Alert
1	5 daqiqa	0:05	±20%	—
2	15 daqiqa	0:20	±20%	—
3	1 soat	1:20	±20%	🟡 P3
4	3 soat	4:20	±20%	🟡 P3
5	6 soat	10:20	±20%	🟠 P2
6	12 soat	22:20	±20%	🟠 P2
7	24 soat	46:20	±20%	🔴 P1
8	**oxirgi**	48:00	—	🔴 P1 + **qo'lda kassa talab**

```python
# apps/integrations/fiscal/retry.py
FISCAL_BACKOFF = [300, 900, 3600, 10800, 21600, 43200, 86400]   # soniya
FISCAL_DEADLINE = timedelta(hours=48)


def schedule_next(receipt: FiscalReceipt) -> bool:
    """Keyingi urinishni rejalashtiradi. False → 48 soat tugadi."""
    n = receipt.attempts
    if n >= len(FISCAL_BACKOFF) or \
       timezone.now() - receipt.created_at > FISCAL_DEADLINE:
        receipt.status = 'failed_manual'
        receipt.save(update_fields=['status'])
        alert_oncall(
            f"🔴 P1 FISKAL 48 SOAT: order #{receipt.order.number} "
            f"(rest={receipt.restaurant_id}) — QO'LDA KASSADA CHEK URILISHI SHART")
        notify_restaurant_admin(receipt.restaurant,
                                f"⚠ Chek yuborilmadi: #{receipt.order.number}. "
                                f"Kassada qo'lda urib qo'ying.")
        return False

    base = FISCAL_BACKOFF[n]
    delay = base * random.uniform(0.8, 1.2)        # ±20% jitter
    receipt.attempts = n + 1
    receipt.next_retry_at = timezone.now() + timedelta(seconds=delay)
    receipt.status = 'queued'
    receipt.save(update_fields=['attempts', 'next_retry_at', 'status'])
    enqueue_delayed('fiscal', receipt.id, delay=delay)
    return True
```

5.6 IKPU (MXIK) — tasnif.soliq.uz

> IKPU = Yagona elektron milliy tovarlar va xizmatlar katalogi kodi.
> **Har chek qatorida majburiy.** Restoran taomlari uchun odatda `10112001001…` guruhi.

```
┌──────────────────────────────────────────────────────────┐
│ IKPU QIDIRUV — 4 QATLAM                                  │
├──────────────────────────────────────────────────────────┤
│ [1] DISH.ikpu_code                                       │
│     Admin qo'lda kiritgan → darhol ishlatiladi ✔         │
├──────────────────────────────────────────────────────────┤
│ [2] REDIS KESH                                           │
│     ikpu:q:<sha256(nom)> → kod    TTL 30 kun             │
├──────────────────────────────────────────────────────────┤
│ [3] LOKAL SNAPSHOT (PostgreSQL)                          │
│     IkpuCode jadvali — oyda 1 marta to'liq yuklab olinadi│
│     ~100k qator, GIN trigram indeks (pg_trgm)            │
│     ⚡ tasnif.soliq.uz yiqilsa ham ishlaydi              │
├──────────────────────────────────────────────────────────┤
│ [4] tasnif.soliq.uz API (jonli)                          │
│     GET /api/cls-api/mxik/search?text=<nom>&lang=uz      │
│     timeout 10s, circuit breaker 3/300s                  │
├──────────────────────────────────────────────────────────┤
│ [5] FALLBACK: AI (Gemini) + kategoriya standart kodi     │
│     "Osh" → 10112001001 (tayyor taomlar) — admin TASDIQLAYDI │
└──────────────────────────────────────────────────────────┘
```

```python
# apps/integrations/fiscal/ikpu.py
DEFAULT_IKPU_BY_CATEGORY = {
    'main':     '10112001001',   # tayyor taomlar
    'soup':     '10112001002',
    'salad':    '10112001003',
    'drink':    '10112002001',
    'dessert':  '10112003001',
    'bakery':   '10112004001',
}


def resolve_ikpu(dish) -> tuple[str, str]:
    """(kod, manba). Manba: manual|cache|local|api|ai_suggest|default"""
    if dish.ikpu_code:
        return dish.ikpu_code, 'manual'

    key = f'ikpu:q:{hashlib.sha256(dish.name_uz.encode()).hexdigest()[:24]}'
    if (hit := cache.get(key)):
        return hit, 'cache'

    # [3] lokal snapshot — trigram similarity
    row = IkpuCode.objects.filter(name__trigram_similar=dish.name_uz) \
                          .order_by('-similarity').first()
    if row:
        cache.set(key, row.code, 2592000)
        return row.code, 'local'

    # [4] jonli API
    resp = ikpu_breaker.call(_soliq_search, dish.name_uz, fallback=lambda: None)
    if resp:
        cache.set(key, resp, 2592000)
        return resp, 'api'

    # [5] AI taklifi (admin tasdiqlashi kerak)
    ai = mcp.run(MCPRequest(
        task='chat', json_mode=True, max_tokens=60,
        system='Sen O\'zbekiston IKPU (MXIK) tasniflagichi mutaxassisisan. '
               'Faqat JSON qaytaring: {"code": "..."}',
        prompt=f'Restoran taomi: "{dish.name_uz}". Eng mos IKPU kodi?'))
    if ai.ok and (code := ai.content.get('code')):
        dish.ikpu_suggested = code
        dish.save(update_fields=['ikpu_suggested'])
        notify_restaurant_admin(dish.restaurant,
            f"✨ \"{dish.name_uz}\" uchun IKPU taklif: {code} — tasdiqlang')
        return code, 'ai_suggest'

    return DEFAULT_IKPU_BY_CATEGORY.get(dish.category.kind, '10112001001'), 'default'
```

```
# Oylik lokal snapshot yangilash (cron: har oy 1-sana 02:00)
/home/armenu/app/venv/bin/python manage.py sync_ikpu_catalog --full
```

5.7 Raqamli chek (2026 dan qonuniy)



Yetkazish kanali	Qachon	Saqlash
Mijoz sahifasi (WS)	Darhol	Sessiya davomida
Telegram (bot ulangan bo'lsa)	Darhol	Doimiy
PDF yuklab olish	Talab bo'yicha	—
Admin panel (arxiv)	Doimiy	**5 yil** (soliq talabi)

---

6. TELEGRAM

6.1 Ikki komponent

Komponent	Nima uchun	Kim ishlatadi
Bot (`@armenu_bot`)	Bildirishnoma, chek, buyurtma holati	Mijoz, restoran admini, ofitsiant
Mini App	Ilova o'rnatmasdan to'liq menyu + buyurtma	Mijoz (Telegram ichidan)

> 🇺🇿 O'zbekistonda Telegram — **27 mln foydalanuvchi**. Mini App = ilovasiz kanal.

6.2 Arxitektura



6.3 Webhook vs polling

	Webhook	Polling
Kechikish	⚡ ~50 ms	1-3 s
Server yuki	Past (push)	Doimiy so'rov
Talab	HTTPS + haqiqiy sertifikat	Yo'q
Bizda	♂ **prod**	🔘 dev/lokal

```
# Prod: webhook o'rnatish
# manage.py setup_telegram_webhook
requests.post(f'https://api.telegram.org/bot{TOKEN}/setWebhook', json={
    'url': f'https://armenu.uz/webhooks/telegram/{settings.TG_SECRET_PATH}/',
    'secret_token': settings.TG_WEBHOOK_SECRET,     # ⚡ header'da qaytadi
    'allowed_updates': ['message', 'callback_query', 'pre_checkout_query'],
    'max_connections': 40,
    'drop_pending_updates': False,
})
```

```
@csrf_exempt
@require_POST
def telegram_webhook(request, secret_path):
    # [1] ikki qatlamli tekshiruv: URL yo'li + header
    if not hmac.compare_digest(secret_path, settings.TG_SECRET_PATH):
        raise Http404
    if not hmac.compare_digest(
            request.headers.get('X-Telegram-Bot-Api-Secret-Token', ''),
            settings.TG_WEBHOOK_SECRET):
        return HttpResponse(status=403)

    update = json.loads(request.body)
    # [2] xom saqlash + [3] navbat + [4] darhol 200
    #     ↳ Telegram 60 s ichida javob kutadi; biz ~5 ms da qaytaramiz
    WebhookEvent.objects.create(provider='telegram', raw=update)
    enqueue('notify', {'kind': 'tg_update', 'update': update})
    return HttpResponse(status=200)
```

6.4 Xabar navbati va rate limit

> Telegram cheklovlari: **30 msg/s** butun bot bo'yicha, **~20 msg/daq** bitta guruhga,
> **1 msg/s** bitta shaxsiy chatga (amaliy chegara).

```
# apps/integrations/messaging/ratelimit.py
class TokenBucket:
    """Redis'ga asoslangan — bir nechta worker bo'lsa ham global limit."""

    def __init__(self, key, rate, capacity):
        self.key, self.rate, self.capacity = key, rate, capacity

    def acquire(self, timeout=5.0) -> bool:
        deadline = time.time() + timeout
        while time.time() < deadline:
            if self._try():                 # Lua skript: atomik
                return True
            time.sleep(0.05)
        return False


GLOBAL_BUCKET = TokenBucket('tg:global', rate=28, capacity=28)      # 30 dan past
CHAT_BUCKETS = lambda cid: TokenBucket(f'tg:chat:{cid}', rate=1, capacity=3)


def send_message(chat_id, text, **kw):
    if not GLOBAL_BUCKET.acquire() or not CHAT_BUCKETS(chat_id).acquire():
        requeue(chat_id, text, kw, delay=2)
        return

    r = requests.post(f'{API}/sendMessage',
                      json={'chat_id': chat_id, 'text': text, **kw}, timeout=10)

    if r.status_code == 429:                        # ⚡ Telegram o'zi aytadi
        retry_after = r.json().get('parameters', {}).get('retry_after', 5)
        metrics.inc('telegram.429')
        requeue(chat_id, text, kw, delay=retry_after + 1)
        return
    if r.status_code == 403:                        # bot bloklangan
        TelegramLink.objects.filter(chat_id=chat_id).update(is_blocked=True)
        return
    if r.status_code >= 500:
        requeue(chat_id, text, kw, delay=30)
```

Navbat prioriteti (Redis ZSET):

Prioritet	Xabar turi	Maks kechikish
0 (eng yuqori)	Yangi buyurtma → ofitsiant	2 s
1	Chek / to'lov tasdig'i	5 s
2	Buyurtma holati o'zgardi	15 s
5	Sodiqlik / keshbek	5 daqiqa
9 (eng past)	Marketing / e'lon	1 soat

6.5 Mini App

```
Mijoz: Telegram → @armenu_bot → [🍽 Menyu] tugma
        │
        ▼
  Mini App ochiladi: https://armenu.uz/tg/<restaurant_slug>/
        │  window.Telegram.WebApp.initData
        ▼
  POST /v1/tg/auth/  { init_data: "<xom string>" }
        │
        ▼  ⚡ Server initData imzosini TEKSHIRADI
        │     secret = HMAC_SHA256("WebAppData", BOT_TOKEN)
        │     hash  = HMAC_SHA256(secret, data_check_string)
        │     auth_date 24 soatdan eski bo'lsa → RAD
        ▼
  GuestSession yaratiladi (telegram_user_id bilan bog'lanadi)
        │
        ▼
  Menyu → savat → buyurtma → to'lov
  (bir xil API, faqat auth manbai boshqa)
```

```
def verify_telegram_init_data(init_data: str, bot_token: str) -> dict | None:
    parsed = dict(urllib.parse.parse_qsl(init_data))
    received_hash = parsed.pop('hash', '')
    data_check = '\n'.join(f'{k}={parsed[k]}' for k in sorted(parsed))
    secret = hmac.new(b'WebAppData', bot_token.encode(), hashlib.sha256).digest()
    calc = hmac.new(secret, data_check.encode(), hashlib.sha256).hexdigest()
    if not hmac.compare_digest(calc, received_hash):
        return None
    if int(time.time()) - int(parsed.get('auth_date', 0)) > 86400:
        return None                    # ⚡ 24 soatdan eski
    return json.loads(parsed.get('user', '{}'))
```

---

7. CDN VA FAYL SAQLASH

7.1 Nima uchun kritik

> 3D fayl yuklanmasa — **AR yo'q**. AR yo'q bo'lsa — ARMENU'ning asosiy qiymati yo'q.
> Shuning uchun CDN kritiklik darajasi 🔴 **Yuqori**.

7.2 Oqim



7.3 3D fayl MIME turlari — ⚡ KRITIK

> Noto'g'ri `Content-Type` → iOS AR Quick Look **ochilmaydi**, Android Scene Viewer **xato beradi**. Bu eng ko'p uchraydigan AR bug'i.

Kengaytma	MIME	Platforma	Maks hajm
`.glb`	`model/gltf-binary`	Android (Scene Viewer), web `<model-viewer>`	**3 MB**
`.gltf`	`model/gltf+json`	web	3 MB
`.usdz`	`model/vnd.usdz+zip`	⚡ **iOS AR Quick Look**	**5 MB**
`.bin`	`application/octet-stream`	gltf yordamchi	—
`.ktx2`	`image/ktx2`	siqilgan tekstura	1 MB
`.basis`	`application/octet-stream`	siqilgan tekstura	1 MB
`.hdr`	`image/vnd.radiance`	muhit yorug'ligi	2 MB
`.webp`	`image/webp`	fallback rasm	300 KB

```
# /etc/nginx/mime.types.armenu  (nginx.conf http{} blokida include qilinadi)
types {
    model/gltf-binary       glb;
    model/gltf+json         gltf;
    model/vnd.usdz+zip      usdz;
    image/ktx2              ktx2;
    image/vnd.radiance      hdr;
    application/wasm        wasm;
}
```

7.4 CORS — AR uchun kritik

> `<model-viewer>` faylni `fetch()` bilan oladi. CORS header bo'lmasa — brauzer bloklaydi va AR ishga tushmaydi. Xato konsolda **jim** ko'rinadi.

```
location /media/ar/ {
    alias /var/armenu/media/ar/;

    # ⚡ AR uchun MAJBURIY
    add_header Access-Control-Allow-Origin  "*"                 always;
    add_header Access-Control-Allow-Methods "GET, HEAD, OPTIONS"  always;
    add_header Access-Control-Allow-Headers "Range, Content-Type"  always;
    add_header Access-Control-Expose-Headers "Content-Length, Content-Range, Accept-Ranges" always;
    add_header Cross-Origin-Resource-Policy "cross-origin"        always;

    # Range so'rovlar — katta GLB progressiv yuklanishi uchun
    add_header Accept-Ranges bytes always;

    # Kesh
    expires 30d;
    add_header Cache-Control "public, max-age=2592000, immutable" always;

    # Preflight
    if ($request_method = OPTIONS) {
        add_header Access-Control-Max-Age 86400;
        add_header Content-Length 0;
        return 204;
    }

    # Fayl yo'q → rasm fallback
    try_files $uri @s3_origin;
}

location @s3_origin {
    proxy_pass https://armenu-media.s3.amazonaws.com;
    proxy_cache ar_cache;
    proxy_cache_valid 200 30d;
    proxy_cache_use_stale error timeout updating http_500 http_502 http_503;
}
```

7.5 Cloudflare konfiguratsiyasi

Sozlama	Qiymat	Nega
Proxy (orange cloud)	ON	DDoS + WAF + kesh
SSL/TLS rejimi	Full (strict)	Origin sertifikati tekshiriladi
Min TLS	1.2	Xavfsizlik
Brotli	ON	`.gltf` (JSON) yaxshi siqiladi
Auto Minify	JS+CSS (HTML yo'q)	Sahifa tezligi
HTTP/3 (QUIC)	ON	Mobil tarmoqda tezroq
Early Hints	ON	3D fayl preload
Polish	Lossy + WebP	Taom rasmlari
Bot Fight Mode	ON (webhook'lar bypass)	Skraping himoyasi
Rate limiting	300 req/min/IP	Qo'shimcha qatlam

Cache Rules:

#	Shart	Harakat	Edge TTL	Brauzer TTL
1	`URI path` starts with `/media/ar/`	Cache Everything	**30 kun**	30 kun
2	`URI path` starts with `/static/`	Cache Everything	**1 yil**	1 yil
3	`URI path` starts with `/media/dish/`	Cache Everything	7 kun	1 kun
4	`URI path` starts with `/api/menu/`	Cache Everything (+ `Vary: Accept-Language`)	**60 s**	0
5	`URI path` starts with `/api/orders/`	**Bypass**	—	—
6	`URI path` starts with `/webhooks/`	**Bypass** + Bot Fight OFF	—	—
7	`URI path` starts with `/ws/`	**Bypass** (WebSocket)	—	—
8	`URI path` starts with `/admin/`	**Bypass**	—	—

```
# Kesh tozalash (deploy'dan keyin, faqat kerakli prefiks)
curl -X POST "https://api.cloudflare.com/client/v4/zones/${CF_ZONE}/purge_cache" \
  -H "Authorization: Bearer ${CF_TOKEN}" \
  -H "Content-Type: application/json" \
  --data '{"prefixes":["armenu.uz/static/","armenu.uz/api/menu/"]}'
```

7.6 S3 bucket tuzilishi

```
s3://armenu-media/                      (versioning ON, lifecycle bor)
├── ar/
│   ├── library/                        ⚡ UMUMIY 3D KUTUBXONA (tenant'siz)
│   │   ├── osh/
│   │   │   ├── model.glb               (2,1 MB)
│   │   │   ├── model.usdz              (3,4 MB)
│   │   │   ├── thumb.webp              (28 KB)
│   │   │   └── meta.json               (o'lcham, poly, litsenziya)
│   │   ├── shashlik/  manti/  lagman/  ...
│   │   └── _index.json                 (semantik qidiruv uchun)
│   │
│   └── r/<restaurant_id>/              ⚡ TENANT-SCOPED
│       └── dish/<dish_id>/
│           ├── v1/model.glb
│           ├── v2/model.glb            (versiyalar saqlanadi)
│           └── current -> v2           (metadata'da)
│
├── dish/<restaurant_id>/<dish_id>/     (taom rasmlari)
│   ├── original.jpg   large.webp   medium.webp   thumb.webp
│
├── qr/<restaurant_id>/                 (stol QR PDF)
│   └── table_<n>.pdf
│
├── receipt/<restaurant_id>/<yyyy>/<mm>/ (fiskal chek PDF, 5 yil)
│
└── backup/                             (alohida bucket tavsiya etiladi)
    ├── db/YYYY-MM-DD/
    └── wal/
```

Bucket siyosati:

Prefiks	Ruxsat	Lifecycle
`ar/library/`	Public read (CDN origin)	— (doimiy)
`ar/r/*/`	Public read (URL taxmin qilib bo'lmaydi — uuid)	Eski versiya 90 kundan keyin Glacier
`dish/`	Public read	—
`receipt/`	⛔ **Private** (faqat imzolangan URL, 15 daq)	**5 yil**, keyin Glacier
`backup/`	⛔ **Private** + MFA delete	30 kun → Glacier → 1 yil

7.7 Origin himoyasi

```
# Origin'ga faqat Cloudflare kirsin
# /etc/nginx/cloudflare-ips.conf (cron bilan yangilanadi)
allow 173.245.48.0/20;
allow 103.21.244.0/22;
# ... (https://www.cloudflare.com/ips-v4)
deny all;

# Va real IP tiklanadi
set_real_ip_from 173.245.48.0/20;
# ...
real_ip_header CF-Connecting-IP;
```

```
# Cloudflare IP ro'yxatini yangilash (cron: haftada 1)
#!/bin/bash
# /usr/local/bin/update-cf-ips.sh
OUT=/etc/nginx/cloudflare-ips.conf
{
  for ip in $(curl -fsS https://www.cloudflare.com/ips-v4); do echo "allow $ip;"; done
  for ip in $(curl -fsS https://www.cloudflare.com/ips-v6); do echo "allow $ip;"; done
  echo "deny all;"
} > "$OUT.tmp" && mv "$OUT.tmp" "$OUT" && nginx -t && systemctl reload nginx
```

8. CHIQUVCHI INTEGRATSIYA JADVALI

8.1 To'liq jadval

Xizmat	Op	SLA (p95)	Timeout	Retry	CB (xato/kutish)	Fallback	Oylik $
Claude	chat/translate	1,4 s	30 s	2× (0,5s, 1,5s)	3 / 120 s	→ Gemini	31
Gemini	chat/translate/vision	0,8 s	30 s	2×	3 / 120 s	→ Groq	42
OpenAI	chat/vision	1,8 s	30 s	2×	3 / 120 s	→ Mistral	0 (zaxira)
Groq	chat (tez)	0,3 s	20 s	2×	3 / 120 s	→ Claude	15
DeepSeek	chat/translate	1,1 s	30 s	2×	3 / 120 s	→ OpenAI	5
Mistral	chat/translate	1,2 s	30 s	2×	3 / 120 s	→ (oxirgi)	0 (zaxira)
Meshy	image_to_3d	4 daq	600 s	3× (1m,5m,15m)	2 / 600 s	→ Tripo	391
Tripo	image_to_3d	6,5 daq	600 s	3×	2 / 600 s	→ Rodin/qo'lda	3
Payme	invoice/refund	0,6 s	15 s	2×	3 / 60 s	→ Click	0*
Click	prepare/complete	0,9 s	15 s	2×	3 / 60 s	→ Uzum	0*
Uzum	charge/refund	0,7 s	15 s	2×	3 / 60 s	→ **naqd**	0*
UZKASSA	send_receipt	1,2 s	25 s	8× (48 soat)	3 / 120 s	→ e-POS	25**
e-POS	send_receipt	1,5 s	25 s	8× (48 soat)	3 / 120 s	→ navbat	0 (zaxira)
tasnif.soliq.uz	ikpu search	2,0 s	10 s	1×	3 / 300 s	→ lokal snapshot	0
Telegram	sendMessage	0,3 s	10 s	5× (1s...10m)	5 / 60 s	→ WS/brauzer push	0
Cloudflare	CDN	30 ms	—	—	—	→ origin	20
S3/MinIO	get/put	120 ms	30 s	3×	5 / 60 s	→ lokal disk	45
Google Places	reviews	1,5 s	12 s	1×	3 / 600 s	→ yashiriladi	8
CBU kurs	rate	0,5 s	10 s	1×	3 / 3600 s	→ oxirgi kurs	0
						JAMI	**~$585**

* To'lov provayderlari komissiya oladi (tranzaksiyadan %), oylik abonent to'lov yo'q.
** OFD har chek uchun to'lov (~50-100 so'm/chek) yoki oylik paket.

8.2 Circuit breaker sozlamalari — nega turlicha

Domen	fail_threshold	timeout	Sabab
AI	3	120 s	Ko'p alternativa bor — tez chetlab o'tish arzon
To'lov	3	60 s	Tez tiklanishi kerak (pul) — qisqa timeout
Fiskal	3	120 s	Navbat bor, shoshilish yo'q
3D (Meshy/Tripo)	2	600 s	Qimmat operatsiya — behuda urinmaslik
Telegram	5	60 s	Vaqtinchalik 429 normal — chidamliroq
S3	5	60 s	Yadro infratuzilma — juda tez ochilmasin
tasnif.soliq.uz	3	300 s	Davlat servisi — uzoq yiqiladi, lokal snapshot bor

---

9. KIRUVCHI INTEGRATSIYA (BIZNING API)

Holat: **v2 rejasi** (yil 5 — "Ochiq API" bosqichi). Arxitektura hozirdan tayyorlanadi,
chunki keyinchalik qo'shish qiyin.

9.1 Kim foydalanadi

Iste'molchi	Nima uchun	Prioritet
Restoran POS tizimi (iiko, r_keeper)	Buyurtmani o'z tizimiga olish	Yuqori
Delivery agregator	Menyu + narx sinxronizatsiya	O'rta
Buxgalteriya dasturi	To'lov/chek eksporti	O'rta
Restoran tarmog'i BI	Analitika	Past

9.2 Versiyalash

```
https://api.armenu.uz/v1/restaurants/<id>/menu/
                     ^^^
                     URL'da versiya (eng oddiy, kesh-do'st)

QOIDALAR:
  • Yangi maydon qo'shish       → versiya O'ZGARMAYDI (backward compatible)
  • Maydon o'chirish/nomi o'zgarishi → yangi versiya (v2)
  • v1 va v2 kamida 12 oy parallel ishlaydi
  • Eskirish: Deprecation + Sunset header (RFC 8594)
```

```
HTTP/1.1 200 OK
Deprecation: Sat, 01 Jun 2027 00:00:00 GMT
Sunset: Tue, 01 Jun 2028 00:00:00 GMT
Link: <https://docs.armenu.uz/api/v2>; rel="successor-version"
```

9.3 API kalit

```
class ApiKey(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=models.CASCADE)  # ⚡ tenant
    name = models.CharField(max_length=80)              # "iiko integratsiya"
    key_prefix = models.CharField(max_length=12, db_index=True)   # 'ak_live_a3f2'
    key_hash = models.CharField(max_length=128)         # ⚡ argon2, xom kalit SAQLANMAYDI
    scopes = models.JSONField(default=list)             # ['menu:read', 'orders:read']
    rate_limit_per_min = models.IntegerField(default=120)
    allowed_ips = models.JSONField(default=list)        # bo'sh = hamma
    expires_at = models.DateTimeField(null=True)
    last_used_at = models.DateTimeField(null=True)
    revoked_at = models.DateTimeField(null=True)
```

```
GET /v1/restaurants/42/menu/ HTTP/1.1
Host: api.armenu.uz
Authorization: Bearer ak_live_a3f2c81d9e4b...
X-Api-Version: 1
```

Scope'lar:

Scope	Ruxsat
`menu:read`	Menyu, kategoriya, taom, narx
`menu:write`	Taom qo'shish/o'zgartirish, stop-list
`orders:read`	Buyurtmalar ro'yxati, holat
`orders:write`	Holat o'zgartirish (tayyor/berildi)
`payments:read`	To'lov holati (✖ karta ma'lumoti hech qachon)
`analytics:read`	Agregat hisobotlar
`webhooks:manage`	Webhook obunasi

⚡ **Tenant izolyatsiya:** API kalit **bitta restoranga** bog'langan.
`/v1/restaurants/<id>/` da `<id>` kalitning restoraniga teng bo'lmasa →**403**.
Bu `test_tenant_isolation.py` da tekshiriladi.

9.4 Rate limit

Tarif	So'rov/daqiqa	Burst	Kunlik
Free	60	20	10 000
Standart	120	40	100 000
Premium	600	200	1 000 000

```
HTTP/1.1 200 OK
X-RateLimit-Limit: 120
X-RateLimit-Remaining: 117
X-RateLimit-Reset: 1757340000

# Oshib ketsa:
HTTP/1.1 429 Too Many Requests
Retry-After: 23
```

9.5 Chiquvchi webhook (biz yuboradigan)



Event	Qachon	Payload
`order.created`	Yangi buyurtma	order, items, table
`order.status_changed`	Holat o'zgardi	order_id, old, new
`order.completed`	Yopildi	order, total
`payment.succeeded`	To'landi	payment, order_id
`payment.refunded`	Qaytarildi	payment, amount
`receipt.issued`	Fiskal chek	fiscal_sign, qr_url
`menu.updated`	Menyu o'zgardi	dish_ids
`dish.out_of_stock`	Stop-list	dish_id

```
POST https://client.example.com/armenu-hook HTTP/1.1
Content-Type: application/json
X-ARMenu-Event: order.created
X-ARMenu-Delivery: 3f2a9c81-...          ← ⚡ idempotency uchun
X-ARMenu-Timestamp: 1757340000
X-ARMenu-Signature: sha256=8f3d...       ← HMAC(secret, ts + '.' + body)

{"event":"order.created","data":{...},"restaurant_id":42}
```

Yetkazish siyosati:

Parametr	Qiymat
Timeout	10 s
Muvaffaqiyat	HTTP 2xx
Retry	6× — 10 s, 1 daq, 5 daq, 30 daq, 2 soat, 6 soat
Deadline	24 soat
Avtomatik o'chirish	100 ketma-ket xato → obuna `paused` + email
Tartib	Kafolatlanmaydi (`timestamp` bilan tartiblang)
Takror	⚡ Bo'lishi mumkin — `X-ARMenu-Delivery` bo'yicha dedup qiling

---

10. INTEGRATSIYA TESTLARI

10.1 Test piramidasi



10.2 Mock strategiyasi

Qatlam	Vosita	Nima mock qilinadi
HTTP	`responses` / `respx`	Tashqi URL javoblari
Yozib olish	`vcrpy` cassette	Real javob 1 marta yoziladi, keyin qayta ijro
Vaqt	`freezegun`	Retry jadvali, timestamp oynalari
Navbat	`pytest` fixture	Worker sinxron ishlaydi
Redis	`fakeredis`	Kesh, circuit breaker, token bucket

```
# tests/integrations/conftest.py
@pytest.fixture
def mock_payme_success(responses):
    responses.add(responses.POST, 'https://checkout.paycom.uz/api',
                  json={'result': {'state': 2}}, status=200)


@pytest.fixture
def mock_all_ai_fail(monkeypatch):
    """Barcha AI provayderlarni yiqitadi — degradation tekshiruvi"""
    from apps.integrations.ai.base import MCPProvider, MCPResponse
    monkeypatch.setattr(MCPProvider, 'call',
                        lambda self, req: MCPResponse(ok=False, error='mocked'))


@pytest.fixture(autouse=True)
def no_real_network(monkeypatch):
    """🛡 Testda HECH QACHON real tarmoqqa chiqilmasin"""
    import socket
    def guard(*a, **kw):
        raise RuntimeError('✖ Testda real tarmoq chaqiruvi! Mock qo\'ying.')
    monkeypatch.setattr(socket.socket, 'connect', guard)
```

10.3 Kontrakt testlari — ⚡ har adapter uchun avtomatik

```
# tests/integrations/test_adapter_contract.py
"""
Har bir adapter interfeysga mos ekanini tekshiradi.
Yangi adapter qo'shilsa — TEST AVTOMATIK QO'LLANADI (registry'dan olinadi).
"""
import pytest
from apps.integrations.registry import ALL_ADAPTERS
from apps.integrations.base import BaseAdapter, IntegrationRequest, IntegrationResponse


@pytest.mark.parametrize('cls', ALL_ADAPTERS, ids=lambda c: c.name)
class TestAdapterContract:

    def test_meros(self, cls):
        assert issubclass(cls, BaseAdapter)

    def test_metadata(self, cls):
        a = cls()
        assert a.name and a.domain
        assert isinstance(a.supported_ops, set) and a.supported_ops
        assert isinstance(a.priority, int)

    def test_is_configured_exception_bermaydi(self, cls):
        cls().is_configured()          # raise qilmasligi kerak

    def test_execute_response_qaytaradi(self, cls, mock_network_error):
        """⚠ ENG MUHIM: tarmoq yiqilsa ham exception EMAS, Response"""
        resp = cls().execute(IntegrationRequest(op=next(iter(cls.supported_ops))))
        assert isinstance(resp, IntegrationResponse)
        assert resp.ok is False
        assert resp.error_code
        assert isinstance(resp.retryable, bool)

    def test_nomalum_op(self, cls):
        resp = cls().execute(IntegrationRequest(op='__yoq_op__'))
        assert resp.ok is False and resp.error_code == 'OP_NOT_SUPPORTED'

    def test_maxfiy_malumot_log_da_yoq(self, cls, caplog):
        cls().execute(IntegrationRequest(op=next(iter(cls.supported_ops)),
                                         payload={'card': '8600123412341234'}))
        assert '8600123412341234' not in caplog.text
```

10.4 Sandbox muhitlar

Xizmat	Sandbox	Qachon ishlatiladi	Kalit manbai
Payme	`test.paycom.uz`	Nightly E2E	`.env.staging`
Click	test `service_id`	Nightly E2E	`.env.staging`
Uzum	`test-api.uzumbank.uz`	Nightly E2E	`.env.staging`
UZKASSA	test terminal	Nightly E2E	`.env.staging`
Telegram	alohida test bot	Har PR	GitHub secret
AI	✍ real (arzon model, `max_tokens=10`)	Nightly	GitHub secret
S3	MinIO konteyner	Har commit	—
tasnif.soliq.uz	✍ real (o'qish)	Nightly	—

```
# .github/workflows/nightly-integration.yml
name: Nightly integratsiya (sandbox)
on:
  schedule: [{cron: '0 1 * * *'}]        # har kuni 01:00 UTC
  workflow_dispatch:

jobs:
  sandbox:
    runs-on: ubuntu-latest
    environment: staging               # ⚠ sirlar faqat shu muhitda
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12'}
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - name: Sandbox testlari
        env:
          PAYME_MERCHANT_ID: ${{ secrets.PAYME_TEST_MERCHANT_ID }}
          PAYME_KEY:         ${{ secrets.PAYME_TEST_KEY }}
          UZKASSA_TOKEN:     ${{ secrets.UZKASSA_TEST_TOKEN }}
          TELEGRAM_BOT_TOKEN: ${{ secrets.TG_TEST_BOT_TOKEN }}
          GEMINI_API_KEY:    ${{ secrets.GEMINI_API_KEY }}
        run: pytest tests/integrations/sandbox/ -v --tb=short -m sandbox
      - name: Xato bo'lsa Telegram'ga
        if: failure()
        run: |
          curl -s -X POST "https://api.telegram.org/bot${{ secrets.TG_ALERT_BOT }}/sendMessage" \
            -d chat_id=${{ secrets.TG_ALERT_CHAT }} \
            -d text="🔴 Nightly sandbox testlari yiqildi: ${{ github.run_id }}"
```

10.5 Majburiy integratsiya testlari (deploy bloklaydi)

```
tests/integrations/
├── test_adapter_contract.py       # ⚠ har adapter interfeysga mos
├── test_payment_idempotency.py    # ⚠ dublikat to'lov = 0
├── test_payment_signature.py      # ⚠ soxta imzo → 401
├── test_payment_amount_check.py   # ⚠ summa serverdan, webhook'ga ishonilmaydi
├── test_fiscal_retry_schedule.py  # ⚠ 48 soat, 8 urinish
├── test_fiscal_ikpu_fallback.py   # 4 qatlam ishlaydi
├── test_mcp_fallback_chain.py     # ⚠ 2 provayder yiqilsa 3-si ishlaydi
├── test_mcp_budget.py             # byudjet oshsa to'xtaydi
├── test_mcp_circuit_breaker.py    # 3 xato → OPEN
├── test_telegram_ratelimit.py     # 30 msg/s oshmaydi
├── test_telegram_initdata.py      # ⚠ soxta initData → rad
├── test_cdn_mime_types.py         # ⚠ .usdz → model/vnd.usdz+zip
├── test_cors_headers.py           # ⚠ AR uchun CORS bor
└── test_no_secrets_in_logs.py     # ⚠ kalit/karta log'da yo'q
```

```
# Namuna: eng kritik test
def test_webhook_summani_ozgartira_olmaydi(order, payme_webhook_body):
    """
    ⚠ OLTIN QOIDA: provayder yuborgan summa Payment.amount ga TENG bo'lishi shart.
    Aks holda to'lov QABUL QILINMAYDI.
    """
    order.total = Decimal('187000')
    order.save()
    body = payme_webhook_body(order, amount=1000)      # tajovuzkor 10 so'm yubordi

    resp = client.post('/webhooks/payme/', body,
                       content_type='application/json', **valid_auth_header())

    assert resp.json()['error']['code'] == ERR_INVALID_AMOUNT
    order.refresh_from_db()
    assert order.payment_status != 'paid'
```

10.6 Chaos testlari (oyiga 1 marta, staging)

Test	Kutilgan natija
Barcha AI provayderni bloklash (iptables)	Chat “Ofitsiantni chaqiring“ beradi, sayt ishlaydi
Payme + Click bloklash	Uzum ishlaydi; u ham yiqilsa → naqd rejimi
OFD bloklash 2 soat	Buyurtma/to'lov ishlaydi, cheklar navbatda, 2 soatdan keyin yuboriladi
S3 bloklash	Menyu ishlaydi, 3D o'rniga rasm
Telegram bloklash	WS bildirishnoma ishlaydi, navbat to'ldi → keyin yuboriladi
tasnif.soliq.uz bloklash	Lokal snapshot'dan IKPU topiladi
Tarmoq kechikishi +2 s	Timeout va circuit breaker ishlaydi, kaskad yo'q

QISM 8

Deploy va operatsiya

Versiya: 1.0 · **Sana:** 2026-09-08
Kim uchun: DevOps · backend dasturchi · on-call navbatchi · texnik direktor
Oldingi: `07_INTEGRATSIYALAR.md` · **Bu — arxitektura hujjatining oxirgi qismi**

1. MUHITLAR

1.1 Uch muhit

	dev	staging	prod
Maqsad	Lokal ishlab chiqish	Sinov, pilot demo, chaos	Ishlab chiqarish
Domen	`localhost:8060`	`stage.armenu.uz`	`armenu.uz`
Server	Dasturchi noutbuki	1 × VPS (2 CPU / 4 GB)	S0→S5 (§2)
DB	`armenu_dev` (lokal PG)	`armenu_stage`	`armenu_db`
Redis db	15 (lokal)	5 / 6 (stage instance)	**5 (kesh) / 6 (channels)**
Ma'lumot	Demo seed (5 restoran)	⚡ **Anonimlashtirilgan** prod nusxasi	Real
To'lov	Mock adapter	Sandbox (test.paycom.uz)	🟢 Real
Fiskal	Mock adapter	Test terminal	🟢 Real OFD
AI	Real (arzon model, past limit)	Real (limit $5/kun)	Real ($500/oy)
CDN	Yo'q (nginx to'g'ridan)	Cloudflare (dev rejim)	Cloudflare (full)
Debug	`DEBUG=True`	`DEBUG=False`	`DEBUG=False`
Kirish	Dasturchi	Jamoa + pilot restoranlar	🔒 Faqat on-call + audit
Backup	Yo'q	Haftalik	⚡ Kunlik + WAL
Monitoring	Yo'q	Prometheus (yengil)	To'liq stek
Deploy	`runserver`	`main` push (auto)	⚡ **Tag push + tasdiq**
Xarajat	$0	~$12/oy	$40 → $4 400/oy

1.2 Anonimlashtirish (staging uchun)

> ⚡ Prod ma'lumoti staging'ga **hech qachon xom holda** ko'chirilmaydi (GDPR + O'zbekiston shaxsiy ma'lumot qonuni).

```
# manage.py anonymize_dump — prod dump'ni staging uchun tozalaydi
/home/armenu/app/venv/bin/python manage.py anonymize_dump \
    --in  /backup/armenu_prod_2026-09-08.dump \
    --out /backup/armenu_stage_2026-09-08.dump

# Nima o'zgaradi:
#   telefon      → +998 90 000 00 XX  (ketma-ket)
#   email        → user<id>@example.invalid
#   mijoz ismi   → "Mijoz <id>"
#   Telegram id  → NULL
#   to'lov karta → allaqachon saqlanmaydi (PCI DSS)
#   fiskal chek  → fiscal_sign randomlashtiriladi
#   API kalitlar → o'chiriladi
#   AI kalitlar  → o'chiriladi (env'dan keladi)
```

1.3 Muhit o'zgaruvchilari (farqlanuvchi qism)

```
# .env.prod (600, armenu:armenu)
DJANGO_ENV=production
DEBUG=0
SECRET_KEY=<64 belgi>
ALLOWED_HOSTS=armenu.uz,www.armenu.uz,api.armenu.uz

DB_NAME=armenu_db
DB_USER=armenu
DB_HOST=127.0.0.1
DB_PORT=5432
DB_CONN_MAX_AGE=60

REDIS_URL=redis://127.0.0.1:6379/5          # ⚡ db=5 kesh
REDIS_CHANNELS_URL=redis://127.0.0.1:6379/6  # ⚡ db=6 channels

WEB_PORT=8060
WS_PORT=8061

MEDIA_ROOT=/var/armenu/media
STATIC_ROOT=/home/armenu/app/staticfiles

AI_DAILY_LIMIT_PER_RESTAURANT=2.0
AI_MONTHLY_LIMIT_TOTAL=500.0
AI_CACHE_TTL=3600

SENTRY_DSN=https://...@sentry.io/...
PROMETHEUS_ENABLED=1
```

2. INFRATUZILMA TARKIBI

2.1 S0 — Start (1-20 restoran)

```
┌──────────────────────────────────────┐
│  1 × VPS — 4 vCPU / 8 GB RAM / 100 GB NVMe
│  Ubuntu 24.04 LTS · Toshkent yoki Frankfurt DC
│                                      │
│  nginx (443/80)                      │
│  ├── armenu.socket → armenu.service (gunicorn×4)
│  ├── armenu-ws.service (daphne ×1) :8061
│  ├── armenu-worker@ ×5               │
│  └── armenu-healer.service           │
│                                      │
│  PostgreSQL 16 + pgvector   (shared_buffers 2GB)
│  Redis 7                    (maxmemory 1GB)
│  MinIO                      (3D fayllar, 40 GB)
└──────────────────────────────────────┘
```

Element	Spec	Rol	Narx/oy
VPS	4 vCPU / 8 GB / 100 GB NVMe	Hammasi	$28
Backup storage	200 GB S3	Kunlik dump + WAL	$5
Cloudflare	Free tarif	CDN + WAF	$0
Domen + TLS	armenu.uz + Let's Encrypt	—	$2
Monitoring	Uptime Robot (free)	Tashqi uptime	$0
JAMI			**~$35-40**

2.2 S3 — Respublika (400-1500 restoran)

#	Server	Spec	Rol	Soni	Narx/oy
1	`lb-01/02`	2 vCPU / 4 GB	nginx LB + TLS	2	$28
2	`web-01..04`	4 vCPU / 8 GB	gunicorn (8 worker)	4	$112
3	`ws-01/02`	4 vCPU / 8 GB	daphne (WebSocket)	2	$56
4	`worker-01..03`	4 vCPU / 8 GB	5 navbat × 3 node	3	$84
5	`pg-primary`	8 vCPU / 32 GB / 500 GB NVMe	PostgreSQL primary	1	$160
6	`pg-replica-01/02`	4 vCPU / 16 GB / 500 GB	Read replica	2	$160
7	`redis-01..03`	2 vCPU / 8 GB	Redis + Sentinel	3	$84
8	`obs-01`	4 vCPU / 8 GB / 200 GB	Prometheus+Grafana+Loki	1	$28
9	S3 (object storage)	2 TB + trafik	3D fayl, backup	—	$50
10	Cloudflare Pro	—	CDN + WAF + analitika	—	$20
				JAMI	**~$700**



2.3 S5 — 1 mln bir vaqtda

Element	Soni	Spec	Narx/oy
LB	4	4 vCPU / 8 GB	$112
WEB	40	4 vCPU / 8 GB	$1 120
WS	3	8 vCPU / 16 GB	$168
WORKER	6	8 vCPU / 16 GB	$336
PG primary	1	16 vCPU / 64 GB / 2 TB	$420
PG replica	3	8 vCPU / 32 GB / 2 TB	$480
Redis cluster	6	4 vCPU / 16 GB	$336
ClickHouse (analitika)	2	8 vCPU / 32 GB / 4 TB	$340
Obs stack	2	8 vCPU / 16 GB / 1 TB	$112
S3 + CDN + trafik	—	20 TB	$700
Zaxira / rezerv	—	—	$276
JAMI			**~$4 400**

> Daromadning **1,9%** (S5 da ~$230k/oy). ✅ Sog'lom.

2.4 Bazaviy o'rnatish (S0)

```
# 1. Foydalanuvchi
sudo adduser --system --group --home /home/armenu --shell /bin/bash armenu

# 2. Paketlar
sudo apt update && sudo apt install -y \
    python3.12 python3.12-venv python3.12-dev build-essential \
    postgresql-16 postgresql-16-pgvector postgresql-contrib \
    redis-server nginx certbot python3-certbot-nginx \
    git curl jq htop ncdu

# 3. DB
sudo -u postgres psql <<'SQL'
CREATE USER armenu WITH PASSWORD 'CHANGE_ME';
CREATE DATABASE armenu_db OWNER armenu ENCODING 'UTF8'
    LC_COLLATE 'en_US.UTF-8' LC_CTYPE 'en_US.UTF-8' TEMPLATE template0;
\c armenu_db
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS btree_gin;
SQL

# 4. Media papkasi — ✗ egasi armenu (root:root QOLDIRILMAYDI)
sudo mkdir -p /var/armenu/media/{ar,dish,qr,receipt} /var/log/armenu
sudo chown -R armenu:armenu /var/armenu /var/log/armenu
sudo chmod 750 /var/armenu/media/receipt        # chek — maxfiy

# 5. Kod
sudo -u armenu git clone git@github.com:<org>/armenu.git /home/armenu/app
sudo -u armenu python3.12 -m venv /home/armenu/app/venv
sudo -u armenu /home/armenu/app/venv/bin/pip install -r /home/armenu/app/requirements.txt

# 6. .env
sudo -u armenu cp /home/armenu/app/.env.example /home/armenu/app/.env
sudo chmod 600 /home/armenu/app/.env
```

3. ⛔ IZOLYATSIYA KAFOLATI

> 🔴 **MUTLAQ QOIDA:** ARMENU `mebelcity_platform` bilan **bir serverda ishlashi mumkin**, lekin unga **HECH QANDAY tarzda tegmaydi**. Bitta buyruq ham `bittada-*` servisini, `tenant_*` bazasini yoki 8040/8041 portini o'zgartirmaydi.

3.1 To'liq ajratish jadvali

Resurs	⬛ mebelcity_platform	⬛ ARMENU	To'qnashuv
Linux user	`user`	`armenu`	✗ yo'q
Home	`/home/user/mebelcity_platform`	`/home/armenu/app`	✗ yo'q
Web port	`127.0.0.1:8040`	`127.0.0.1:8060`	✗ yo'q
WS port	`127.0.0.1:8041`	`127.0.0.1:8061`	✗ yo'q
PostgreSQL DB	`tenant_mebelcity`, `bittada_*`	`armenu_db`	✗ yo'q
PostgreSQL user	`bittada`	`armenu`	✗ yo'q
Redis db	0-4	**5 (kesh), 6 (channels)**	✗ yo'q
systemd unit	`bittada-*`	`armenu*`	✗ yo'q
nginx site	`bittada.conf`, `mebelcity.conf`	`armenu.conf`	✗ yo'q
Domen	`mebelcity.bittada.uz`, `admin.bittada.uz`	`armenu.uz`	✗ yo'q
Media	`/home/user/mebelcity_platform/media`	`/var/armenu/media`	✗ yo'q
Log	`/var/log/bittada/`	`/var/log/armenu/`	✗ yo'q
venv	`platform_venv`	`/home/armenu/app/venv`	✗ yo'q
Cron	`user` crontab	`armenu` crontab	✗ yo'q
Git repo	`mebelcity_platform`	`armenu` (alohida)	✗ yo'q

3.2 sudoers cheklovi — ⚡ eng muhim kafolat

```
# /etc/sudoers.d/armenu   (chmod 440, visudo -c bilan tekshiring)
# ● armenu foydalanuvchisi FAQAT armenu* servislarini boshqara oladi.
#     bittada-* ga urinish "command not allowed" beradi.

Cmnd_Alias ARMENU_SVC = \
    /usr/bin/systemctl reload  armenu, \
    /usr/bin/systemctl restart armenu, \
    /usr/bin/systemctl start   armenu, \
    /usr/bin/systemctl stop    armenu, \
    /usr/bin/systemctl status  armenu, \
    /usr/bin/systemctl reload  armenu.service, \
    /usr/bin/systemctl restart armenu.service, \
    /usr/bin/systemctl restart armenu-ws, \
    /usr/bin/systemctl restart armenu-ws.service, \
    /usr/bin/systemctl restart armenu-healer, \
    /usr/bin/systemctl restart armenu-workers.target, \
    /usr/bin/systemctl restart "armenu-worker@*", \
    /usr/bin/systemctl status  "armenu*", \
    /usr/bin/systemctl is-active "armenu*"

Cmnd_Alias ARMENU_NGINX = \
    /usr/sbin/nginx -t, \
    /usr/bin/systemctl reload nginx

armenu ALL=(root) NOPASSWD: ARMENU_SVC, ARMENU_NGINX

# ● QAT'IY TAQIQ (aniq bloklash)
armenu ALL=(ALL) !/usr/bin/systemctl *bittada*
armenu ALL=(ALL) !/usr/bin/systemctl *mebelcity*
armenu ALL=(ALL) !/bin/su, !/usr/bin/sudo -u user *
armenu ALL=(ALL) !/usr/bin/psql -d tenant_*
armenu ALL=(ALL) !/usr/bin/psql -d bittada*
```

3.3 PostgreSQL darajasida bloklash

```
-- armenu FAQAT armenu_db ga kira oladi
REVOKE ALL ON DATABASE tenant_mebelcity FROM armenu;
REVOKE ALL ON DATABASE postgres FROM armenu;
REVOKE CONNECT ON DATABASE tenant_mebelcity FROM PUBLIC;

-- teskarisi ham
REVOKE ALL ON DATABASE armenu_db FROM bittada;
REVOKE CONNECT ON DATABASE armenu_db FROM PUBLIC;
GRANT  CONNECT ON DATABASE armenu_db TO armenu;
```

```
# /etc/postgresql/16/main/pg_hba.conf — aniq qatorlar
# TYPE  DATABASE        USER      ADDRESS         METHOD
local   armenu_db       armenu                    scram-sha-256
host    armenu_db       armenu    127.0.0.1/32    scram-sha-256
# ● armenu boshqa hech qaysi bazaga qator OLMAYDI
```

3.4 Redis darajasida bloklash (ACL)

```
# /etc/redis/redis.conf
# ARMENU faqat db 5 va 6 ga, faqat o'z prefiksiga
user armenu on >CHANGE_ME ~armenu:* ~asgi:* &armenu:* \
    +@read +@write +@keyspace +@pubsub -@dangerous -flushall -flushdb -config

# ● FLUSHALL taqiqlangan — mebelcity keshini o'chira olmaydi
```

```
# config/settings.py — ⚡ ARMENU tomonidan qo'shimcha himoya
CACHES = {'default': {
    'BACKEND': 'django_redis.cache.RedisCache',
    'LOCATION': os.environ['REDIS_URL'],          # .../5
    'KEY_PREFIX': 'armenu',                       # ⚡ har kalit armenu: bilan
    'OPTIONS': {'CLIENT_CLASS': 'django_redis.client.DefaultClient'},
}}
CHANNEL_LAYERS = {'default': {
    'BACKEND': 'channels_redis.core.RedisChannelLayer',
    'CONFIG': {'hosts': [os.environ['REDIS_CHANNELS_URL']],   # .../6
               'prefix': 'armenu:asgi'},
}}
```

3.5 Tekshiruv buyruqlari — ⚡ deploy'dan OLDIN va KEYIN

```
#!/bin/bash
# /home/armenu/app/scripts/izolyatsiya_tekshir.sh
# Har deploy'dan oldin va keyin ishga tushiriladi. Xato → deploy TO'XTAYDI.
set -uo pipefail
FAIL=0
ok()   { echo "✓ $1"; }
bad()  { echo "● $1"; FAIL=1; }

echo "=== ARMENU izolyatsiya tekshiruvi ==="

# 1. PORT — 8040/8041 ga tegilmaganmi
ss -ltnp 2>/dev/null | grep -q '127.0.0.1:8040' \
  && ok "8040 (mebelcity) ochiq va tegilmagan" \
  || bad "8040 YO'Q — mebelcity yiqilgan bo'lishi mumkin!"
ss -ltnp 2>/dev/null | grep -q '127.0.0.1:8060' \
  && ok "8060 (armenu) ochiq" || bad "8060 yo'q"

# 2. SERVIS — bittada-* hammasi active
for s in bittada-manager bittada-manager-ws; do
  systemctl is-active --quiet "$s" && ok "$s active" || bad "$s ACTIVE EMAS!"
done

# 3. armenu FAQAT armenu* servislarini ko'radi
systemctl list-units --no-legend 'armenu*' | awk '{print $1}' | \
  grep -v '^armenu' && bad "armenu* dan tashqari unit topildi" || ok "unit nomlari toza"

# 4. DB — armenu foydalanuvchisi tenant_* ga kira olmasligi SHART
PGPASSWORD="$DB_PASSWORD" psql -h 127.0.0.1 -U armenu -d tenant_mebelcity \
  -c 'SELECT 1' >/dev/null 2>&1 \
  && bad "● armenu tenant_mebelcity ga KIRA OLDI!" \
  || ok "armenu tenant_mebelcity ga kira olmaydi"

# 5. REDIS — faqat db 5/6 va armenu: prefiksi
redis-cli -n 5 --scan --pattern '*' -count 500 | head -50 | \
  grep -v '^armenu:' | head -1 | grep -q . \
  && bad "db=5 da armenu: prefiksisiz kalit bor" || ok "Redis db=5 prefiks toza"
redis-cli -n 0 DBSIZE >/dev/null 2>&1 && ok "db=0 (mebelcity) tirik"

# 6. FAYL TIZIMI — armenu mebelcity papkasiga yoza olmasligi SHART
sudo -u armenu touch /home/user/mebelcity_platform/.armenu_test 2>/dev/null \
  && { bad "● armenu mebelcity papkasiga YOZA OLDI!"; \
       rm -f /home/user/mebelcity_platform/.armenu_test; } \
  || ok "armenu mebelcity papkasiga yoza olmaydi"

# 7. SUDO — bittada servisiga urinish rad etilishi SHART
sudo -u armenu sudo -n systemctl restart bittada-manager 2>&1 | \
  grep -qi 'not allowed\|password' \
  && ok "sudo bittada-manager RAD ETILDI" \
  || bad "● armenu bittada-manager ni restart qila oldi!"

# 8. NGINX — konfiglar aralashmagan
grep -rl '8060\|8061' /etc/nginx/sites-enabled/ | grep -v armenu \
  && bad "armenu porti boshqa nginx konfigida" || ok "nginx konfiglar ajratilgan"
grep -rl '8040\|8041' /etc/nginx/sites-enabled/armenu.conf 2>/dev/null \
  && bad "armenu.conf da mebelcity porti bor!" || ok "armenu.conf toza"

echo "======================================="
[ $FAIL -eq 0 ] && { echo "✓ IZOLYATSIYA BUTUN"; exit 0; } \
                || { echo "● IZOLYATSIYA BUZILGAN — DEPLOY TO'XTATILDI"; exit 1; }
```

```
# Tez qo'lda tekshiruv (bir qator)
systemctl list-units 'armenu*' 'bittada*' --no-pager | \
  awk '{printf "%-32s %s\n", $1, $4}'
```

3.6 Nima QILINMAYDI (qora ro'yxat)

```
● /etc/systemd/system/bittada-*.service       — TAHRIRLANMAYDI, O'QILMAYDI ham shart emas
● /etc/nginx/sites-*/bittada*                 — TEGILMAYDI
● /etc/nginx/nginx.conf                       — faqat `include` qatori qo'shiladi, 1 marta
● psql -d tenant_mebelcity                    — HECH QACHON
● redis-cli -n 0..4 (yoki FLUSHALL/FLUSHDB)   — HECH QACHON
● systemctl restart bittada-*                 — HECH QACHON
● /home/user/mebelcity_platform/**            — o'qilmaydi, yozilmaydi
● platform_venv/bin/*                         — ishlatilmaydi
● Umumiy `sudo systemctl daemon-reload` restart kaskadi — faqat armenu unitlari
```

---

4. SYSTEMD ARXITEKTURASI

4.1 Bog'liqlik grafi

```
                        multi-user.target
                                │
      ┌─────────────────┬───────┼──────────┬──────────────────┐
      ▼                 ▼       ▼          ▼                  ▼
  sockets.target    armenu-ws.service  armenu-healer   armenu-workers.target
      │                 │              .service             │
      ▼                 │                                   ├── armenu-worker@model3d
  armenu.socket         │                                   ├── armenu-worker@fiscal
  (:8060 ushlab turadi) │                                   ├── armenu-worker@notify
      │                 │                                   ├── armenu-worker@ai
      ▼                 │                                   └── armenu-worker@analytics
  armenu.service        │
  (gunicorn, Type=notify) │
      │                 │
      └────────┬────────┴──────────────────────┐
               ▼                               ▼
        postgresql.service            redis-server.service
               │                               │
               └──────────────┬────────────────┘
                              ▼
                     network-online.target

⚡ SOCKET ACTIVATION: armenu.socket portni USHLAB TURADI.
   armenu.service restart bo'lganda ham port ochiq qoladi →
   kiruvchi so'rovlar navbatga tushadi → ZERO-DOWNTIME.
```

4.2 `armenu.socket`

```
# /etc/systemd/system/armenu.socket
[Unit]
Description=ARMENU web socket (:8060)
PartOf=armenu.service

[Socket]
ListenStream=127.0.0.1:8060
NoDelay=true
ReusePort=true
Backlog=2048
# ⚡ Servis restart bo'lganda ham socket ochiq qoladi
Accept=no
SocketUser=armenu
SocketGroup=armenu
SocketMode=0660

[Install]
WantedBy=sockets.target
```

4.3 `armenu.service` (gunicorn)

```
# /etc/systemd/system/armenu.service
[Unit]
Description=ARMENU web (gunicorn, :8060)
Requires=armenu.socket
After=network-online.target postgresql.service redis-server.service armenu.socket
Wants=network-online.target

[Service]
Type=notify
User=armenu
Group=armenu
WorkingDirectory=/home/armenu/app
EnvironmentFile=/home/armenu/app/.env
Environment=PYTHONUNBUFFERED=1
Environment=DJANGO_SETTINGS_MODULE=config.settings

ExecStartPre=/home/armenu/app/venv/bin/python manage.py check --deploy --fail-level ERROR
ExecStart=/home/armenu/app/venv/bin/gunicorn config.wsgi:application \
    --bind fd://3 \
    --workers 8 --threads 4 --worker-class gthread \
    --timeout 60 --graceful-timeout 30 --keep-alive 5 \
    --max-requests 2000 --max-requests-jitter 200 \
    --worker-tmp-dir /dev/shm \
    --access-logfile - --error-logfile - --log-level info \
    --access-logformat '%({X-Request-Id}i)s %(h)s "%(r)s" %(s)s %(b)s %(M)sms'

ExecReload=/bin/kill -s HUP $MAINPID
KillSignal=SIGTERM
TimeoutStopSec=45
Restart=always
RestartSec=5

# Resurs chegaralari
LimitNOFILE=65535
MemoryMax=4G
MemoryHigh=3G
CPUWeight=200
TasksMax=4096

# ● IZOLYATSIYA — mebelcity papkasiga umuman kira olmaydi
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=read-only
ReadWritePaths=/var/armenu /var/log/armenu /home/armenu/app/staticfiles
InaccessiblePaths=/home/user
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictSUIDSGID=true
RestrictRealtime=true
LockPersonality=true

[Install]
WantedBy=multi-user.target
```

4.4 `armenu-ws.service` (daphne)

```
# /etc/systemd/system/armenu-ws.service
[Unit]
Description=ARMENU WebSocket (daphne, :8061)
After=network-online.target postgresql.service redis-server.service
Wants=network-online.target

[Service]
Type=simple
User=armenu
Group=armenu
WorkingDirectory=/home/armenu/app
EnvironmentFile=/home/armenu/app/.env
Environment=DJANGO_SETTINGS_MODULE=config.settings

ExecStart=/home/armenu/app/venv/bin/daphne \
    -b 127.0.0.1 -p 8061 \
    --proxy-headers \
    --application-close-timeout 20 \
    --websocket_timeout 3600 \
    --access-log - \
    config.asgi:application

# ⚡ WS stateful — SIGTERM'da klientlar reconnect qiladi (exponential backoff)
KillSignal=SIGTERM
TimeoutStopSec=30
Restart=always
RestartSec=5

LimitNOFILE=131072          # ⚡ WS ulanishlar ko'p
MemoryMax=4G
TasksMax=8192

NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=read-only
ReadWritePaths=/var/armenu /var/log/armenu
InaccessiblePaths=/home/user

[Install]
WantedBy=multi-user.target
```

4.5 `armenu-worker@.service` (template — 5 instans)

```
# /etc/systemd/system/armenu-worker@.service
[Unit]
Description=ARMENU worker — %i navbati
After=network-online.target postgresql.service redis-server.service
PartOf=armenu-workers.target

[Service]
Type=simple
User=armenu
Group=armenu
WorkingDirectory=/home/armenu/app
EnvironmentFile=/home/armenu/app/.env
Environment=DJANGO_SETTINGS_MODULE=config.settings
Environment=WORKER_QUEUE=%i

ExecStart=/home/armenu/app/venv/bin/python manage.py run_worker \
    --queue=%i --concurrency=2 --heartbeat=30

# ⚡ Graceful: joriy vazifa tugaguncha kutadi
KillSignal=SIGTERM
TimeoutStopSec=120
Restart=always
RestartSec=10

# Navbatga qarab resurs (drop-in bilan override qilinadi)
MemoryMax=2G
MemoryHigh=1500M
CPUWeight=100

NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=read-only
ReadWritePaths=/var/armenu /var/log/armenu
InaccessiblePaths=/home/user

[Install]
WantedBy=armenu-workers.target
```

```
# /etc/systemd/system/armenu-worker@model3d.service.d/override.conf
# ⚡ 3D generatsiya og'ir va uzoq (10 daqiqagacha)
[Service]
MemoryMax=4G
TimeoutStopSec=900
Environment=WORKER_TASK_TIMEOUT=900
```

```
# /etc/systemd/system/armenu-worker@fiscal.service.d/override.conf
# ⚡ Fiskal — qonuniy, hech qachon to'xtamasin
[Service]
Restart=always
RestartSec=3
MemoryMax=1G
```

4.6 `armenu-workers.target`

```
# /etc/systemd/system/armenu-workers.target
[Unit]
Description=ARMENU barcha worker navbatlari
Requires=armenu-worker@model3d.service \
         armenu-worker@fiscal.service \
         armenu-worker@notify.service \
         armenu-worker@ai.service \
         armenu-worker@analytics.service
After=armenu.service

[Install]
WantedBy=multi-user.target
```

Navbat	Vazifa	Konkurentlik	RAM	Timeout
`model3d`	AI 3D generatsiya, GLB→USDZ, Draco siqish	2	4 GB	900 s
`fiscal`	OFD chek yuborish, 48 soatlik retry	4	1 GB	60 s
`notify`	Telegram, WS push, email, printer	8	1 GB	30 s
`ai`	Tarjima, sentiment, embedding	4	2 GB	120 s
`analytics`	Event agregatsiya, hisobot, reconciliation	2	2 GB	600 s

4.7 `armenu-healer.service`

```
# /etc/systemd/system/armenu-healer.service
[Unit]
Description=ARMENU self-healing daemon (15 mexanizm)
After=armenu.service postgresql.service redis-server.service

[Service]
Type=simple
User=armenu
Group=armenu
WorkingDirectory=/home/armenu/app
EnvironmentFile=/home/armenu/app/.env
ExecStart=/home/armenu/app/venv/bin/python manage.py run_healer --interval=60
Restart=always
RestartSec=15
MemoryMax=512M
Nice=10                         # past prioritet — asosiy ishga xalaqit bermasin

NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=read-only
ReadWritePaths=/var/armenu /var/log/armenu
InaccessiblePaths=/home/user

[Install]
WantedBy=multi-user.target
```

4.8 Ishga tushirish va boshqarish

```
# Birinchi o'rnatish
sudo systemctl daemon-reload
sudo systemctl enable --now armenu.socket
sudo systemctl enable --now armenu.service
sudo systemctl enable --now armenu-ws.service
sudo systemctl enable --now armenu-worker@model3d armenu-worker@fiscal \
                            armenu-worker@notify  armenu-worker@ai \
                            armenu-worker@analytics
sudo systemctl enable --now armenu-workers.target
sudo systemctl enable --now armenu-healer.service

# Holat
systemctl status 'armenu*' --no-pager

# Kundalik operatsiyalar
sudo systemctl reload  armenu                # ⚡ Python/template — UZILISHSIZ
sudo systemctl restart armenu-ws             # WS consumers o'zgarsa
sudo systemctl restart armenu-workers.target  # worker kodi o'zgarsa
sudo systemctl restart 'armenu-worker@fiscal' # bitta navbat

# Log
journalctl -u armenu -f --output=cat
journalctl -u 'armenu-worker@*' --since '10 min ago'
```

5. NGINX

5.1 To'liq konfiguratsiya

```
# /etc/nginx/sites-available/armenu.conf
# ⚙ Bu fayl FAQAT ARMENU uchun. bittada*.conf ga TEGILMAYDI.

# ---------- Upstream ----------
upstream armenu_web {
    server 127.0.0.1:8060 max_fails=3 fail_timeout=15s;
    keepalive 64;
    keepalive_requests 1000;
    keepalive_timeout 60s;
}

upstream armenu_ws {
    server 127.0.0.1:8061 max_fails=3 fail_timeout=15s;
    # ⚡ WS — ip_hash kerak emas (channels layer Redis'da umumiy)
    keepalive 32;
}

# ---------- Rate limit zonalari ----------
limit_req_zone $binary_remote_addr zone=general:20m  rate=120r/m;
limit_req_zone $binary_remote_addr zone=menu:20m     rate=120r/m;
limit_req_zone $binary_remote_addr zone=qr:10m       rate=60r/m;
```

```
limit_req_zone $binary_remote_addr zone=orders:10m   rate=10r/m;
limit_req_zone $binary_remote_addr zone=aichat:10m   rate=2r/m;
limit_req_zone $binary_remote_addr zone=login:10m    rate=5r/m;
limit_req_zone $binary_remote_addr zone=webhook:10m  rate=600r/m;
limit_req_zone $binary_remote_addr zone=api:10m      rate=120r/m;
limit_conn_zone $binary_remote_addr zone=perip:10m;
limit_req_status 429;
limit_conn_status 429;

# ---------- 3D fayl kesh (S3 origin uchun) ----------
proxy_cache_path /var/cache/nginx/ar levels=1:2 keys_zone=ar_cache:100m
                 max_size=20g inactive=30d use_temp_path=off;

# ---------- Maintenance flag ----------
map $http_x_armenu_bypass $maintenance_bypass {
    default 0;
    "~^${MAINT_TOKEN}$" 1;
}

# ---------- WebSocket upgrade map ----------
map $http_upgrade $connection_upgrade {
    default upgrade;
    ''      close;
}

# ---------- HTTP → HTTPS ----------
server {
    listen 80;
    listen [::]:80;
    server_name armenu.uz www.armenu.uz api.armenu.uz cdn.armenu.uz stage.armenu.uz;

    location /.well-known/acme-challenge/ { root /var/www/certbot; }
    location / { return 301 https://$host$request_uri; }
}

# ---------- ASOSIY SERVER ----------
server {
    listen 443 ssl;
    listen [::]:443 ssl;
    http2 on;
    server_name armenu.uz www.armenu.uz;

    # --- TLS ---
    ssl_certificate     /etc/letsencrypt/live/armenu.uz/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/armenu.uz/privkey.pem;
    ssl_trusted_certificate /etc/letsencrypt/live/armenu.uz/chain.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305;
    ssl_prefer_server_ciphers off;
    ssl_session_cache shared:SSL_ARMENU:20m;
    ssl_session_timeout 1d;
    ssl_session_tickets off;
    ssl_stapling on;
    ssl_stapling_verify on;
    resolver 1.1.1.1 8.8.8.8 valid=300s;

    # --- Xavfsizlik header'lari ---
    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;
    add_header X-Content-Type-Options    "nosniff" always;
    add_header X-Frame-Options           "SAMEORIGIN" always;
    add_header Referrer-Policy           "strict-origin-when-cross-origin" always;
    add_header Permissions-Policy        "camera=(self), xr-spatial-tracking=(self), geolocation=(), microphone=()" always;

    # --- Cloudflare real IP ---
    include /etc/nginx/cloudflare-ips.conf;
    real_ip_header CF-Connecting-IP;

    # --- Umumiy ---
    client_max_body_size 50m;             # 3D fayl yuklash
    client_body_timeout 60s;
    client_header_timeout 20s;
    send_timeout 60s;
    server_tokens off;
    limit_conn perip 60;

    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_comp_level 5;
    gzip_types text/plain text/css application/javascript application/json
               application/xml image/svg+xml model/gltf+json;
    # ⚠ .glb / .usdz allaqachon siqilgan — gzip QILINMAYDI

    access_log /var/log/nginx/armenu_access.log combined buffer=32k flush=5s;
    error_log  /var/log/nginx/armenu_error.log warn;

    # ========== MAINTENANCE REJIMI ==========
    # Yoqish:  sudo touch /var/armenu/MAINTENANCE
    # O'chirish: sudo rm  /var/armenu/MAINTENANCE
    set $maint 0;
    if (-f /var/armenu/MAINTENANCE) { set $maint 1; }
    if ($maintenance_bypass = 1)    { set $maint 0; }   # jamoa uchun bypass
    if ($request_uri ~ ^/(health|metrics)) { set $maint 0; }

    location @maintenance {
        root /var/armenu/static_maint;
        try_files /maintenance.html =503;
        add_header Retry-After 300 always;
        add_header Cache-Control "no-store" always;
        internal;
    }

    # ========== SOG'LIQ ==========
    location = /health/ {
        access_log off;
        proxy_pass http://armenu_web;
        proxy_read_timeout 5s;
    }
    location = /health/deep/ {
        access_log off;
        allow 127.0.0.1;
        deny all;
        proxy_pass http://armenu_web;
    }
    location = /metrics {
        access_log off;
        allow 127.0.0.1;
        allow 10.0.0.0/8;                 # obs-01
        deny all;
        proxy_pass http://armenu_web;
    }

    # ========== STATIK ==========
    location /static/ {
        alias /home/armenu/app/staticfiles/;
        expires 1y;
        add_header Cache-Control "public, immutable" always;
        access_log off;
        try_files $uri =404;
    }

    # ========== ⚡ 3D FAYLLAR — AR UCHUN KRITIK ==========
    location /media/ar/ {
        if ($maint = 1) { return 200; }    # 3D AR maintenance'da ham ishlaydi

        alias /var/armenu/media/ar/;

        # MIME (nginx mime.types.armenu dan keladi, bu yerda majburlaymiz)
        location ~* \.glb$  { alias /var/armenu/media/ar/; default_type model/gltf-binary;   }
        location ~* \.usdz$ { alias /var/armenu/media/ar/; default_type model/vnd.usdz+zip;  }
        location ~* \.gltf$ { alias /var/armenu/media/ar/; default_type model/gltf+json;     }
        location ~* \.ktx2$ { alias /var/armenu/media/ar/; default_type image/ktx2;          }

        # ⚡ CORS — bu bo'lmasa AR ISHLAMAYDI
        add_header Access-Control-Allow-Origin   "*"                 always;
        add_header Access-Control-Allow-Methods  "GET, HEAD, OPTIONS" always;
        add_header Access-Control-Allow-Headers  "Range, Content-Type" always;
        add_header Access-Control-Expose-Headers "Content-Length, Content-Range, Accept-Ranges" always;
        add_header Cross-Origin-Resource-Policy  "cross-origin"      always;
        add_header Accept-Ranges bytes always;
        add_header Cache-Control "public, max-age=2592000, immutable" always;
        expires 30d;

        if ($request_method = OPTIONS) {
            add_header Access-Control-Max-Age 86400;
            add_header Content-Length 0;
            return 204;
        }

        gzip off;                         # allaqachon siqilgan
        try_files $uri @ar_origin;
    }

    location @ar_origin {
        internal;
        proxy_pass https://armenu-media.s3.amazonaws.com;
        proxy_cache ar_cache;
        proxy_cache_valid 200 206 30d;
        proxy_cache_valid 404 1m;
        proxy_cache_use_stale error timeout updating http_500 http_502 http_503 http_504;
        proxy_cache_background_update on;
        proxy_cache_lock on;
        add_header X-Cache-Status $upstream_cache_status always;
        add_header Access-Control-Allow-Origin "*" always;
    }

    # ========== BOSHQA MEDIA ==========
    location /media/dish/ {
        alias /var/armenu/media/dish/;
        expires 7d;
        add_header Cache-Control "public" always;
    }
```

```
    location /media/receipt/ {
        # ● Fiskal cheklar — faqat ilova orqali (X-Accel-Redirect)
        internal;
        alias /var/armenu/media/receipt/;
    }

    # =========== WEBSOCKET ===========
    location /ws/ {
        if ($maint = 1) { return 503; }
        proxy_pass http://armenu_ws;
        proxy_http_version 1.1;
        proxy_set_header Upgrade    $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_set_header Host       $host;
        proxy_set_header X-Real-IP  $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout  3600s;
        proxy_send_timeout  3600s;
        proxy_connect_timeout 10s;
        proxy_buffering off;
    }

    # =========== WEBHOOK (rate limit yumshoq, maintenance'da ham ishlaydi) ===========
    location /webhooks/ {
        limit_req zone=webhook burst=100 nodelay;
        proxy_pass http://armenu_web;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 30s;
        # ⚡ webhook maintenance'da ham qabul qilinadi (pul yo'qolmasin)
    }

    # =========== API — endpointga qarab rate limit ===========
    location /v1/orders/ {
        if ($maint = 1) { error_page 503 = @maintenance; return 503; }
        limit_req zone=orders burst=5 nodelay;
        include /etc/nginx/snippets/armenu-proxy.conf;
    }
    location /v1/ai/chat/ {
        limit_req zone=aichat burst=3 nodelay;
        include /etc/nginx/snippets/armenu-proxy.conf;
        proxy_read_timeout 45s;
    }
    location /v1/menu/ {
        limit_req zone=menu burst=40 nodelay;
        include /etc/nginx/snippets/armenu-proxy.conf;
    }
    location /staff/login/ {
        limit_req zone=login burst=3 nodelay;
        include /etc/nginx/snippets/armenu-proxy.conf;
    }
    location /t/ {                          # QR kirish
        limit_req zone=qr burst=20 nodelay;
        include /etc/nginx/snippets/armenu-proxy.conf;
    }

    # =========== QOLGANI ===========
    location / {
        if ($maint = 1) { error_page 503 = @maintenance; return 503; }
        limit_req zone=general burst=40 nodelay;
        include /etc/nginx/snippets/armenu-proxy.conf;
    }
}

# ---------- API subdomen (3rd party) ----------
server {
    listen 443 ssl;
    http2 on;
    server_name api.armenu.uz;
    ssl_certificate     /etc/letsencrypt/live/armenu.uz/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/armenu.uz/privkey.pem;

    location / {
        limit_req zone=api burst=60 nodelay;
        include /etc/nginx/snippets/armenu-proxy.conf;
    }
}
```

```
# /etc/nginx/snippets/armenu-proxy.conf
proxy_pass http://armenu_web;
proxy_http_version 1.1;
proxy_set_header Connection "";
proxy_set_header Host              $host;
proxy_set_header X-Real-IP         $remote_addr;
proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Request-Id      $request_id;    # ⚡ trace uchun
proxy_connect_timeout 5s;
proxy_send_timeout    60s;
proxy_read_timeout    60s;
proxy_next_upstream error timeout http_502 http_503 http_504;
proxy_next_upstream_tries 2;
```

5.2 Rate limit zonalari — jamlanma

Zona	Tezlik	Burst	Kalit	Endpoint
`general`	120/daq	40	IP	Barcha qolgan
`menu`	120/daq	40	IP	`/v1/menu/`
`qr`	60/daq	20	IP	`/t/<token>`
`orders`	10/daq	5	IP	`/v1/orders/`
`aichat`	2/daq	3	IP	`/v1/ai/chat/`
`login`	5/daq	3	IP	`/staff/login/`
`webhook`	600/daq	100	IP	`/webhooks/`
`api`	120/daq	60	IP	`api.armenu.uz`
`perip` (conn)	60 ulanish	—	IP	Barchasi

5.3 Maintenance rejimi

```
# YOQISH
sudo -u armenu touch /var/armenu/MAINTENANCE
# → barcha foydalanuvchi 503 + chiroyli sahifa
# → webhook, /health, /metrics, /media/ar/ ISHLASHDA DAVOM ETADI

# Jamoa uchun bypass (test qilish)
curl -H "X-ARMenu-Bypass: $MAINT_TOKEN" https://armenu.uz/

# O'CHIRISH
sudo -u armenu rm /var/armenu/MAINTENANCE
```

```
<!-- /var/armenu/static_maint/maintenance.html -->
<!doctype html><html lang="uz"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>ARMENU — texnik ishlar</title>
<style>
:root{color-scheme:light dark}
body{margin:0;min-height:100vh;display:grid;place-items:center;
     font:16px/1.6 system-ui,-apple-system,sans-serif;
     background:#0f1115;color:#e8eaf0;text-align:center;padding:24px}
@media(prefers-color-scheme:light){body{background:#f7f8fa;color:#1a1d24}}
h1{font-size:22px;margin:16px 0 8px}
p{opacity:.75;max-width:34ch;margin:0 auto}
</style></head><body><div>
<div style="font-size:56px">🛠</div>
<h1>Texnik ishlar olib borilmoqda</h1>
<p>Bir necha daqiqadan so'ng qaytadan urinib ko'ring.<br>
Buyurtma berish uchun ofitsiantni chaqiring.</p>
</div></body></html>
```

5.4 TLS sertifikat

```
# Birinchi olish
sudo certbot certonly --webroot -w /var/www/certbot \
  -d armenu.uz -d www.armenu.uz -d api.armenu.uz -d cdn.armenu.uz \
  --email ops@armenu.uz --agree-tos --no-eff-email

# Avtomatik yangilash (certbot o'z timer'ini o'rnatadi)
sudo systemctl status certbot.timer

# ⚡ Yangilangach FAQAT nginx reload (armenu servislariga tegilmaydi)
sudo tee /etc/letsencrypt/renewal-hooks/deploy/reload-nginx.sh >/dev/null <<'EOF'
#!/bin/bash
/usr/sbin/nginx -t && /usr/bin/systemctl reload nginx
EOF
sudo chmod +x /etc/letsencrypt/renewal-hooks/deploy/reload-nginx.sh
```

6. CI/CD QUVURI

6.1 Umumiy oqim



6.2 To'liq YAML

```yaml
# .github/workflows/ci-cd.yml
name: ARMENU CI/CD

on:
  push:
    branches: [main]
    tags: ['v*']
  pull_request:
    branches: [main]

concurrency:
  group: armenu-${{ github.ref }}
  cancel-in-progress: true

env:
  PYTHON_VERSION: '3.12'
  DJANGO_SETTINGS_MODULE: config.settings_ci

jobs:
  # ═══════════ DARVOZA 1 — LINT ═══════════
  lint:
    name: 1️⃣ Lint va format
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12', cache: pip}
      - run: pip install ruff black mypy django-stubs
      - name: ruff
        run: ruff check . --output-format=github
      - name: black
        run: black --check --diff .
      - name: mypy (yumshoq — xato bloklamaydi)
        run: mypy apps/ --ignore-missing-imports || true

  # ═══════════ DARVOZA 2 — ● TENANT IZOLYATSIYA (MAJBURIY) ═══════════
  tenant-isolation:
    name: 2️⃣ ● Tenant izolyatsiya (MAJBURIY)
    runs-on: ubuntu-latest
    services:
      postgres:
        image: pgvector/pgvector:pg16
        env: {POSTGRES_PASSWORD: test, POSTGRES_DB: armenu_test}
        ports: ['5432:5432']
        options: >-
          --health-cmd pg_isready --health-interval 10s
          --health-timeout 5s --health-retries 5
      redis:
        image: redis:7
        ports: ['6379:6379']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12', cache: pip}
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: python manage.py migrate --noinput
      - name: ● Cross-tenant kirish testi
        run: pytest tests/test_tenant_isolation.py -v --tb=short -p no:randomly
      - name: ● Har model TenantScoped ekanini tekshirish
        run: python manage.py check_tenant_scoping --strict
      - name: ● Filtrsiz queryset qidiruvi (statik tahlil)
        run: python scripts/find_unscoped_queries.py --fail-on-find

  # ═══════════ DARVOZA 3 — i18n ═══════════
  i18n:
    name: 3️⃣ Tarjima to'liqligi (uz/ru/en)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12', cache: pip}
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: sudo apt-get install -y gettext
      - name: Tarjima fayllari yangilanganmi
        run: |
          python manage.py makemessages -l uz -l ru -l en --no-obsolete
          git diff --exit-code locale/ || {
            echo "❌ Tarjima fayllari eskirgan. makemessages ishga tushiring."; exit 1; }
      - name: 100% tarjima qilinganmi
        run: pytest tests/test_i18n_completeness.py -v
      - run: python manage.py compilemessages

  # ═══════════ DARVOZA 4 — TESTLAR + QAMROV ═══════════
  test:
    name: 4️⃣ Testlar va qamrov (≥75%)
    runs-on: ubuntu-latest
    services:
      postgres:
        image: pgvector/pgvector:pg16
        env: {POSTGRES_PASSWORD: test, POSTGRES_DB: armenu_test}
        ports: ['5432:5432']
        options: >-
          --health-cmd pg_isready --health-interval 10s
          --health-timeout 5s --health-retries 5
      redis:
        image: redis:7
        ports: ['6379:6379']
      minio:
        image: bitnami/minio:latest
        env: {MINIO_ROOT_USER: test, MINIO_ROOT_PASSWORD: testtest}
        ports: ['9000:9000']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12', cache: pip}
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: python manage.py migrate --noinput
      - name: Majburiy kritik testlar
        run: |
          pytest tests/test_order_state.py \
                 tests/test_price_calculation.py \
                 tests/integrations/test_payment_idempotency.py \
                 tests/integrations/test_payment_signature.py \
                 tests/integrations/test_adapter_contract.py \
                 -v --tb=short
      - name: To'liq test to'plami + qamrov
        run: pytest --cov=apps --cov-report=xml --cov-report=term-missing --cov-fail-under=75
      - uses: actions/upload-artifact@v4
        if: always()
        with: {name: coverage, path: coverage.xml}

  # ═══════════ DARVOZA 5 — XAVFSIZLIK ═══════════
  security:
    name: 5️⃣ Xavfsizlik skani
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: {fetch-depth: 0}
      - uses: actions/setup-python@v5
        with: {python-version: '3.12', cache: pip}
```

```
      - run: pip install bandit pip-audit safety
      - name: bandit (kod zaifliklari)
        run: bandit -r apps/ config/ -ll -ii -f screen
      - name: pip-audit (bog'liqlik CVE)
        run: pip-audit -r requirements.txt --strict
      - name: gitleaks (sir sizishi)
        uses: gitleaks/gitleaks-action@v2
        env: {GITHUB_TOKEN: '${{ secrets.GITHUB_TOKEN }}'}
      - name: ⚙ .env commit qilinmaganmi
        run: |
          git ls-files | grep -E '^\.env$|\.env\.(prod|production)$' && {
            echo "❌ .env repozitoriyaga qo'shilgan!"; exit 1; } || echo "✓ .env yo'q"
      - name: ⚙ Django deploy tekshiruvi
        run: python manage.py check --deploy --fail-level WARNING

  # ═══════ DARVOZA 6 — MIGRATSIYA ═══════
  migrations:
    name: 🗄 Migratsiya tekshiruvi
    runs-on: ubuntu-latest
    services:
      postgres:
        image: pgvector/pgvector:pg16
        env: {POSTGRES_PASSWORD: test, POSTGRES_DB: armenu_test}
        ports: ['5432:5432']
        options: >-
          --health-cmd pg_isready --health-interval 10s
          --health-timeout 5s --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12', cache: pip}
      - run: pip install -r requirements.txt -r requirements-dev.txt django-migration-linter
      - name: Yetishmayotgan migratsiya bormi
        run: python manage.py makemigrations --check --dry-run
      - name: ⚡ Backward-compatible ekanmi (bloklovchi o'zgarish yo'q)
        run: |
          python manage.py lintmigrations --exclude-migration-tests \
            IGNORE_MIGRATION_FILES --warnings-as-errors
      - name: Migratsiya + rollback sinovi
        run: |
          python manage.py migrate --noinput
          python manage.py migrate menu zero --noinput
          python manage.py migrate --noinput
      - name: ⚡ NOT NULL ustun DEFAULT'siz emasmi
        run: python scripts/check_notnull_defaults.py

  # ═══════ DEPLOY ═══════
  deploy:
    name: 🚀 Deploy (prod)
    needs: [lint, tenant-isolation, i18n, test, security, migrations]
    if: github.ref == 'refs/heads/main' || startsWith(github.ref, 'refs/tags/v')
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://armenu.uz
    concurrency: {group: armenu-deploy, cancel-in-progress: false}
    steps:
      - uses: actions/checkout@v4

      - name: SSH kaliti
        uses: webfactory/ssh-agent@v0.9.0
        with: {ssh-private-key: '${{ secrets.DEPLOY_SSH_KEY }}'}

      - name: known_hosts
        run: |
          mkdir -p ~/.ssh
          ssh-keyscan -H ${{ secrets.PROD_HOST }} >> ~/.ssh/known_hosts

      # ---------- ● IZOLYATSIYA (deploy'dan OLDIN) ----------
      - name: ● Izolyatsiya tekshiruvi (oldin)
        run: |
          ssh armenu@${{ secrets.PROD_HOST }} \
            'bash /home/armenu/app/scripts/izolyatsiya_tekshir.sh'

      # ---------- 💾 BACKUP ----------
      - name: 💾 Rollback nuqtasi
        run: |
          ssh armenu@${{ secrets.PROD_HOST }} bash -s <<'EOSSH'
          set -euo pipefail
          cd /home/armenu/app
          TS=$(date +%Y%m%d_%H%M%S)
          echo "$TS" > /home/armenu/.last_deploy_ts
          git rev-parse HEAD > /home/armenu/.rollback_commit
          pg_dump -Fc -Z6 -d "$DB_NAME" -f "/backup/predeploy_${TS}.dump"
          ls -lh "/backup/predeploy_${TS}.dump"
          # 7 kundan eski predeploy dump'lar o'chiriladi
          find /backup -name 'predeploy_*.dump' -mtime +7 -delete
          EOSSH

      # ---------- 🚀 DEPLOY ----------
      - name: 🚀 Deploy
        run: |
          ssh armenu@${{ secrets.PROD_HOST }} bash -s <<'EOSSH'
          set -euo pipefail
          cd /home/armenu/app
          source .env

          echo "▶ [1/8] Kod"
          git fetch --all --tags --prune
          git checkout --force "${GITHUB_SHA:-origin/main}"

          echo "▶ [2/8] Bog'liqliklar"
          venv/bin/pip install -q -r requirements.txt

          echo "▶ [3/8] Django tekshiruvi"
          venv/bin/python manage.py check --deploy --fail-level ERROR

          echo "▶ [4/8] Migratsiya (backward compatible)"
          venv/bin/python manage.py migrate --noinput

          echo "▶ [5/8] Statik + tarjima"
          venv/bin/python manage.py collectstatic --noinput --clear
          venv/bin/python manage.py compilemessages

          echo "▶ [6/8] Web — GRACEFUL RELOAD (uzilishsiz)"
          sudo systemctl reload armenu

          echo "▶ [7/8] WS va workerlar"
          sudo systemctl restart armenu-ws
          sudo systemctl restart armenu-workers.target

          echo "▶ [8/8] Kesh tozalash (menyu + tarjima)"
          venv/bin/python manage.py clear_cache --prefix=menu --prefix=i18n
          EOSSH

      - name: ☁ Cloudflare kesh
        run: |
          curl -sf -X POST \
            "https://api.cloudflare.com/client/v4/zones/${{ secrets.CF_ZONE }}/purge_cache" \
            -H "Authorization: Bearer ${{ secrets.CF_TOKEN }}" \
            -H "Content-Type: application/json" \
            --data '{"prefixes":["armenu.uz/static/","armenu.uz/v1/menu/"]}'

      # ---------- 🩺 SMOKE TEST ----------
      - name: 🩺 Smoke test (12 tekshiruv)
        id: smoke
        run: |
          set -e
          B=https://armenu.uz
          echo "1 health";       curl -fsS   "$B/health/" | grep -q '"ok":true'
          echo "2 health deep";  ssh armenu@${{ secrets.PROD_HOST }} \
                                   'curl -fsS localhost:8060/health/deep/' | grep -q '"db":"ok"'
          echo "3 bosh sahifa";  curl -fsS -o /dev/null -w '%{http_code}' "$B/" | grep -q 200
          echo "4 menyu API";    curl -fsS "$B/v1/menu/demo/" | jq -e '.dishes|length>0'
          echo "5 statik";       curl -fsS -o /dev/null -w '%{http_code}' "$B/static/css/app.css" | grep -q 200
          echo "6 3D MIME";      curl -fsSI "$B/media/ar/library/osh/model.glb" | grep -qi 'model/gltf-binary'
          echo "7 USDZ MIME";    curl -fsSI "$B/media/ar/library/osh/model.usdz" | grep -qi 'model/vnd.usdz+zip'
          echo "8 CORS";         curl -fsSI "$B/media/ar/library/osh/model.glb" | grep -qi 'access-control-allow-origin'
          echo "9 WS";           curl -fsS -o /dev/null -w '%{http_code}' \
                                   -H 'Connection: Upgrade' -H 'Upgrade: websocket' \
                                   -H 'Sec-WebSocket-Version: 13' \
                                   -H 'Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==' "$B/ws/guest/" | grep -qE '101|400'
          echo "10 metrics";     ssh armenu@${{ secrets.PROD_HOST }} \
                                   'curl -fsS localhost:8060/metrics' | grep -q 'armenu_'
          echo "11 servislar";   ssh armenu@${{ secrets.PROD_HOST }} \
                                   'systemctl is-active armenu armenu-ws armenu-workers.target' | grep -vq inactive
          echo "12 IZOLYATSIYA"; ssh armenu@${{ secrets.PROD_HOST }} \
                                   'bash /home/armenu/app/scripts/izolyatsiya_tekshir.sh'

      # ---------- ↩ ROLLBACK ----------
      - name: ↩ AVTOMATIK ROLLBACK
        if: failure() && steps.smoke.conclusion == 'failure'
        run: |
          ssh armenu@${{ secrets.PROD_HOST }} bash -s <<'EOSSH'
          set -euo pipefail
          cd /home/armenu/app
          COMMIT=$(cat /home/armenu/.rollback_commit)
          echo "↩ Rollback → $COMMIT"
          git checkout --force "$COMMIT"
          venv/bin/pip install -q -r requirements.txt
          venv/bin/python manage.py collectstatic --noinput
          venv/bin/python manage.py compilemessages
          sudo systemctl reload armenu
          sudo systemctl restart armenu-ws armenu-workers.target
          sleep 5
          curl -fsS https://armenu.uz/health/ || echo "⚠ Rollback'dan keyin ham xato!"
          EOSSH

      # ---------- 📣 XABAR ----------
      - name: 📣 Telegram (muvaffaqiyat)
        if: success()
        run: |
```

```
          curl -s -X POST "https://api.telegram.org/bot${{ secrets.TG_BOT }}/sendMessage" \
            -d chat_id=${{ secrets.TG_CHAT }} -d parse_mode=HTML \
            -d text="✅ <b>ARMENU deploy OK</b>%0A<code>${GITHUB_SHA:0:8}</code>%0A${{ github.event.head_commit.message }}"

      - name: 📲 Telegram (xato)
        if: failure()
        run: |
          curl -s -X POST "https://api.telegram.org/bot${{ secrets.TG_BOT }}/sendMessage" \
            -d chat_id=${{ secrets.TG_CHAT }} -d parse_mode=HTML \
            -d text="🔴 <b>ARMENU deploy YIQILDI — rollback bajarildi</b>%0A<code>${GITHUB_SHA:0:8}</code>%0Ahttps://github.com/${{ github.repository }}/actions/runs/${{ github.run_id }}"
```

6.3 Darvozalar jadvali

#	Darvoza	Bloklaydi	Vaqt	Nima tekshiriladi
1	Lint	✅	~40 s	`ruff` · `black` · `mypy` (yumshoq)
2	⛔ **Tenant izolyatsiya**	✅ **MAJBURIY**	~2 daq	Cross-tenant 403, har model scoped, filtrsiz queryset
3	i18n	✅	~1 daq	uz/ru/en 100%, `.po` yangilangan
4	Testlar	✅	~5 daq	Qamrov ≥75%, 6 kritik test
5	Security	✅	~1,5 daq	`bandit` · `pip-audit` · `gitleaks` · `check --deploy`
6	Migratsiya	✅	~2 daq	Yetishmayotgan yo'q, backward compatible, NOT NULL DEFAULT
	JAMI		**~7 daq** (parallel)	

---

7. DEPLOY STRATEGIYASI

7.1 Zero-downtime qadamlar

```
┌──────────────────────────────────────────────────────────┐
│ [1] armenu.socket PORTNI USHLAB TURADI                   │
│     → gunicorn qayta ishga tushsa ham :8060 ochiq qoladi │
│     → kiruvchi so'rovlar backlog'da kutadi (2048 ta)     │
├──────────────────────────────────────────────────────────┤
│ [2] MIGRATSIYA — faqat BACKWARD COMPATIBLE               │
│     eski kod ham, yangi kod ham ishlaydigan sxema        │
├──────────────────────────────────────────────────────────┤
│ [3] collectstatic --clear → yangi hash'li fayllar        │
│     eski sahifa eski faylni topadi (1 yil kesh, nom o'zgargan)  │
├──────────────────────────────────────────────────────────┤
│ [4] systemctl reload armenu  (SIGHUP)                    │
│     gunicorn: eski worker'lar joriy so'rovni TUGATADI (30 s),  │
│               yangi worker'lar yangi so'rovlarni oladi   │
│     ⚡ 0 ta 502                                          │
├──────────────────────────────────────────────────────────┤
│ [5] armenu-ws restart                                    │
│     WS klientlar uziladi → exponential backoff (1-30 s) →      │
│     avtomatik qayta ulanadi. Foydalanuvchi ~2 s sezmaydi │
├──────────────────────────────────────────────────────────┤
│ [6] worker restart (TimeoutStopSec=120)                  │
│     joriy vazifa tugaydi, yangi vazifa navbatda kutadi   │
├──────────────────────────────────────────────────────────┤
│ [7] Kesh tozalash — FAQAT kerakli prefiks (menu, i18n)   │
│     ⛔ FLUSHALL HECH QACHON                              │
└──────────────────────────────────────────────────────────┘
```

7.2 Migratsiya tartibi — backward compatible

> ⚡ **Oltin qoida:** har migratsiya **eski kod bilan ham ishlashi** kerak.
> Chunki reload paytida bir necha soniya eski va yangi worker birga yashaydi.

Amaliyot	✅ Xavfsiz	⛔ Xavfli	To'g'ri yo'l
Ustun qo'shish	`null=True` yoki `default=`	`NOT NULL` DEFAULT'siz	Avval nullable → to'ldirish → `NOT NULL`
Ustun o'chirish	—	Darhol `DROP`	1) koddan olib tashlash → deploy 2) keyingi relizda `DROP`
Nom o'zgartirish	—	`RENAME`	Yangi qo'shish → ikkalasiga yozish → ko'chirish → eskisini o'chirish
Indeks	`CONCURRENTLY`	Oddiy `CREATE INDEX` (jadval qulflanadi)	`AddIndexConcurrently` + `atomic=False`
Tur o'zgartirish	Kengaytirish (varchar 50→200)	Toraytirish	Yangi ustun + backfill
Katta backfill	Chunk (10k) worker'da	Bitta `UPDATE`	Data migration → worker vazifasi
`UNIQUE`	`CONCURRENTLY` indeks → constraint	Darhol constraint	2 bosqich

```
# ✅ TO'G'RI — bloklamaydigan indeks
from django.contrib.postgres.operations import AddIndexConcurrently
from django.db import migrations, models


class Migration(migrations.Migration):
    atomic = False                          # ⚡ CONCURRENTLY uchun MAJBURIY
    dependencies = [('orders', '0012_...')]
    operations = [
        AddIndexConcurrently(
            model_name='order',
            index=models.Index(fields=['restaurant', '-created_at'],
                               name='idx_order_rest_created'),
        ),
    ]
```

```
# ✅ TO'G'RI — NOT NULL ustun 3 bosqichda
# Migratsiya 0013: nullable qo'shish
migrations.AddField('dish', 'ikpu_code',
                    models.CharField(max_length=32, null=True, blank=True))

# Migratsiya 0014: backfill (data migration, chunk bilan)
def backfill(apps, schema_editor):
    Dish = apps.get_model('menu', 'Dish')
    qs = Dish.objects.filter(ikpu_code__isnull=True)
    while (batch := list(qs.values_list('pk', flat=True)[:5000])):
        Dish.objects.filter(pk__in=batch).update(ikpu_code='10112001001')

# Migratsiya 0015 (KEYINGI relizda): NOT NULL
migrations.AlterField('dish', 'ikpu_code',
                      models.CharField(max_length=32, default='10112001001'))
```

7.3 Statik fayllar

```
# config/settings.py
STORAGES = {
    'staticfiles': {
        # ⚡ Fayl nomiga hash qo'shadi: app.a3f2c81d.css
        #   → 1 yil kesh xavfsiz, eski sahifa eski faylni topadi
        'BACKEND': 'django.contrib.staticfiles.storage.ManifestStaticFilesStorage',
    },
}
```

```
# Deploy'da
venv/bin/python manage.py collectstatic --noinput --clear
# --clear: eski hash'siz qoldiqlar tozalanadi
# ⚠ Ba'zi eski hash'lar 24 soat saqlanishi kerak (ochiq sahifalar uchun) →
#    --clear o'rniga haftalik cron bilan 7 kundan eskisini tozalash tavsiya etiladi
```

7.4 Kesh tozalash

```
# ⛔ HECH QACHON: redis-cli FLUSHALL / FLUSHDB
# ✅ Faqat prefiks bo'yicha
venv/bin/python manage.py clear_cache --prefix=menu --prefix=i18n

# manage.py clear_cache ichida:
#   django_redis: cache.delete_pattern('armenu:menu:*')
```

Kesh	Deploy'da tozalanadimi	Sabab
`menu:*`	✅ Ha	Menyu strukturasi o'zgargan bo'lishi mumkin
`i18n:*`	✅ Ha	Tarjima yangilangan
`ai:menuctx:*`	✅ Ha	Menyu konteksti eskirgan
`mcp:*` (AI javoblar)	❌ Yo'q	Qimmat, o'zgarmaydi
`ikpu:*`	❌ Yo'q	Tashqi katalog, o'zgarmaydi
`sess:*`	⛔ **Hech qachon**	Foydalanuvchilar chiqib ketadi
`idem:*`	⛔ **Hech qachon**	Dublikat to'lov xavfi
Cloudflare `/static/`	✅ Ha	Yangi hash
Cloudflare `/media/ar/`	❌ Yo'q	Immutable, 30 kun

7.5 Rollback

Qachon rollback qilinadi:

Belgi	Rollback	Izoh
Smoke test yiqildi	✅ **Avtomatik**	CI o'zi qiladi
5xx > 5% (5 daqiqa)	✅ Darhol	Alert → on-call
To'lov ishlamayapti	✅ **Darhol**	Pul yo'qolishi
Fiskal chek yuborilmayapti	✅ Darhol	Qonuniy risk
Tenant sizishi shubhasi	✅ **Darhol** + incident	Kritik
p95 2× sekinlashdi	🔍 Tekshirish	Balki kesh sovuq — 10 daq kuting
Bitta sahifada UI bug	❌ Yo'q	Hotfix yetarli

```
#!/bin/bash
# /home/armenu/app/scripts/rollback.sh [commit|last]
set -euo pipefail
cd /home/armenu/app
source .env

TARGET="${1:-$(cat /home/armenu/.rollback_commit)}"
TS=$(cat /home/armenu/.last_deploy_ts)

echo "↩ ROLLBACK → $TARGET"
read -rp "Migratsiya ham qaytarilsinmi? (ha/yo'q): " REVERT_MIGR

# 1. Kod
git checkout --force "$TARGET"
venv/bin/pip install -q -r requirements.txt

# 2. Migratsiya (⚠ FAQAT zarur bo'lsa — ma'lumot yo'qolishi mumkin)
if [ "$REVERT_MIGR" = "ha" ]; then
  echo "⚠ DB predeploy dump'dan tiklanadi: /backup/predeploy_${TS}.dump"
  read -rp "TASDIQLANG (BAJAR deb yozing): " C
  [ "$C" = "BAJAR" ] || { echo "Bekor qilindi"; exit 1; }
  sudo systemctl stop armenu armenu-ws armenu-workers.target
  pg_restore -c -d "$DB_NAME" "/backup/predeploy_${TS}.dump"
fi

# 3. Statik
venv/bin/python manage.py collectstatic --noinput
venv/bin/python manage.py compilemessages

# 4. Servislar
sudo systemctl reload armenu
sudo systemctl restart armenu-ws armenu-workers.target

# 5. Tekshirish
sleep 5
curl -fsS https://armenu.uz/health/ && echo "✓ Rollback tugadi" \
  || { echo "🔴 Rollback'dan keyin ham xato!"; exit 1; }

bash scripts/izolyatsiya_tekshir.sh
```

> ⚡ **Muhim:** Migratsiya backward compatible bo'lsa — **kod rollback yetarli**,
> DB'ga tegilmaydi. Shuning uchun §7.2 qoidalari qat'iy.

---

8. MONITORING VA KUZATUV

8.1 Stek

```
┌──────────────────────────────────────────────────────────┐
│                    obs-01 (kuzatuv node)                 │
│                                                          │
│  ┌────────────┐  ┌────────────┐  ┌────────────┐  ┌──────┐ │
│  │ Prometheus │  │  Grafana   │  │    Loki    │  │Alert-│ │
│  │ 15s scrape │  │ 14 panel   │  │ log 30d    │  │manag.│ │
│  │ 30 kun     │  │            │  │            │  │→ TG  │ │
│  └─────┬──────┘  └─────┬──────┘  └─────┬──────┘  └──┬───┘ │
└────────┼───────────────┼───────────────┼────────────┼─────┘
         │ scrape        │               │ push       │
  ┌──────┴──────┬────────┴──┬────────────┴──┐         │
  ▼             ▼           ▼               ▼         │
django_prometheus  node_    postgres_    promtail   Sentry (SaaS)
/metrics           exporter  exporter    (journald)  xato + trace
                   redis_exporter
                   nginx_exporter
                                  UptimeRobot (tashqi) ──┘
```

8.2 Prometheus metrikalari (25+)

```
# BIZNES metrikalari
armenu_orders_total{restaurant,status}              # counter
armenu_order_value_sum{restaurant,currency}         # counter
armenu_order_items_total{restaurant}                # counter
armenu_ar_views_total{restaurant,dish}              # counter
armenu_ar_conversion_ratio{restaurant}              # gauge  ⚡ ROI ko'rsatkichi
armenu_active_restaurants                           # gauge
armenu_active_sessions{restaurant}                  # gauge
armenu_table_occupancy_ratio{restaurant}            # gauge
armenu_chat_messages_total{restaurant,lang}         # counter
armenu_avg_cook_time_seconds{restaurant}            # histogram

# TEXNIK metrikalari
armenu_http_request_duration_seconds{endpoint,method,status}   # histogram
armenu_http_requests_total{endpoint,method,status}             # counter
armenu_ws_connections{node,group_type}                         # gauge
armenu_ws_messages_total{direction,type}                       # counter
armenu_db_query_duration_seconds{query_type}                   # histogram
armenu_db_connections{state}                                   # gauge  ⚡ pool
armenu_cache_hit_ratio{cache_name}                             # gauge
armenu_queue_depth{queue}                                      # gauge  ⚡
armenu_queue_oldest_age_seconds{queue}                         # gauge
armenu_job_duration_seconds{job_type,status}                   # histogram
armenu_job_failures_total{job_type,error}                      # counter
armenu_circuit_breaker_state{service}                          # gauge (0=closed,1=half,2=open)
armenu_healer_actions_total{check,result}                      # counter
armenu_worker_heartbeat_age_seconds{queue,instance}            # gauge

# TASHQI integratsiya metrikalari
armenu_external_api_duration_seconds{domain,provider,op}       # histogram
armenu_external_api_total{domain,provider,op,status}           # counter
armenu_ai_cost_usd_total{provider,model,task}                  # counter ⚡ byudjet
armenu_ai_tokens_total{provider,direction}                     # counter
armenu_fiscal_queue_depth                                      # gauge  ⚡ P1
armenu_fiscal_receipts_total{provider,status}                  # counter
armenu_fiscal_oldest_pending_seconds                           # gauge  ⚡ 48 soat
armenu_payment_total{provider,status}                          # counter
armenu_payment_amount_sum{provider}                            # counter
armenu_payment_reconcile_mismatch{provider,kind}               # gauge  ⚡ P1
armenu_telegram_queue_depth{priority}                          # gauge
armenu_telegram_429_total                                      # counter
armenu_s3_operation_duration_seconds{op}                       # histogram
armenu_cdn_cache_hit_ratio                                     # gauge
```

```
# /etc/prometheus/prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels: {env: prod, app: armenu}

alerting:
  alertmanagers: [{static_configs: [{targets: ['localhost:9093']}]}]

rule_files: ['/etc/prometheus/rules/armenu-*.yml']

scrape_configs:
  - job_name: armenu-web
    metrics_path: /metrics
    static_configs: [{targets: ['127.0.0.1:8060']}]
  - job_name: node
    static_configs: [{targets: ['127.0.0.1:9100']}]
  - job_name: postgres
    static_configs: [{targets: ['127.0.0.1:9187']}]
  - job_name: redis
    static_configs: [{targets: ['127.0.0.1:9121']}]
  - job_name: nginx
    static_configs: [{targets: ['127.0.0.1:9113']}]
```

8.3 Grafana panellari (14 ta)

#	Panel	Kim ko'radi	Asosiy grafik
1	🚦 **Umumiy sog'liq** (NOC)	Hamma, TV monitorda	Uptime, 5xx %, p95, active restoran
2	⚡ **Oltin signallar**	On-call	Latency / Traffic / Errors / Saturation
3	🍽 **Biznes**	CEO, sotuv	Buyurtma/soat, o'rtacha chek, konversiya
4	🥽 **AR analitikasi**	Product	AR ko'rish, AR→buyurtma konversiya, chek farqi
5	🌐 **HTTP**	Backend	Endpoint bo'yicha p50/p95/p99, RPS, status
6	🗄 **PostgreSQL**	Backend/DBA	Conn pool, sekin so'rov, replication lag, cache hit
7	🧠 **Redis**	Backend	Memory, evicted, hit ratio, channels
8	⚙ **Navbatlar**	Backend	5 navbat depth, eng eski vazifa, xato darajasi
9	🔌 **WebSocket**	Backend	Ulanishlar, xabar/s, uzilish darajasi
10	🤖 **AI/MCP**	Backend, moliya	Provayder bo'yicha chaqiruv/xarajat/latency/xato
11	💳 **To'lov**	Moliya, on-call	Provayder bo'yicha muvaffaqiyat %, summa, reconcile farqi
12	🧾 **Fiskal**	On-call, moliya	Navbat depth, eng eski pending, 48 soat xavfi
13	🖥 **Infratuzilma**	DevOps	CPU/RAM/disk/tarmoq (barcha node)
14	🐳 **Izolyatsiya**	DevOps	armenu vs bittada port/servis holati, cross-check

8.4 Loki log

```
# /etc/promtail/config.yml
server: {http_listen_port: 9080}
positions: {filename: /var/lib/promtail/positions.yaml}
clients: [{url: http://localhost:3100/loki/api/v1/push}]

scrape_configs:
  - job_name: armenu-journal
    journal:
      max_age: 12h
      labels: {job: armenu, env: prod}
    relabel_configs:
      - source_labels: ['__journal__systemd_unit']
        target_label: unit
      - source_labels: ['__journal_priority_keyword']
        target_label: level
    pipeline_stages:
      - match:
          selector: '{job="armenu"}'
          stages:
            - json:
                expressions:
                  request_id: request_id
                  restaurant_id: restaurant_id
                  level: level
            - labels: {request_id: '', restaurant_id: '', level: ''}

  - job_name: armenu-nginx
    static_configs:
      - targets: [localhost]
        labels: {job: nginx, app: armenu, __path__: /var/log/nginx/armenu_*.log}
```

```
# Foydali LogQL so'rovlari
{job="armenu", level="ERROR"} |= "payment"
{job="armenu"} | json | request_id="3f2a9c81-..."        # bitta so'rovni kuzatish
{job="armenu"} | json | restaurant_id="42" | level=~"WARN|ERROR"
sum by (unit) (rate({job="armenu", level="ERROR"}[5m]))
{job="nginx", app="armenu"} | pattern `<_> <_> "<method> <path> <_>" <status> <_>` | status >= 500
```

8.5 Sentry

```
# config/settings.py
import sentry_sdk
from sentry_sdk.integrations.django import DjangoIntegration
from sentry_sdk.integrations.redis import RedisIntegration

sentry_sdk.init(
    dsn=os.environ['SENTRY_DSN'],
    environment=os.environ.get('DJANGO_ENV', 'production'),
    release=os.environ.get('GIT_SHA', 'unknown'),
    integrations=[DjangoIntegration(), RedisIntegration()],
    traces_sample_rate=0.05,          # 5% trace (xarajat)
    profiles_sample_rate=0.01,
    send_default_pii=False,           # ⚡ PII YUBORILMAYDI
    before_send=scrub_sensitive,      # karta/token/telefon maskalanadi
    ignore_errors=[
        'django.security.DisallowedHost',
        'ClientDisconnected',
    ],
)


def scrub_sensitive(event, hint):
    """Sentry'ga maxfiy ma'lumot BORMAYDI"""
    SENSITIVE = {'password', 'token', 'api_key', 'card', 'pan', 'cvv',
                 'authorization', 'secret', 'phone', 'X-Signature'}
    def walk(d):
        if isinstance(d, dict):
            return {k: ('***' if k.lower() in SENSITIVE else walk(v))
                    for k, v in d.items()}
        if isinstance(d, list):
            return [walk(i) for i in d]
        return d
    return walk(event)
```

8.6 Uptime monitoring (tashqi)

Tekshiruv	URL	Chastota	Joy
Bosh sahifa	`https://armenu.uz/`	1 daq	5 mintaqa
Health	`https://armenu.uz/health/`	1 daq	5 mintaqa
Menyu API	`https://armenu.uz/v1/menu/demo/`	5 daq	3 mintaqa
3D fayl (CDN)	`.../model.glb`	5 daq	3 mintaqa
TLS muddati	armenu.uz	Kunlik	—
Domen muddati	armenu.uz	Haftalik	—

8.7 Alert qoidalari

#	Alert	Shart	Daraja	Kimga	Kanal
1	**Sayt yiqilgan**	`up{job="armenu-web"} == 0` (1 daq)	🔴 P1	On-call	Telegram + **qo'ng'iroq**
2	**Xatolik darajasi**	5xx > 1% (5 daq)	🔴 P1	On-call	Telegram + qo'ng'iroq
3	**To'lov yiqilgan**	`payment{status="failed"}` > 20% (5 daq)	🔴 P1	On-call + moliya	Telegram + qo'ng'iroq
4	**To'lov nomuvofiqligi**	`reconcile_mismatch > 0`	🔴 P1	Moliya + CTO	Telegram + email
5	**Fiskal navbat**	`fiscal_queue_depth > 50` (10 daq)	🔴 P1	On-call	Telegram + qo'ng'iroq
6	**Fiskal 48 soat**	`fiscal_oldest_pending > 158400`	🔴 P1	On-call + restoran	Telegram
7	**DB yiqilgan**	`pg_up == 0` (30 s)	🔴 P1	On-call	Telegram + qo'ng'iroq
8	**Tenant izishi**	`tenant_violation_total > 0`	🔴 **P0**	CTO + hamma	Telegram + qo'ng'iroq
9	Javob sekin	p95 > 1 s (5 daq)	🟠 P2	On-call	Telegram
10	DB conn pool	> 85%	🟠 P2	On-call	Telegram
11	Replication lag	> 30 s	🟠 P2	On-call	Telegram
12	Navbat to'lgan	`queue_depth > 1000`	🟠 P2	On-call	Telegram
13	Worker o'lgan	`heartbeat_age > 180`	🟠 P2	On-call	Telegram
14	Disk	> 85%	🟠 P2	DevOps	Telegram
15	RAM	> 90% (10 daq)	🟠 P2	DevOps	Telegram
16	Circuit breaker OPEN	`cb_state == 2` (5 daq)	🟠 P2	Backend	Telegram
17	S3 xatolari	> 5% (5 daq)	🟠 P2	DevOps	Telegram
18	WS uzilish	> 20% (5 daq)	🟡 P3	Backend	Telegram
19	AI xarajat (kunlik)	> $50/kun	🟡 P3	CTO	Telegram
20	AI byudjet 80%	oylik > $400	🟡 P3	CTO	Telegram
21	Kesh hit past	< 70% (30 daq)	🟡 P3	Backend	Telegram
22	CDN hit past	< 85% (1 soat)	🟡 P3	DevOps	Telegram
23	Telegram 429	> 100/soat	🟡 P3	Backend	Telegram
24	TLS tugayapti	< 14 kun	🟡 P3	DevOps	Telegram + email
25	Backup yiqildi	kunlik dump yo'q	🟠 P2	DevOps	Telegram
26	Backup verify	`pg_restore` sinovi yiqildi	🟠 P2	DevOps	Telegram
27	Restoran nofaol	3 kun buyurtma yo'q	🟢 P4	Sotuv	Telegram (sotuv guruhi)
28	⛔ Izolyatsiya	`bittada-*` inactive yoki port 8040 yo'q	🔴 **P0**	CTO + DevOps	Telegram + qo'ng'iroq

```
# /etc/prometheus/rules/armenu-critical.yml
groups:
  - name: armenu-p1
    interval: 15s
    rules:
      - alert: ARMenuDown
        expr: up{job="armenu-web"} == 0
        for: 1m
        labels: {severity: p1, team: oncall}
        annotations:
          summary: "🔴 ARMENU web javob bermayapti"
          runbook: "https://docs.armenu.uz/runbook#sayt-ochilmayapti"

      - alert: HighErrorRate
        expr: |
          sum(rate(armenu_http_requests_total{status=~"5.."}[5m]))
          / sum(rate(armenu_http_requests_total[5m])) > 0.01
        for: 5m
        labels: {severity: p1, team: oncall}
        annotations:
          summary: "🔴 5xx darajasi {{ $value | humanizePercentage }}"

      - alert: FiscalQueueBacklog
        expr: armenu_fiscal_queue_depth > 50
        for: 10m
        labels: {severity: p1, team: oncall}
        annotations:
          summary: "🔴 Fiskal navbat {{ $value }} — chek yuborilmayapti"
          runbook: "https://docs.armenu.uz/runbook#fiskal-yiqilgan"

      - alert: TenantIsolationViolation
        expr: increase(armenu_tenant_violation_total[1m]) > 0
        for: 0m
        labels: {severity: p0, team: all}
        annotations:
          summary: "⛔⛔ TENANT IZOLYATSIYA BUZILDI — DARHOL ARALASHUV"

      - alert: MebelcityAffected
        expr: |
          probe_success{job="mebelcity-guard"} == 0
          or absent(probe_success{job="mebelcity-guard"})
        for: 2m
        labels: {severity: p0, team: all}
        annotations:
          summary: "⛔ mebelcity (8040) javob bermayapti — ARMENU ta'sir qilgan bo'lishi mumkin"
          runbook: "Darhol ARMENU deploy'ini rollback qiling va izolyatsiya_tekshir.sh"
```

9. LOG STRATEGIYASI

9.1 Darajalar

Daraja	Qachon	Namuna	Saqlash
`DEBUG`	Faqat dev	SQL, kesh hit	✖ prod'da o'chirilgan
`INFO`	Normal biznes hodisa	"Buyurtma #047 yaratildi"	30 kun
`WARNING`	G'ayrioddiy, lekin ishlaydi	"Gemini yiqildi → Groq"	90 kun
`ERROR`	Operatsiya bajarilmadi	"Fiskal chek yuborilmadi"	90 kun
`CRITICAL`	Tizim xavf ostida	"Barcha to'lov provayderi yiqilgan"	1 yil
`AUDIT`	Moliyaviy/xavfsizlik	To'lov, chek, rol o'zgarishi	⚡ **90 kun** (moliya 5 yil)

9.2 JSON format

```
# config/logging.py
import json
import logging
from datetime import datetime, timezone


class JsonFormatter(logging.Formatter):
    def format(self, record):
        payload = {
            'ts': datetime.now(timezone.utc).isoformat(),
            'level': record.levelname,
            'logger': record.name,
            'msg': record.getMessage(),
            'module': record.module,
            'line': record.lineno,
        }
        # Kontekst (middleware o'rnatadi — contextvars)
        for key in ('request_id', 'restaurant_id', 'user_id', 'session_id',
                    'endpoint', 'method', 'status', 'duration_ms'):
            val = getattr(record, key, None)
            if val is not None:
                payload[key] = val
        if record.exc_info:
            payload['exc'] = self.formatException(record.exc_info)
        return json.dumps(payload, ensure_ascii=False)   # ⚡ o'zbekcha matn buzilmasin


LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'json': {'()': 'config.logging.JsonFormatter'},
    },
    'filters': {
        'scrub': {'()': 'config.logging.ScrubFilter'},   # ⚡ PII maskalash
    },
    'handlers': {
        'console': {                        # journald → promtail → Loki
            'class': 'logging.StreamHandler',
            'formatter': 'json',
            'filters': ['scrub'],
        },
        'audit': {                          # ⚡ moliyaviy log ALOHIDA fayl
            'class': 'logging.handlers.TimedRotatingFileHandler',
            'filename': '/var/log/armenu/audit.log',
            'when': 'midnight',
            'backupCount': 90,              # 90 kun
            'formatter': 'json',
            'filters': ['scrub'],
        },
        'finance': {                        # ⚡ to'lov/chek — 5 yil (soliq)
            'class': 'logging.handlers.TimedRotatingFileHandler',
            'filename': '/var/log/armenu/finance.log',
            'when': 'midnight',
            'backupCount': 1826,            # 5 yil
            'formatter': 'json',
            'filters': ['scrub'],
        },
    },
    'loggers': {
        'django':       {'handlers': ['console'], 'level': 'INFO'},
        'armenu':       {'handlers': ['console'], 'level': 'INFO'},
        'integrations': {'handlers': ['console'], 'level': 'INFO'},
        'ai.mcp':       {'handlers': ['console'], 'level': 'INFO'},
        'audit':        {'handlers': ['audit', 'console'], 'level': 'INFO',
                         'propagate': False},
        'finance':      {'handlers': ['finance', 'console'], 'level': 'INFO',
                         'propagate': False},
    },
}
```

9.3 Nima yoziladi / yozilmaydi

✅ YOZILADI	⛔ HECH QACHON YOZILMAYDI
`request_id`, `restaurant_id`, `session_id` (uuid)	🔴 Karta raqami / PAN / CVV
Buyurtma raqami, summa, holat	🔴 API kalitlar, tokenlar, parollar
Provayder nomi, latency, xato kodi	🔴 Mijoz telefoni (to'liq) — oxirgi 4 raqam mumkin
Endpoint, HTTP metod, status	🔴 Mijoz ismi / familiyasi
Foydalanuvchi ID (raqam)	🔴 Telegram user id (xesh qilinadi)
IP (⚠ 90 kun, keyin anonimlashtiriladi)	🔴 `Authorization` header
Fiskal belgi (fiscal_sign)	🔴 Webhook imzo qiymati
Xato stack trace	🔴 Xom webhook body (maskalanmagan)

```
# config/logging.py — ScrubFilter
import re

PATTERNS = [
    (re.compile(r'\b\d{16}\b'),                         '****CARD****'),
    (re.compile(r'\b(?:sk|pk|ak)[-_][A-Za-z0-9_\-]{16,}'), '***KEY***'),
    (re.compile(r'\bBearer\s+[A-Za-z0-9._\-]+'),        'Bearer ***'),
    (re.compile(r'\+?998\d{9}'),                        '+998*******'),
    (re.compile(r'"(password|token|secret|cvv|pan)"\s*:\s*"[^"]*"'),
     r'"\1":"****"'),
]


class ScrubFilter(logging.Filter):
    def filter(self, record):
        msg = record.getMessage()
        for pat, repl in PATTERNS:
            msg = pat.sub(repl, msg)
        record.msg, record.args = msg, ()
        return True
```

9.4 Saqlash muddati

Log turi	Joy	Muddat	Sabab
Ilova log (INFO/WARN)	Loki	30 kun	Debug
Xato log (ERROR/CRIT)	Loki + Sentry	90 kun	Tahlil
nginx access	Fayl + Loki	30 kun	Trafik tahlili
AuditLog (DB jadval)	PostgreSQL	**3 yil**	Xavfsizlik, kim nima qildi
finance.log	Fayl + S3	⚡ **5 yil**	Soliq talabi
Fiskal cheklar (DB + PDF)	PG + S3	⚡ **5 yil**	Qonun
systemd journal	Lokal	7 kun (`SystemMaxUse=2G`)	Disk tejash
Backup log	Fayl	1 yil	Tiklash tarixi

```
# /etc/logrotate.d/armenu
/var/log/armenu/*.log {
    daily
    rotate 90
    compress
    delaycompress
    missingok
    notifempty
    create 0640 armenu armenu
    sharedscripts
    postrotate
        systemctl reload armenu >/dev/null 2>&1 || true
    endscript
}

/var/log/armenu/finance.log {
    daily
    rotate 1826
    compress
    missingok
    notifempty
    create 0600 armenu armenu      # ⚡ faqat egasi o'qiydi
    olddir /var/log/armenu/finance-archive
    postrotate
        # kunlik arxiv S3 ga (WORM bucket)
        aws s3 cp /var/log/armenu/finance-archive/ \
            s3://armenu-audit/finance/ --recursive --sse AES256 >/dev/null 2>&1 || true
    endscript
}
```

10. BACKUP VA TIKLASH

10.1 Nima / qachon / qayerda / qancha

Ma'lumot	Chastota	Usul	Joy	Saqlash	RPO	RTO
PostgreSQL to'liq	Kunlik 03:00	`pg_dump -Fc -Z6`	S3 (boshqa region) + lokal	30 kun	24 s	30 daq
PostgreSQL WAL	Uzluksiz	`pgbackrest` / `archive_command`	S3	7 kun	⚡ **~0**	15 daq
Predeploy dump	Har deploy	`pg_dump -Fc`	Lokal `/backup`	7 kun	—	10 daq
Redis snapshot	6 soatda	`BGSAVE` + `rdb` nusxa	S3	3 kun	6 s	5 daq
3D fayllar (S3)	Real-time	S3 versioning + cross-region replica	S3 ×2	Doimiy	~0	1 soat
Media (rasm)	Kunlik	`rclone sync`	S3	30 kun	24 s	2 soat
Fiskal cheklar	Real-time	DB + S3 (WORM)	S3	⚡ 5 yil	0	—
Kod	Har commit	Git	GitHub + lokal mirror	Doimiy	0	5 daq
Konfiguratsiya	Har o'zgarish	Git (shifrlangan, `sops`)	GitHub	Doimiy	0	10 daq
`.env` sirlari	Qo'lda	`age` / `sops` shifrlangan	1Password + offline	Doimiy	—	—

10.2 Backup skripti

```
#!/bin/bash
# /home/armenu/app/scripts/backup.sh
# Cron: 0 3 * * *  (armenu crontab)
set -euo pipefail
source /home/armenu/app/.env

TS=$(date +%Y%m%d_%H%M%S)
DAY=$(date +%Y-%m-%d)
DIR=/backup
LOG=/var/log/armenu/backup.log
exec >> "$LOG" 2>&1
echo "===== $TS backup boshlandi ====="

# --- 1. PostgreSQL ---
DUMP="$DIR/armenu_${TS}.dump"
pg_dump -Fc -Z6 -d "$DB_NAME" -f "$DUMP"
SIZE=$(du -h "$DUMP" | cut -f1)
echo "✓ PG dump: $DUMP ($SIZE)"

# --- 2. Yaxlitlik tekshiruvi (dump o'qiladimi) ---
pg_restore --list "$DUMP" > /dev/null
echo "✓ Dump yaxlitligi OK"

# --- 3. Redis ---
redis-cli -n 5 BGSAVE > /dev/null
sleep 5
cp /var/lib/redis/dump.rdb "$DIR/redis_${TS}.rdb"
echo "✓ Redis snapshot"

# --- 4. Konfiguratsiya ---
tar czf "$DIR/config_${TS}.tar.gz" \
    /etc/nginx/sites-available/armenu.conf \
    /etc/systemd/system/armenu*.service \
    /etc/systemd/system/armenu*.socket \
    /etc/systemd/system/armenu*.target \
    2>/dev/null || true
echo "✓ Konfiguratsiya"

# --- 5. S3 (shifrlangan) ---
aws s3 cp "$DUMP"                "s3://armenu-backup/db/$DAY/"     --sse AES256
aws s3 cp "$DIR/redis_${TS}.rdb"  "s3://armenu-backup/redis/$DAY/"  --sse AES256
aws s3 cp "$DIR/config_${TS}.tar.gz" "s3://armenu-backup/config/$DAY/" --sse AES256
echo "✓ S3 ga yuklandi"

# --- 6. Media sinxronizatsiya ---
rclone sync /var/armenu/media s3:armenu-media/ --exclude '/tmp/**' --quiet
echo "✓ Media sync"

# --- 7. Lokal tozalash ---
find "$DIR" -name 'armenu_*.dump' -mtime +7  -delete
find "$DIR" -name 'redis_*.rdb'   -mtime +3  -delete
find "$DIR" -name 'config_*.tar.gz' -mtime +14 -delete

# --- 8. Xabar ---
curl -s -X POST "https://api.telegram.org/bot${TG_BOT}/sendMessage" \
  -d chat_id="${TG_CHAT}" \
  -d text="✓ ARMENU backup: ${SIZE} · ${DAY}" > /dev/null

echo "===== backup tugadi ====="
```

10.3 ⚡ Avtomatik tiklash sinovi (verify)

> Sinalmagan backup — backup emas.**Har kecha 04:30 da haqiqiy `pg_restore` sinovi.**

```
#!/bin/bash
# /home/armenu/app/scripts/backup_verify.sh
# Cron: 30 4 * * *
set -euo pipefail
source /home/armenu/app/.env

LATEST=$(ls -t /backup/armenu_*.dump | head -1)
TESTDB="armenu_verify_$(date +%s)"
echo "🔍 Tekshiruv: $LATEST → $TESTDB"

createdb -O armenu "$TESTDB"
trap 'dropdb --if-exists "$TESTDB"' EXIT

# 1. Tiklash
if ! pg_restore -d "$TESTDB" -j 4 --no-owner "$LATEST" 2>/tmp/restore_err.log; then
  grep -qv 'already exists' /tmp/restore_err.log && {
    curl -s -X POST "https://api.telegram.org/bot${TG_BOT}/sendMessage" \
      -d chat_id="${TG_CHAT}" \
      -d text="🔴 P2 BACKUP TIKLANMADI: $LATEST%0A$(head -5 /tmp/restore_err.log)"
    exit 1; }
fi

# 2. Mazmun tekshiruvi — bo'sh baza tiklanmasin
ORDERS=$(psql -tAd "$TESTDB" -c 'SELECT count(*) FROM orders_order')
DISHES=$(psql -tAd "$TESTDB" -c 'SELECT count(*) FROM menu_dish')
RESTS=$(psql  -tAd "$TESTDB" -c 'SELECT count(*) FROM tenants_restaurant')
PAY=$(psql    -tAd "$TESTDB" -c "SELECT count(*) FROM payments_payment WHERE state='paid'")

echo "Buyurtma=$ORDERS Taom=$DISHES Restoran=$RESTS To'lov=$PAY"

if [ "$RESTS" -lt 1 ] || [ "$DISHES" -lt 1 ]; then
  curl -s -X POST "https://api.telegram.org/bot${TG_BOT}/sendMessage" \
    -d chat_id="${TG_CHAT}" \
    -d text="🔴 P2 BACKUP BO'SH! restoran=$RESTS taom=$DISHES"
  exit 1
fi

# 3. Migratsiya holati
psql -tAd "$TESTDB" -c 'SELECT count(*) FROM django_migrations' > /dev/null

# 4. Metrikaga yozish
cat > /var/lib/node_exporter/textfile/backup_verify.prom <<EOF
armenu_backup_verify_success 1
armenu_backup_verify_timestamp $(date +%s)
armenu_backup_verify_orders $ORDERS
armenu_backup_verify_size_bytes $(stat -c%s "$LATEST")
EOF

curl -s -X POST "https://api.telegram.org/bot${TG_BOT}/sendMessage" \
  -d chat_id="${TG_CHAT}" \
  -d text="✓ Backup verify OK · restoran=$RESTS taom=$DISHES buyurtma=$ORDERS"
echo "✓ Verify muvaffaqiyatli"
```

10.4 Tiklash runbook

```
┌──────────────────────────────────────────────────────┐
│ SSENARIY A — TASODIFAN O'CHIRILGAN MA'LUMOT (bir jadval)  │
├──────────────────────────────────────────────────────┤
│ 1. ● DARHOL to'xating: sudo -u armenu touch /var/armenu/MAINTENANCE
│ 2. Vaqtinchalik bazaga tiklang (prod'ga TEGMANG):
│    createdb armenu_rec
│    pg_restore -d armenu_rec -t <jadval> /backup/armenu_<TS>.dump
│ 3. Faqat kerakli qatorlarni ko'chiring:
│    psql -d armenu_rec -c "COPY (SELECT * FROM t WHERE ...) TO '/tmp/x.csv' CSV"
│    psql -d armenu_db  -c "COPY t FROM '/tmp/x.csv' CSV"
│ 4. Tekshiring, maintenance o'chiring
│ ⏲ RTO ~20 daqiqa
└──────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────┐
│ SSENARIY B — DB BUZILGAN (to'liq tiklash)                 │
├──────────────────────────────────────────────────────┤
│ 1. sudo -u armenu touch /var/armenu/MAINTENANCE
│ 2. sudo systemctl stop armenu armenu-ws armenu-workers.target
│ 3. Eski bazani saqlab qo'ying (o'CHIRMANG!):
│    psql -c 'ALTER DATABASE armenu_db RENAME TO armenu_db_broken_$(date +%s)"
│ 4. createdb -O armenu armenu_db
│    psql -d armenu_db -c 'CREATE EXTENSION vector; CREATE EXTENSION pg_trgm;'
│ 5. pg_restore -d armenu_db -j 4 --no-owner /backup/armenu_<TS>.dump
│ 6. ⚡ WAL bilan PITR (oxirgi daqiqagacha):
│    pgbackrest --stanza=armenu --type=time \
│               --target="2026-09-08 14:23:00" restore
│ 7. python manage.py migrate --check
│ 8. sudo systemctl start armenu armenu-ws armenu-workers.target
│ 9. bash scripts/smoke.sh
│ 10. rm /var/armenu/MAINTENANCE
│ 11. ⚡ RECONCILIATION: to'lov/fiskal farqini tekshiring
│     python manage.py reconcile_payments --from=<TS> --to=now
│ ⏲ RTO ~30 daqiqa
└──────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────┐
│ SSENARIY C — SERVER BUTUNLAY YO'Q (falokat)               │
├──────────────────────────────────────────────────────┤
│ 1. Yangi VPS oling (bir xil OS: Ubuntu 24.04)
│ 2. bash <(curl -fsSL https://raw.../scripts/bootstrap.sh)   ← §2.4
│ 3. Sirlarni tiklang: sops -d secrets/env.enc > /home/armenu/app/.env
│ 4. aws s3 cp s3://armenu-backup/db/<sana>/armenu_<TS>.dump /backup/
│ 5. Ssenariy B ning 4-7 qadamlari
│ 6. aws s3 sync s3://armenu-media/ /var/armenu/media/
│ 7. certbot certonly ... (yangi sertifikat)
│ 8. DNS A yozuvini yangi IP ga (TTL 300 → tez tarqaladi)
│ 9. Cloudflare origin IP ni yangilang
│ 10. Smoke test + reconciliation
│ ⏲ RTO ~2-4 soat
└──────────────────────────────────────────────────────┘
```

10.5 Choraklik tiklash mashqi

Qadam	Kim	Vaqt
1. Staging'ga prod backup'idan to'liq tiklash	DevOps	40 daq
2. Smoke test + funksional tekshiruv	Backend	20 daq
3. RTO o'lchash va yozib olish	DevOps	—
4. Runbook'dagi noaniqliklarni tuzatish	Hamma	30 daq
5. Hisobot (nima ishlamadi)	DevOps	15 daq

---

11. RUNBOOK — 10 TIPIK HOLAT

🔴 R1 — Sayt ochilmayapti

Belgisi: 502/503/504, UptimeRobot alert, "sayt ishlamayapti" xabarlari.

```
# --- DIAGNOSTIKA ---
systemctl status armenu armenu.socket armenu-ws --no-pager
ss -ltnp | grep -E '8060|8061'
curl -sv http://127.0.0.1:8060/health/
journalctl -u armenu -n 100 --no-pager
nginx -t && systemctl status nginx --no-pager
df -h /                              # disk to'lgan bo'lishi mumkin
free -h                              # OOM bo'lganmi
dmesg | tail -20 | grep -i 'killed process'
```

Tuzatish:

Sabab	Qadam
gunicorn o'lgan	`sudo systemctl restart armenu`
Socket buzilgan	`sudo systemctl restart armenu.socket armenu.service`
nginx konfig xato	`nginx -t` → xatoni tuzating → `sudo systemctl reload nginx`
OOM kill	`MemoryMax` oshiring yoki `--workers` kamaytiring
Disk 100%	R7 ga o'ting
DB yiqilgan	R3 ga o'ting
Yomon deploy	`bash scripts/rollback.sh`

Eskalatsiya: 10 daqiqada tiklanmasa → CTO. 30 daqiqada → falokat rejimi (§10.4 C).

🟠 R2 — Sekin ishlayapti (p95 > 1 s)

```
# --- DIAGNOSTIKA ---
# 1. Qaysi endpoint sekin
curl -s localhost:8060/metrics | grep armenu_http_request_duration | sort -t= -k2 -rn | head

# 2. DB sekin so'rovlar
psql -d armenu_db -c "
SELECT round(mean_exec_time::numeric,1) ms, calls, round(total_exec_time::numeric) total,
       left(query, 90) q
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 15;"

# 3. Bloklovchi qulflar
psql -d armenu_db -c "
SELECT pid, now()-query_start AS dur, wait_event_type, wait_event, left(query,80)
FROM pg_stat_activity WHERE state<>'idle' AND now()-query_start > interval '3s'
ORDER BY dur DESC;"

# 4. Kesh hit
redis-cli -n 5 INFO stats | grep -E 'keyspace_hits|keyspace_misses'

# 5. Yuk
uptime; iostat -x 2 3; htop
```

Tuzatish:

Sabab	Qadam
Indeks yo'q	`EXPLAIN ANALYZE` → `AddIndexConcurrently` migratsiyasi
Kesh sovuq (deploy'dan keyin)	`python manage.py warm_cache` — 5 daqiqa kuting
N+1 so'rov	`select_related` / `prefetch_related` qo'shing
Uzoq qulf	`SELECT pg_cancel_backend(<pid>)` (avval `terminate` emas!)
Worker yetmaydi	`--workers` oshiring yoki node qo'shing
Redis to'lgan	R7 (`maxmemory-policy allkeys-lru` tekshiring)

🟠 R3 — DB ulanishlari tugadi

Belgisi: `FATAL: sorry, too many clients already`, `OperationalError`.

```
# --- DIAGNOSTIKA ---
psql -d armenu_db -c "SELECT count(*), state FROM pg_stat_activity GROUP BY state;"
psql -d armenu_db -c "SHOW max_connections;"
psql -d armenu_db -c "
SELECT pid, application_name, state, now()-state_change AS idle_for, left(query,60)
FROM pg_stat_activity WHERE state='idle in transaction'
ORDER BY idle_for DESC LIMIT 20;"
```

```
# --- TUZATISH ---
# 1. "idle in transaction" 10 daqiqadan ortiq — bu leak
psql -d armenu_db -c "
SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE state='idle in transaction' AND now()-state_change > interval '10 min';"

# 2. Sabab: worker'da ochiq tranzaksiya (⚡ TenantAwareThread finally bloki yo'q)
sudo systemctl restart armenu-workers.target

# 3. Doimiy yechim: PgBouncer
sudo apt install -y pgbouncer
# /etc/pgbouncer/pgbouncer.ini
#   pool_mode = transaction
#   max_client_conn = 1000
#   default_pool_size = 25
# settings.py: DB_PORT=6432, DB_CONN_MAX_AGE=0  ⚡ pgbouncer bilan 0 bo'lishi SHART
```

🟠 R4 — Navbat to'lgan

```
# --- DIAGNOSTIKA ---
python manage.py queue_stats            # har navbat: depth, oldest, failed
redis-cli -n 5 LLEN armenu:queue:model3d
systemctl status 'armenu-worker@*' --no-pager
psql -d armenu_db -c "
SELECT queue, status, count(*), max(now()-created_at) AS oldest
FROM jobs_job GROUP BY queue, status ORDER BY 3 DESC;"
```

Tuzatish:

Sabab	Qadam
Worker o'lgan	`sudo systemctl restart armenu-worker@<navbat>`
Orphan job (`running` , heartbeat eski)	`python manage.py requeue_orphans --queue=<n>` (healer ham qiladi)
Tashqi API sekin (Meshy)	`armenu_circuit_breaker_state` ni tekshiring — normal, kuting
Poison message (bir job cheksiz xato)	`python manage.py move_to_dlq --job-id=<id>`
Haqiqiy yuk	Konkurentlik oshiring: `--concurrency=4` yoki worker node qo'shing
Fiskal navbat	⚡ R5 ga o'ting — bu P1

🔴 R5 — Fiskal (OFD) yiqilgan

Belgisi: `armenu_fiscal_queue_depth > 50` , cheklar yuborilmayapti. ⚡**Qonuniy risk.**

```
# --- DIAGNOSTIKA ---
python manage.py fiscal_status
# → navbat, eng eski pending, provayder holati, circuit breaker

curl -s localhost:8060/metrics | grep -E 'fiscal_queue_depth|fiscal_oldest_pending'
journalctl -u 'armenu-worker@fiscal' -n 200 --no-pager | grep -i error

# Provayderlarni tekshirish
python manage.py shell -c "
from apps.integrations.fiscal.registry import ALL_ADAPTERS
for c in ALL_ADAPTERS:
    a = c(); print(a.name, a.is_configured(), a.health_check())"
```

Tuzatish:

```
# 1. Circuit breaker OPEN bo'lsa va provayder tiklangan bo'lsa — qo'lda yopish
python manage.py reset_circuit --service=fiscal.uzkassa

# 2. Zaxira provayderga o'tkazish
python manage.py fiscal_set_primary --provider=epos

# 3. Navbatni qayta ishga tushirish
python manage.py fiscal_retry_all --max-age-hours=48

# 4. ⚡ 48 SOAT YAQINLASHSA — restoranlarga xabar
python manage.py fiscal_notify_manual --older-than-hours=40
# → har restoran adminiga: "Chekni kassada qo'lda uring"
```

Vaqt	Harakat
0-1 soat	Kuzatish, retry o'z-o'zidan ishlaydi
1-6 soat	🟠 Zaxira provayderga o'tish, OFD support'ga murojaat
6-24 soat	🔴 Restoranlarga xabar: qo'lda kassa tayyorlansin
24-48 soat	🔴 P1 — CTO + huquqiy maslahat
>48 soat	⛔ Qo'lda chek MAJBURIY, soliq bilan yozishma

🟠 R6 — WebSocket uzilyapti

```
# --- DIAGNOSTIKA ---
systemctl status armenu-ws --no-pager
ss -tn state established '( sport = :8061 )' | wc -l
journalctl -u armenu-ws -n 100 --no-pager
redis-cli -n 6 INFO clients
redis-cli -n 6 PUBSUB CHANNELS 'armenu:asgi*' | head
curl -s localhost:8060/metrics | grep armenu_ws_connections
```

Tuzatish:

Sabab	Qadam
daphne o'lgan	`sudo systemctl restart armenu-ws`
`LimitNOFILE` yetmaydi	`LimitNOFILE=131072` (§4.4) → `daemon-reload` → restart
nginx `proxy_read_timeout` past	`3600s` ga qo'ying → `reload nginx`
Redis channels to'lgan	`redis-cli -n 6 INFO memory: maxmemory` oshiring
Cloudflare WS bloklayapti	CF'da `/ws/` uchun Bypass rule tekshiring
Klient tarmog'i (mobil)	Normal — auto-reconnect ishlaydi, polling fallback bor

> ⚡ **Muhim:** WS yiqilsa ham menyu, buyurtma, to'lov ISHLAYDI (polling 10 s).
> Bu P3, P1 emas.

🟠 R7 — Disk to'ldi

```
# --- DIAGNOSTIKA ---
df -h
du -sh /var/log/* /var/armenu/* /backup/* /var/lib/postgresql/* 2>/dev/null | sort -rh | head -15
ncdu /var                   # interaktiv
journalctl --disk-usage
psql -d armenu_db -c "
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) sz
FROM pg_catalog.pg_statio_user_tables ORDER BY pg_total_relation_size(relid) DESC LIMIT 10;"
```

```
# --- TUZATISH (tartib bilan, xavfsizdan boshlab) ---
# 1. journald
sudo journalctl --vacuum-size=500M

# 2. Eski backup (lokal — S3 da nusxasi bor)
find /backup -name '*.dump' -mtime +7 -delete
find /backup -name '*.rdb'  -mtime +3 -delete

# 3. nginx kesh (3D fayl — S3 dan qayta yuklanadi)
sudo rm -rf /var/cache/nginx/ar/* && sudo systemctl reload nginx

# 4. Eski log
sudo logrotate -f /etc/logrotate.d/armenu
find /var/log/armenu -name '*.gz' -mtime +90 -delete

# 5. Eski media (S3 ga ko'chirilgan, lokal nusxa kerak emas)
python manage.py prune_local_media --older-than-days=30 --verify-s3

# 6. ⚠ PostgreSQL — ehtiyot bo'ling
psql -d armenu_db -c "VACUUM (ANALYZE);"
# VACUUM FULL jadval qulflaydi — faqat maintenance oynasida!

# 7. ● HECH QACHON: /home/user/... dagi hech narsani o'chirmang
```

Oldini olish: `armenu-healer` disk >85% da avtomatik 1-4 qadamlarni bajaradi.

🟠 R8 — AI byudjeti tugadi

Belgisi: Chatbot javob bermayapti, log'da `AI_BUDGET_EXCEEDED`.

```
# --- DIAGNOSTIKA ---
python manage.py ai_usage --period=month --group-by=provider
python manage.py ai_usage --period=today --group-by=restaurant --top=20
curl -s localhost:8060/metrics | grep armenu_ai_cost_usd_total
```

```
# --- TUZATISH ---
# 1. Bitta restoran ko'p sarflayaptimi (abuse?)
python manage.py ai_usage --period=today --group-by=restaurant --top=5
#    → shubhali bo'lsa: rate limit qattiqlashtiring

# 2. Vaqtincha limitni oshirish (asoslangan bo'lsa)
python manage.py set_ai_limit --restaurant=42 --daily=5.0

# 3. Arzonroq provayderga o'tkazish (Gemini/DeepSeek prioritetini oshiring)
python manage.py set_ai_priority --task=chat --order=gemini,deepseek,groq,claude

# 4. Kesh TTL uzaytirish (chat javoblari)
# .env: AI_CACHE_TTL=7200 → sudo systemctl reload armenu

# 5. Menyu konteksti kattami (token isrofi)
python manage.py ai_context_size --restaurant=42
#    > 2500 token bo'lsa → build_menu_context() ni qisqartiring
```

Sabab	Doimiy yechim
Restoran soni o'sdi	Byudjetni qayta hisoblang (restoran×$1/oy)
3D generatsiya ko'paydi	Kutubxonadan foydalanish ulushini oshiring (80/20)
Bitta restoran abuse	Kunlik limit + chat rate limit (10/5daq)
Kesh hit past	TTL uzaytirish, prompt normallashtirish

🟠 R9 — To'lov kelmayapti

Belgisi: Mijoz to'ladi, lekin buyurtma `unpaid` qolyapti.

```
# --- DIAGNOSTIKA ---
# 1. Webhook keldimi
psql -d armenu_db -c "
SELECT provider, method, received_at, raw->>'method' m
FROM integrations_webhookevent
WHERE received_at > now()-interval '1 hour' ORDER BY received_at DESC LIMIT 20;"

# 2. nginx webhook so'rovlarini ko'rdimi
grep '/webhooks/' /var/log/nginx/armenu_access.log | tail -30

# 3. Xato log
journalctl -u armenu --since '1 hour ago' | grep -i -E 'payme|click|uzum|webhook'

# 4. Circuit breaker
curl -s localhost:8060/metrics | grep 'circuit_breaker_state{service="payment'

# 5. Konkret buyurtma
python manage.py payment_trace --order=4471
```

Tuzatish:

Sabab	Qadam
Webhook umuman kelmagan	Provayder kabinetida webhook URL ni tekshiring; nginx IP allowlist'ni tekshiring
Imzo xato (401)	`.env` dagi `PAYME_KEY` / `CLICK_SECRET` ni provayder kabinetiga solishtiring
Summa mos emas	⚡ Bu **himoya ishladi** — buyurtma summasi o'zgargan. Qo'lda tekshiring
Kod xatosi (500)	Sentry'da stack trace → hotfix; `WebhookEvent` xom saqlangan → qayta ishlang
Provayder tomonida kechikish	Kuting; 30 daqiqadan keyin reconciliation topadi

```
# ⚡ Yo'qolgan to'lovni tiklash (xom event'dan qayta ishlash)
python manage.py replay_webhooks --provider=payme --since='2026-09-08 12:00' --dry-run
python manage.py replay_webhooks --provider=payme --since='2026-09-08 12:00'

# ⚡ Darhol reconciliation (kunlik cron'ni kutmasdan)
python manage.py reconcile_payments --provider=payme --date=today
```

Eskalatsiya: Pul ta'sirlangan → darhol CTO + moliya. Har holat hujjatlashtiriladi.

🔴 R10 — Migratsiya xato bilan tugadi

Belgisi: Deploy'da `migrate` yiqildi, DB yarim holatda.

```
# --- DIAGNOSTIKA ---
python manage.py showmigrations | grep -v '\[X\]'
psql -d armenu_db -c "SELECT app, name, applied FROM django_migrations ORDER BY id DESC LIMIT 10;"
psql -d armenu_db -c "
SELECT pid, now()-query_start dur, left(query,100) FROM pg_stat_activity
WHERE query ILIKE '%ALTER TABLE%' OR query ILIKE '%CREATE INDEX%';"
```

```
# --- TUZATISH ---
# A) Migratsiya HALI ishlayapti (uzoq indeks) — SABR QILING
#    CONCURRENTLY indeks katta jadvalda 10-30 daqiqa olishi mumkin
#    ● TO'XTATMANG — yarim indeks qoladi

# B) Qulflanib qolgan
psql -d armenu_db -c "SELECT pg_cancel_backend(<pid>);"       # avval cancel
# ishlamasa:
psql -d armenu_db -c "SELECT pg_terminate_backend(<pid>);"

# C) Yaroqsiz indeks qolgan (CONCURRENTLY yarim qolganda)
psql -d armenu_db -c "SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;"
psql -d armenu_db -c "DROP INDEX CONCURRENTLY <nom>;"
python manage.py migrate                      # qaytadan

# D) Migratsiya qo'llanmagan, lekin sxema o'zgargan
python manage.py migrate <app> <oldingi_nomer> --fake
# ⚠ FAQAT sxemani qo'lda tekshirgandan KEYIN

# E) Umid yo'q — predeploy dump'dan tiklash
sudo systemctl stop armenu armenu-ws armenu-workers.target
TS=$(cat /home/armenu/.last_deploy_ts)
pg_restore -c -d armenu_db "/backup/predeploy_${TS}.dump"
bash scripts/rollback.sh
```

Oldini olish: CI darvoza 6 (`lintmigrations`, rollback sinovi) — §6.2.

---

12. OPERATSION RITM

12.1 Kunlik (~15 daqiqa, ertalab)

```
# /home/armenu/app/scripts/kunlik_tekshiruv.sh
```

#	Tekshiruv	Buyruq / joy	Yashil mezon
1	Servislar holati	`systemctl status 'armenu*'`	Hammasi `active`
2	⛔ Izolyatsiya	`bash scripts/izolyatsiya_tekshir.sh`	✅ chiqishi
3	Kechagi xatolar	Grafana panel 2	5xx < 0,5%
4	p95 latency	Grafana panel 5	< 250 ms
5	Backup	`ls -lh /backup/ | tail -3`	Kechagi dump bor
6	Backup verify	Telegram xabari	✅ keldi
7	Fiskal navbat	`python manage.py fiscal_status`	depth < 10
8	To'lov reconcile	Kechagi hisobot	0 farq
9	Navbatlar	`python manage.py queue_stats`	Hammasi < 100
10	Disk	`df -h /`	< 75%
11	AI xarajat	Grafana panel 10	Byudjet ichida
12	Sentry yangi xatolar	Sentry inbox	0 yangi kritik

12.2 Haftalik (~1 soat, dushanba)

#	Ish	Kim
1	`pip-audit` + `npm audit` — yangi CVE	Backend
2	Sekin so'rovlar top-15 (`pg_stat_statements`) tahlili	Backend
3	Log'dagi takrorlanuvchi WARNING'lar	Backend
4	Disk o'sish tendensiyasi (30 kunlik grafik)	DevOps
5	AI xarajat tendensiyasi va provayder samaradorligi	CTO
6	Circuit breaker ochilish statistikasi	Backend
7	Cloudflare IP ro'yxatini yangilash	Avtomatik (cron)
8	Alert "shovqini" ko'rigi — noto'g'ri alert bormi	On-call
9	Ochiq incident'lar ko'rigi	Hamma
10	⛔ mebelcity holati — hech narsa buzilmaganmi	DevOps

12.3 Oylik (~half kun)

#	Ish	Kim
1	🌡 **Chaos engineering** (§10 dagi 7 ta test)	DevOps + Backend
2	⚡ **Yuklama testi** (k6, `p95<250ms`, `fail<1%`)	Backend
3	Sig'im rejalashtirish (o'sish → keyingi node qachon)	CTO
4	Xarajat tahlili (server + AI + CDN + OFD)	CTO
5	Backup'dan **to'liq tiklash** (staging'da)	DevOps
6	Bog'liqliklarni yangilash (minor/patch)	Backend
7	IKPU katalog snapshot yangilash	Avtomatik (cron)
8	Alert qoidalarini qayta ko'rish	On-call
9	Grafana panellarini yangilash	DevOps
10	Runbook'ni yangilash (o'tgan oy incident'lari asosida)	On-call

12.4 Choraklik (1-2 kun)

#	Ish	Kim
1	🔑 **API kalitlar rotatsiyasi** (AI, to'lov, OFD, Telegram)	CTO
2	🔒 Xavfsizlik auditi (`bandit`, `check --deploy`, tashqi pentest yiliga 1)	CTO
3	🚨 **Falokatdan tiklash mashqi** (§10.5)	Hamma
4	📉 SLA/SLO hisoboti (99,9% bajarildimi)	CTO
5	🗄 Django/PostgreSQL major versiya rejasi	Backend
6	📜 Log saqlash siyosati auditi (5 yil moliya, PII yo'qmi)	CTO
7	🏗 Arxitektura ko'rigi (bu hujjat yangilanadi)	Arxitektor
8	👥 On-call navbat va eskalatsiya ro'yxatini yangilash	CTO
9	💰 Server xarajati vs daromad nisbati	CEO + CTO
10	⛔ Izolyatsiya auditi (sudoers, pg_hba, Redis ACL)	DevOps

13. ON-CALL

13.1 Kim va qachon

Bosqich	Jamoa hajmi	Sxema
0-6 oy	2 dasturchi	Ikkalasi ham, "kim bo'sh bo'lsa"; kechasi faqat P0/P1
6-12 oy	4 texnik	Haftalik navbat, 1 asosiy + 1 zaxira
12+ oy	6+ texnik	Haftalik, follow-the-sun emas (bitta vaqt zonasi)

Ish vaqti: O'zbekiston restoranlarining pik vaqti — **12:00-15:00** va **18:00-23:00**.
⚡ Bu oynalarda deploy **QILINMAYDI**.

Vaqt	Javob berish	Kim
09:00-23:00	P1: 5 daq · P2: 30 daq	Asosiy on-call
23:00-09:00	P0/P1: 15 daq · P2: ertalab	Asosiy on-call (qo'ng'iroq)
Dam olish kunlari	Xuddi shunday	Navbatchi

13.2 Daraja va javob vaqti

Daraja	Ta'rif	Javob	Tiklash maqsadi	Kanal
🔴 **P0**	Tenant sizishi, ma'lumot yo'qolishi, mebelcity ta'sirlangan	**Darhol**	1 soat	Qo'ng'iroq + Telegram + hamma
🟠 **P1**	Sayt yiqilgan, to'lov/fiskal ishlamayapti	5 daq (kunduzi) / 15 daq (kechasi)	2 soat	Qo'ng'iroq + Telegram
🟡 **P2**	Sekin, navbat to'lgan, bir modul ishlamayapti	30 daqiqa	8 soat	Telegram
🔵 **P3**	WS uzilishi, AI xarajat, kesh	Ish vaqtida	3 kun	Telegram
⚪ **P4**	Kosmetik, biznes bildirishnoma	Sprint'da	—	Telegram (guruh)

13.3 Eskalatsiya zanjiri

```
        ALERT (Alertmanager)
              │
              ▼
   ┌──────────────────────┐
   │ 1. ASOSIY ON-CALL    │  Telegram + (P0/P1: qo'ng'iroq)
   │    Backend dasturchi │
   └──────────┬───────────┘
              │ 15 daqiqa javob yo'q
              │ yoki "yordam kerak"
              ▼
   ┌──────────────────────┐
   │ 2. ZAXIRA ON-CALL    │  Telegram + qo'ng'iroq
   │    2-dasturchi       │
   └──────────┬───────────┘
              │ 30 daqiqa (P0/P1 hal bo'lmadi)
              ▼
   ┌──────────────────────┐
   │ 3. CTO               │  Qo'ng'iroq
   │  qaror: rollback /   │
   │  maintenance /       │
   │  tashqi yordam       │
   └──────────┬───────────┘
              │ 1 soat (P0 davom etyapti)
              ▼
   ┌──────────────────────┐
   │ 4. CEO + huquqiy     │  ⚡ Mijozlarga xabar
   │    + PR              │  Status sahifa yangilanadi
   └──────────────────────┘

TASHQI ESKALATSIYA (parallel):
  • To'lov muammosi   → Payme/Click/Uzum support (shartnomadagi raqam)
  • Fiskal muammosi   → OFD support + zarurat bo'lsa Soliq qo'mitasi
  • Server/tarmoq     → Hosting provayder support
  • DDoS              → Cloudflare "Under Attack" rejimi + support
```

13.4 On-call to'plami

```
# ~/.bashrc yoki ~/armenu-oncall.sh
alias a-status='systemctl status "armenu*" --no-pager'
alias a-log='journalctl -u armenu -f --output=cat'
alias a-wslog='journalctl -u armenu-ws -f --output=cat'
alias a-err='journalctl -u armenu --since "1 hour ago" | grep -i error'
alias a-health='curl -s localhost:8060/health/deep/ | jq'
alias a-queues='cd /home/armenu/app && venv/bin/python manage.py queue_stats'
alias a-fiscal='cd /home/armenu/app && venv/bin/python manage.py fiscal_status'
alias a-db='psql -d armenu_db'
alias a-conns='psql -d armenu_db -c "SELECT count(*),state FROM pg_stat_activity GROUP BY state"'
alias a-slow='psql -d armenu_db -c "SELECT round(mean_exec_time::numeric,1) ms,calls,left(query,80) FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10"'
alias a-izol='bash /home/armenu/app/scripts/izolyatsiya_tekshir.sh'
alias a-maint-on='sudo -u armenu touch /var/armenu/MAINTENANCE && echo "🚧 Maintenance YOQILDI"'
alias a-maint-off='sudo -u armenu rm -f /var/armenu/MAINTENANCE && echo "✅ Maintenance o'\''chirildi"'
alias a-rollback='bash /home/armenu/app/scripts/rollback.sh'
alias a-smoke='bash /home/armenu/app/scripts/smoke.sh'
```

Kerakli havolalar (on-call kartochkasida):

Nima	Havola
Grafana	`https://obs.armenu.uz/d/armenu-health`
Sentry	`https://sentry.io/organizations/armenu/issues/`
Status sahifa	`https://status.armenu.uz`
Runbook	Bu hujjat §11
Payme support	Shartnomadagi raqam
OFD support	Shartnomadagi raqam
Hosting support	Panel + tiket

13.5 Post-mortem shabloni

> ⚡ **Har P0/P1 uchun MAJBURIY**, 48 soat ichida.**Aybdorsiz (blameless)** — maqsad odamni emas, tizimni tuzatish.

```
# Post-mortem: <qisqa sarlavha>

**Sana:** 2026-09-08
**Daraja:** P1
**Davomiyligi:** 14:23 – 15:07 (44 daqiqa)
**Mualliflar:** <ismlar>
**Holat:** Yopilgan / Harakat davom etyapti

---

## 1. Qisqacha (1 abzas)
Nima bo'ldi, kim ta'sirlandi, qancha davom etdi.

## 2. Ta'sir
| Ko'rsatkich | Qiymat |
|---|---|
| Ta'sirlangan restoranlar | 42 (jami 168 dan) |
| Yo'qolgan buyurtmalar | 0 (navbatda saqlandi) |
| Moliyaviy ta'sir | ~$0 / N so'm |
| SLA buzilishi | 44 daq (oylik byudjet: 43 daq) – 🔴 buzildi |
| Mijoz shikoyatlari | 3 |

## 3. Vaqt jadvali (UTC+5)
| Vaqt | Hodisa |
|---|---|
| 14:23 | Alert: `HighErrorRate` (5xx 8%) |
| 14:25 | On-call javob berdi, Grafana ochildi |
| 14:31 | Sabab aniqlandi: DB conn pool 100% |
| 14:38 | `idle in transaction` ulanishlar terminate qilindi |
| 14:41 | Xato darajasi 1% ga tushdi |
| 14:52 | Worker'lar restart qilindi (asl sabab) |
| 15:07 | Barcha metrikalar normal, incident yopildi |

## 4. Asl sabab (root cause)
Texnik tushuntirish. "5 nega" usuli:
1. Nega sayt 5xx berdi? → DB ulanishlari tugadi
2. Nega tugadi? → worker'da 40 ta `idle in transaction`
3. Nega idle qoldi? → `TenantAwareThread` da `finally` bloki yo'q edi
4. Nega yo'q edi? → yangi worker turi qo'shilganda ko'chirilmadi
5. Nega ko'chirilmadi? → kod ko'rigida bu naqsh tekshirilmadi

## 5. Nima yaxshi ishladi ✅
- Alert 2 daqiqada ishga tushdi
- Runbook R3 to'g'ri qadamlarni berdi
- Buyurtmalar navbatda saqlandi, 0 ta yo'qolmadi
- Fiskal cheklar keyin muvaffaqiyatli yuborildi

## 6. Nima yomon ketdi ❌
- Sabab topish 8 daqiqa oldi (dashboard'da conn pool paneli yo'q edi)
- Zaxira on-call javob bermadi (telefon jim rejimda)

## 7. Harakat rejasi
| # | Harakat | Kim | Muddat | Holat |
|---|---|---|---|---|
| 1 | `TenantAwareThread` ga `finally` qo'shish | Backend | 09-09 | ✅ |
| 2 | Grafana'ga "DB conn pool" paneli | DevOps | 09-10 | ⏳ |
| 3 | Alert: `idle in transaction > 10` | DevOps | 09-10 | ⏳ |
| 4 | PgBouncer o'rnatish | DevOps | 09-20 | ☐ |
| 5 | Kod ko'rigi ro'yxatiga "thread finally" bandi | Backend | 09-12 | ☐ |
| 6 | On-call telefon sozlamalari yo'riqnomasi | CTO | 09-09 | ✅ |

## 8. Saboqlar
Boshqa joyda ham shu muammo bormi? Qanday umumlashtiriladi?
```

14. XULOSA

14.1 Bu qismda nima hal qilindi

#	Mavzu	Natija
1	Muhitlar	dev / staging / prod aniq ajratilgan, staging anonimlashtirilgan
2	Infratuzilma	S0 ($40) → S3 ($700) → S5 ($4 400) yo'li hisoblangan
3	⛔ **Izolyatsiya**	14 o'lchamda ajratish + sudoers + pg_hba + Redis ACL + avtomatik tekshiruv skripti
4	systemd	Socket activation → **zero-downtime**; 9 unit, to'liq konfig, security sandbox
5	nginx	To'liq konfig: 9 rate limit zonasi, AR CORS+MIME, maintenance, TLS 1.3
6	CI/CD	6 darvoza (tenant izolyatsiya MAJBURIY) → backup → deploy → 12 smoke → avto-rollback
7	Deploy	Backward-compatible migratsiya qoidalari, kesh strategiyasi, rollback mezonlari
8	Monitoring	40+ metrika, 14 panel, 28 alert qoidasi (P0-P4)
9	Log	JSON, PII maskalash, moliya logi 5 yil
10	Backup	Kunlik + WAL + **avtomatik** `pg_restore` **verify** + 3 ssenariyli tiklash
11	Runbook	10 tipik holat: belgi → diagnostika → tuzatish → eskalatsiya
12	Ritm	Kunlik 12 · haftalik 10 · oylik 10 · choraklik 10 tekshiruv
13	On-call	4 bosqichli eskalatsiya, aybdorsiz post-mortem shabloni

14.2 Operatsion tayyorlik mezoni (v1 ishga tushirishdan oldin)

- [ ] `izolyatsiya_tekshir.sh` ✅ qaytaradi (mebelcity buzilmagan)
- [ ] 9 ta systemd unit `active`, socket activation ishlaydi
- [ ] nginx `-t` toza, TLS A+ (SSL Labs)
- [ ] 6 CI darvozasi yashil, tenant izolyatsiya testi 100%
- [ ] Deploy → smoke → rollback bir marta **mashq qilingan**
- [ ] Grafana 14 panel to'ldirilgan, 28 alert Telegram'ga keladi
- [ ] Backup 3 kun ketma-ket muvaffaqiyatli + verify ✅
- [ ] Staging'da to'liq tiklash mashqi bajarilgan (RTO o'lchangan)
- [ ] On-call navbat va telefon raqamlari tasdiqlangan
- [ ] Runbook 10 holat o'qilgan va tushunilgan

14.3 Arxitektura hujjati — to'liq to'plam

Qism	Fayl	Mavzu
0	`00_UMUMIY_KORINISH.md`	Bir qarashda, C4 kontekst, 8 sifat talabi
1	`01_PRINTSIPLAR_VA_QARORLAR.md`	12 printsip, 15 ADR
2	`02_KOMPONENTLAR.md`	Har komponent: vazifa, texnologiya, chegara
3	`03_MALUMOT_ARXITEKTURASI.md`	DB sxema, indeks, partition, kesh
4	`04_MASSHTABLASH.md`	S0→S5, 1 mln foydalanuvchi
5	`05_ISHONCHLILIK.md`	Fault-tolerance, self-healing, SLA
6	`06_XAVFSIZLIK.md`	Tahdid modeli, multi-tenant, to'lov
7	`07_INTEGRATSIYALAR.md`	AI/MCP, to'lov, fiskal, Telegram, CDN
8	**`08_DEPLOY_VA_OPERATSIYA.md`**	**Infra, CI/CD, monitoring, runbook ← bu fayl**

14.4 Yakuniy fikr

> ARMENU arxitekturasi **oddiy texnologiyalar bilan murakkab talablarni** hal qiladi: modulli monolit + systemd + nginx + PostgreSQL. Kubernetes yo'q, mikroservis yo'q — chunki 1-2 dasturchi uchun bular **muammo yechmaydi, muammo qo'shadi**.
>
> Chidamlilik **adapter naqshi** (16 tashqi xizmat, hech biri kodga bog'lanmagan), **circuit breaker** (kaskadli yiqilish yo'q), **graceful degradation** (har komponent uchun fallback zanjiri) va **self-healing** (15 mexanizm) orqali erishiladi.
>
> ⛔ Va eng muhimi: butun tizim `mebelcity_platform` bilan yonma-yon ishlaydi, lekin unga **hech qachon tegmaydi** — bu 14 o'lchamda ajratish, sudoers cheklovi, DB/Redis darajasidagi bloklash va har deploy'da avtomatik tekshiriladigan skript bilan kafolatlanadi.

Versiya 1.0 · 2026-09-08 · ARMENU loyihasi